Scrollutil

Programmer's guide

2026-10-25

Contents

index **3**
pagesman **3**
plainnotebook **10**
scrollableframe **21**
scrollarea **35**
scrollednotebook **43**
scrollsync **55**
scrollutil **60**
wheelEvent **110**
License **118**

index

The Scrolling Utilities Package Scrollutil 2.11

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Scrollutil Programmer's Guide
The scrollutil::scrollarea and scrollutil::getscrollarea Commands
The scrollutil::scrollsync and scrollutil::getscrollsync Commands
The scrollutil::scrollableframe and scrollutil::adaptBtn2EventHandling Commands
The scrollutil::scrollednotebook Command and the closetab Style Element
The scrollutil::plainnotebook Command
The scrollutil::pagesman Command
Commands Related to Mouse Wheel and <TouchpadScroll> Event Handling

Keywords: index

pagesman

The `scrollutil::pagesman` Command

For Scrollutil Version 2.11

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- Quick Reference
- Detailed Reference

Quick Reference

NAME

`scrollutil::pagesman` – Create and manipulate pagesman widgets

SYNOPSIS

```
scrollutil::pagesman pathName ?options?
```

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

WIDGET-SPECIFIC OPTIONS

```
  -forgetcommand command

  -height screenDistance

  -leavecommand command

  -takefocus 0|1|""|command

  -width screenDistance
```

PAGE CONFIGURATION OPTIONS

```
  -padding padding

  -sticky stickyness
```

WIDGET COMMAND

```
  pathName add window ?option value option value ...?

  pathName adjustsize

  pathName attrib ?name ?value name value ...??

  pathName cget option

  pathName configure ?option ?value option value ...??

  pathName forget pageIndex

  pathName hasattrib name

  pathName haspageattrib pageIndex name

  pathName index window

  pathName insert pageIndex window ?option value option value ...?

  pathName pageattrib pageIndex ?name ?value name value ...??

  pathName pagecget pageIndex option

  pathName pageconfigure pageIndex ?option ?value option value ...??

  pathName pages
```

```
pathName select ?pageIndex?

pathName size

pathName unsetattrib name

pathName unsetpageattrib pageIndex name

pathName window pageIndex
```

KEYWORDS

pagesman, widget

Detailed Reference

NAME

`scrollutil::pagesman` – Create and manipulate pagesman widgets

SYNOPSIS

```
scrollutil::pagesman pathName ?options?
```

DESCRIPTION

The `scrollutil::pagesman` command creates a new window named `pathName` and of the class `Pagesman`, and makes it into a **pagesman** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the pagesman widget such as its width, height, borderwidth, and relief. The `scrollutil::pagesman` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

A pagesman is a mega-widget that provides the basic functionality of a pages manager, meaning that it manages a list of windows, called **pages**, of which only one is visible at a time. By using it with plainnotebook widgets as pages, it is quite easy to write applications in which the user can descend from a plainnotebook to another one with a single mouse click and switch back in the same way to the original one. Everything needed for this navigation is provided by appropriate options and subcommands of the plainnotebook widget.

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

See the **options** manual entry for details on the standard Tk widget options. The `-background`, `-highlightbackground`, `-highlightcolor`, and `-highlightthickness` options are only supported by the Scrollutil package, but not by Scrollutil_tile. They have the same default values as the options of the same names for Tk frame widgets. The default values of the remaining standard options are:

```
-borderwidth 0 -cursor "" -relief flat
```

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-forgetcommand
Database Name:	forgetCommand
Database Class:	ForgetCommand

Specifies a command to be invoked when attempting to unmanage a window with the aid of the `forget` subcommand. If the option's value is a nonempty string then it is concatenated with the path name of the pagesman widget and that of the window to be unmanaged, and the resulting script, which must return a boolean, is evaluated in the global scope. If the return value of this script is false then the `forget` subcommand is aborted, i.e., the window is not unmanaged. The default is an empty string.

Command-Line Name:	-height
Database Name:	height
Database Class:	Height

If present and greater than zero, specifies the desired total height for the widget, including the border and highlight rectangle (if any) drawn around the outside of the widget, in any of the forms acceptable to `Tk_GetPixels`. Otherwise, the maximum requested height of all pages is used. The default is `0`.

Command-Line Name:	-leavecommand
Database Name:	leaveCommand
Database Class:	LeaveCommand

Specifies a command to be invoked when attempting to leave the currently selected window by selecting a different one with the aid of the `select` subcommand. If the option's value is a nonempty string then it is concatenated with the path name of the pagesman widget and that of the currently selected window, and the resulting script, which must return a boolean, is evaluated in the global scope. If the return value of this script is false then the operation of selecting a different window is aborted. The default is an empty string.

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the pagesman widget accepts the focus during keyboard

traversal. It is almost identical to the standard option of the same name (see the **options** manual entry for details). The only difference is that not the pagesman widget itself but the currently selected window (if any) will receive the focus during keyboard traversal with the standard keys (`Tab` and `Shift-Tab`). The default is an empty string.

Command-Line Name:	-width
Database Name:	width
Database Class:	Width

If present and greater than zero, specifies the desired total width for the widget, including the border and highlight rectangle (if any) drawn around the outside of the widget, in any of the forms acceptable to `Tk_GetPixels`. Otherwise, the maximum requested width of all pages is used. The default is `0`.

PAGE CONFIGURATION OPTIONS

The following options are currently supported by the `add`, `insert`, `pagecget`, and `pageconfigure` commands:

`-padding padding`

Specifies the amount of extra space to add between the widget and this page. `padding` must be a list of up to four nonnegative screen distances `{left top right bottom}`. If fewer than four elements are specified, `bottom` defaults to `top`, `right` defaults to `left`, and `top` defaults to `left`. A padding value specified as an empty list is equivalent to `0`, which is the default.

`-sticky stickyness`

Specifies how the page window will be positioned within the widget. `stickyness` must be a string containing zero or more of the characters `n`, `s`, `e`, or `w`. Each letter refers to a side (north, south, east, or west) that the page window will "stick" to, as per the `grid` geometry manager. The default stickyness is `nsew`.

WIDGET COMMAND

The `scrollutil::pagesman` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `args` determine the exact behavior of the command. In the subcommand descriptions below, `pageIndex` always denotes a numerical page index, which, with the exception of the `insert` subcommand, must be greater than or equal to zero and less than the number of pages. The following subcommands are possible for pagesman widgets:

`pathName add window ?option value option value ...?`

Adds a new page consisting of the specified window to the pagesman widget. See the PAGE CONFIGURATION OPTIONS section for the list of available options. If `window` is already managed by the pagesman widget, then the subcommand uses its optional arguments to update the options of the corresponding page.

`pathName adjustsize`

Sets the pagesman widget's `-height` option to the maximum requested height of all pages and its `-width` option to the maximum requested width of all pages.

REMARK: If all pages of the pagesman widget are plainnotebooks whose `-height` and `-width` options have their default value `0` then in many cases there is no need to precede an invocation of this subcommand for the pagesman widget by invocations of the subcommand of the same name for its pages, because each plainnotebook will automatically adapt its requested size to the maximum requested size of its panes. For example, a font size change in some panes of some of the pages might make it necessary to invoke this subcommand for the pagesman widget, and at the invocation time its pages will already have their requested sizes automatically adapted to the new font size.

`pathName attrib ?name ?value name value ...??`
Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the `hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName cget option`
Returns the current value of the configuration option given by `option`, which may have any of the values accepted by the `scrollutil::pagesman` command.

`pathName configure ?option ?value option value ...??`
Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `scrollutil::pagesman` command.

`pathName forget pageIndex`
Unmaps and unmanages the page identified by `pageIndex`. The subcommand returns `0` if it was aborted by the command specified as the value of the `-forgetcommand` option, and `1` otherwise.

`pathName hasattrib name`
Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName haspageattrib pageIndex name`
Returns `1` if the attribute `name` for the page given by `pageIndex` exists and `0` otherwise.

`pathName index window`
Returns the numerical index of the page identified by the specified window, which must have been added to the widget with the aid of the `add` or `insert` subcommand.

`pathName insert pageIndex window ?option value option value ...?`
Inserts a new page consisting of the specified window at the position given by `pageIndex`.

See the PAGE CONFIGURATION OPTIONS section for the list of available options. If `window` is already managed by the pagesman widget, then the subcommand moves it to the specified position and uses its optional arguments to update the options of the corresponding page. If `pageIndex` is less than or equal to zero then the window is prepended to the list of pages; if it is greater than or equal to the number of pages then the window is appended to the list of pages.

`pathName pageattrib pageIndex ?name ?value name value ...??`
Queries or modifies the attributes of the page given by `pageIndex`. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for the page. If `name` is specified with no `value`, then the command returns the value of the one named page attribute, or an empty string if no corresponding value exists (you can use the `haspageattrib` subcommand to distinguish this case from the one that the value of an *existing* page attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given page attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName pagecget pageIndex option`
Returns the current value of the page configuration option given by `option` for the page specified by `pageIndex`. `option` may have any of the values accepted by the `pageconfigure` command.

`pathName pageconfigure pageIndex ?option ?value option value ...??`
Queries or modifies the configuration options of the page given by `pageIndex`. If no `option` is specified, the command returns a list describing all of the available options for the page (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given page option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values described in the PAGE CONFIGURATION OPTIONS section.

`pathName pages`
Returns the list of all windows managed by the pagesman widget, in the order of their numerical page indices.

`pathName select ?pageIndex?`
If `pageIndex` is omitted, the subcommand returns the path name of the currently selected window. Otherwise it selects the specified page by mapping the corresponding window and unmapping the previously-selected one (if different). In this case, the return value is `0` if the operation of selecting a different window was aborted by the command specified as the value of the `-leavecommand` option, and `1` otherwise.

`pathName size`
Returns the number of pages, i.e., the number of windows managed by the pagesman widget.

`pathName unsetattrib name`
Unsets the attribute `name`. Returns an empty string.

`pathName unsetpageattrib pageIndex name`
Unsets the attribute `name` for the page given by `pageIndex`. Returns an empty string.

`pathName window pageIndex`

Returns the path name of the page window identified by `pageIndex`.

VIRTUAL EVENTS

After a new page is selected, the widget generates a `<<PagesmanPageChanged>>` virtual event.

BINDINGS

When a new pagesman widget is created, it has no default event bindings: pagesman widgets are not intended to be interactive.

KEYWORDS

pagesman, pages manager, widget

Keywords: pagesman

plainnotebook

The `scrollutil::plainnotebook` Command

For Scrollutil Version 2.11

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- Quick Reference
- Detailed Reference

Quick Reference

NAME

`scrollutil::plainnotebook` – Create and manipulate plainnotebook widgets

SYNOPSIS

```
package require scrollutil_tile
scrollutil::plainnotebook pathName ?options?
```

STANDARD OPTIONS

```
-class  -cursor  -style
```

WIDGET-SPECIFIC OPTIONS

```
-caller pageIndex

-closabletabs boolean

-forgetcommand command

-height screenDistance
```

```
  -leavecommand command

  -movabletabs boolean

  -side left|right

  -takefocus 0|1|""|command

  -title text

  -width screenDistance
```

WIDGET COMMAND

```
  pathName add window ?options?

  pathName addbutton pageIndex text ?image?

  pathName addlabel text ?image?

  pathName addseparator

  pathName adjustsize

  pathName attrib ?name ?value name value ...??

  pathName cget option

  pathName closablestate tabId ?boolean?

  pathName configure ?option ?value option value ...??

  pathName forget tabId

  pathName hasattrib name

  pathName hastabattrib tabId name

  pathName hide tabId

  pathName index tabId

  pathName insert pos window ?options?

  pathName insertbutton pos pageIndex text ?image?

  pathName insertlabel pos text ?image?
```

```
pathName insertseparator pos

pathName instate stateSpec ?script?

pathName see tabId

pathName select ?tabId?

pathName state ?stateSpec?

pathName style

pathName tab tabId ?option ?value option value ...??

pathName tabattrib tabId ?name ?value name value ...??

pathName tabpath tabId

pathName tabs

pathName titlepath

pathName unsetattrib name

pathName unsettabattrib tabId name
```

KEYWORDS

plainnotebook, widget, notebook, closebtn

Detailed Reference

NAME

`scrollutil::plainnotebook` – Create and manipulate plainnotebook widgets

SYNOPSIS

```
package require scrollutil_tile
scrollutil::plainnotebook pathName ?options?
```

Note that the `scrollutil::plainnotebook` command is provided by the Scrollutil_tile package, but not by Scrollutil.

DESCRIPTION

The `scrollutil::plainnotebook` command creates a new window named `pathName` and makes it into a **plainnotebook** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the plainnotebook widget such as its width, height, and side. The `scrollutil::plainnotebook` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s

parent must exist.

A plainnotebook is a mega-widget that extends a ttk::notebook having an arbitrary number of pages with invisible tabs by a ttk::frame to its left or right containing, among others, a scrollableframe whose content frame is the parent of a series of widgets that play the role of vertically laid-out notebook tabs. Currently not visible "tabs" can be brought into view by navigating with the mouse wheel, touchpad, or keyboard, and by scrolling with the vertical scrollbar of the scrollarea containing the scrollableframe. An individual "tab" can be made visible programmatically with the aid of the `see` subcommand. Unlike a ttk::notebook widget with vertically aligned tabs, which in most themes has a suboptimal look and whose `-height` option is quite often overridden by the total height of the tabs, the plainnotebook widget respects the value of its `-height` option, regardless of the space required by the "tabs".

In the rest of this document, the widgets that play the role of vertically laid-out notebook tabs, will be called **tabs** themselves.

The upper part of the frame to the left or right of the ttk::notebook component contains a toolbutton and a ttk::label widget, called **ascend button** and **title label**, respectively. Their visibility is controlled by the `-caller` and `-title` widget options. The ascend button displays an image whose shape resembles that of a < character.

The API of the plainnotebook widget is quite similar to that of ttk::notebook. Apart from a few additional, plainnotebook-specific subcommands and the missing `identify` subcommand, the Tcl command associated with a plainnotebook widget provides the same subcommands as the one associated with a ttk::notebook. Note that the API doesn't provide any command for retrieving the path name of the ttk::notebook widget contained in a plainnotebook, because there is no need to access that widget directly. Moreover, using its options or subcommands instead of those of the plainnotebook widget will more often than not result in an unpredictable behavior.

Notice also that only the tabs created with the `add` and `insert` subcommands are selectable. The ones created with the plainnotebook-specific subcommands `addbutton` and `insertbutton` are designed for descending to another plainnotebook widget, while the ones created via `addlabel`, `addseparator`, `insertlabel`, and `insertseparator` can be used as titles and delimiters of tab groups.

STANDARD OPTIONS

```
-class  -cursor  -style
```

See the **ttk_widget** manual entry for details on the standard options. The default value of the `-class` option is `Plainnotebook` and that of `-cursor` an empty string. The `-style` option, whose value defaults to `Plain.TNotebook`, refers to the style of the ttk::notebook widget contained in the plainnotebook. The "plain" look of this ttk::notebook widget is achieved by setting the layout of the style `Plain.TNotebook.Tab` to `null` and the `-mintabwidth`, `-tabmargins`, and `-padding` options of the style `Plain.TNotebook` to `0`.

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-caller
Database Name:	caller
Database Class:	Caller

The value of this option must be an integer. If it is nonnegative then the widget's parent must be a pagesman widget. In this case the value is viewed as the numerical index of an already existing page of the parent pagesman widget, and the plainnotebook's ascend button will select that page, which is assumed to be the "caller" of the current plainnotebook widget, meaning that the current plainnotebook was previously selected from within that page. For example, if the option's value is `0` then pressing and later releasing the left mouse button over the ascend button will ascend back to the pagesman page of index `0`, which is assumed to be the page from which the user previously descended to the current one. The default is `-1`.

Command-Line Name:	-closabletabs
Database Name:	closableTabs
Database Class:	ClosableTabs

Specifies a boolean value that determines whether the tabs created with the `add` and `insert` subcommands (only) will contain the `closebtn` element. This style element is automatically created by the Scrollutil_tile package and is quite similar to the `closetab` element. See the FURTHER BINDINGS section below for details on closing the tabs with the aid of the `closebtn` element. The default is `0`. This option can be overridden for individual tabs with the aid of the `closablestate` subcommand.

Command-Line Name:	-forgetcommand
Database Name:	forgetCommand
Database Class:	ForgetCommand

Specifies a command to be invoked when attempting to remove a tab and unmanage the associated window with the aid of the `forget` subcommand. If the option's value is a nonempty string then it is concatenated with the path name of the plainnotebook widget and that of the window to be unmanaged, and the resulting script, which must return a boolean, is evaluated in the global scope. If the return value of this script is false then the `forget` subcommand is aborted, i.e., the tab is not removed and the associated window is not unmanaged. The default is an empty string.

Command-Line Name:	-height

Database Name:	height
Database Class:	Height

If present and greater than zero, specifies the desired height of the pane area in any of the forms acceptable to `Tk_GetPixels`. Otherwise, the maximum requested height of all panes is used. The default is `0`.

Command-Line Name:	-leavecommand
Database Name:	leaveCommand
Database Class:	LeaveCommand

Specifies a command to be invoked when attempting to leave the currently selected window by selecting a different one. If the option's value is a nonempty string then it is concatenated with the path name of the plainnotebook widget and that of the currently selected window, and the resulting script, which must return a boolean, is evaluated in the global scope. If the return value of this script is false then the operation of selecting a different window is aborted. The default is an empty string.

Command-Line Name:	-movabletabs
Database Name:	movableTabs
Database Class:	MovableTabs

Specifies a boolean value that determines whether the tabs can be moved with the mouse. See the FURTHER BINDINGS section below for details. The default is `1`.

Command-Line Name:	-side
Database Name:	side
Database Class:	Side

Specifies on which side (`left` or `right`) of the widget to show the ttk::frame containing the tabs, the ascend button, and the title label. The default is `left`.

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the plainnotebook widget accepts the focus during keyboard traversal. It is almost identical to the standard option of the same name (see the **options**

manual entry for details). The only difference is that not the plainnotebook widget itself but the content frame of the scrollableframe contained in it will receive the focus during keyboard traversal with the standard keys (`Tab` and `Shift-Tab`). The default is `"ttk::takefocus"` (just like for ttk::notebook and several other Tk themed widgets).

Command-Line Name:	-title
Database Name:	title
Database Class:	Title

Specifies the text to be displayed in the plainnotebook's title label, which is a ttk::label widget of the style `PnbTitle.TLabel`. The font used for this style is derived from `TkDefaultFont` by increasing the latter's size by a factor of 1.2. To set a different font, use the `ttk::style` command or retrieve the path name of this widget with the `titlepath` subcommand and then invoke`... configure -font ...`.

Command-Line Name:	-width
Database Name:	width
Database Class:	Width

If present and greater than zero, specifies the desired width of the pane area in any of the forms acceptable to `Tk_GetPixels`. Otherwise, the maximum requested width of all panes is used. The default is `0`.

TAB OPTIONS

See the **ttk_notebook** manual entry.

TAB IDENTIFIERS

See the **ttk_notebook** manual entry. Note that in positional specifications of the form `"@x,y"`, the coordinates are expected to be relative to the plainnotebook window.

WIDGET COMMAND

The `scrollutil::plainnotebook` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `args` determine the exact behavior of the command. The following commands are possible for plainnotebook widgets:

`pathName add window ?options?`

See the **ttk_notebook** manual entry. As mentioned there, if `window` is currently managed by the notebook but hidden, the subcommand restores it to its previous position. This holds equally true for the non-selectable (dummy) windows associated with the tabs created with the `addbutton`, `addlabel`, `addseparator`, `insertbutton`, `insertlabel`, and `insertseparator` subcommands. If `window` is not yet managed by the notebook then the corresponding tab will be a ttk::radiobutton widget of a toolbutton style (`Page.Toolbutton`

or `ClosablePage.Toolbutton`, depending on the value of the `-closabletabs` option). The return value is the path name of the widget used for the tab corresponding to `window` (a ttk::radiobutton or ttk::button, both using a toolbutton style, or a ttk::label, or a ttk::separator).

`pathName addbutton pageIndex text ?image?`

Adds a new tab implemented as a ttk::button widget of a toolbutton style (`Desc.Toolbutton`) that displays, besides the specified `text` and optional `image`, the `descend` element, whose shape resembles that of a > character. This style element is automatically created by the Scrollutil_tile package. Its size depends on Tk's scaling factor, while its foreground color is theme-specific and is updated whenever the theme changes. The return value is the path name of the above-mentioned ttk::button widget.

For this subcommand to work, the widget's parent must be a pagesman widget. `pageIndex`, which must be a nonnegative integer, specifies the numerical index of a page within the pagesman widget that will be selected when invoking the toolbutton. It is not expected that at the time this subcommand is invoked, the page identified by `pageIndex` already exists; it may be created and added to the pagesman widget later.

This subcommand, together with the `-caller` option, makes it easy to to write applications in which the user can descend from a plainnotebook to another one within a common pagesman parent with a single mouse click and switch back in the same way to the original one. For example, if the two plainnotebook widgets have the page indices `0` and `1` then all you need is to pass the page index `1` to the first plainnotebook's `addbutton` subcommand and set the `-caller` option of the second one to the page index `0`.

`pathName addlabel text ?image?`

Adds a new tab implemented as a ttk::label widget of the style `PnbLabel.TLabel` that displays the specified `text` and optional `image`, and is typically used as a title of a group of tabs. The font used for this style is derived from `TkDefaultFont` by increasing the latter's size by a factor of 1.2 and can be changed with the `ttk::style` command. The return value is the path name of the above-mentioned ttk::label widget; you can use it, e.g., to override the style's font for this particular label via`... configure -font ...`.

`pathName addseparator`

Adds a new tab implemented as a horizontal ttk::separator widget, which is typically used as a delimiter of a group of tabs. The return value is the path name of the above-mentioned ttk::separator widget.

`pathName adjustsize`

Sets the plainnotebook's `-height` option to the maximum requested height of all panes and its `-width` option to the maximum requested width of all panes.

REMARK: If the plainnotebook's `-height` and `-width` options have their default value `0` then in many cases there is no need for invocations of this subcommand, since the widget will automatically attempt to adapt its size to the maximum requested size of its panes.

`pathName attrib ?name ?value name value ...??`

Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the

one named attribute, or an empty string if no corresponding value exists (you can use the `hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName cget option`
Returns the current value of the configuration option given by `option`, which may have any of the values accepted by the `scrollutil::plainnotebook` command.

`pathName closablestate tabId ?boolean?`
Sets or queries the presence of the `closebtn` element in the tab specified by `tabId`. The optional argument is only supported for tabs created via `add` or `insert`. This subcommand overrides the `-closabletabs` option for the specified tab.

`pathName configure ?option ?value option value ...??`
Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `scrollutil::plainnotebook` command.

`pathName forget tabId`
See the **ttk_notebook** manual entry.

`pathName hasattrib name`
Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName hastabattrib tabId name`
Returns `1` if the attribute `name` for the tab given by `tabId` exists and `0` otherwise.

`pathName hide tabId`
See the **ttk_notebook** manual entry.

`pathName index tabId`
See the **ttk_notebook** manual entry.

`pathName insert pos window ?options?`
See the **ttk_notebook** manual entry. As mentioned there, if `window` is already managed by the notebook, the subcommand moves it to the specified position. This holds equally true for the non-selectable (dummy) windows associated with the tabs created with the `addbutton`, `addlabel`, `addseparator`, `insertbutton`, `insertlabel`, and `insertseparator` subcommands. If `window` is not yet managed by the notebook then the corresponding tab will be a ttk::radiobutton widget of a toolbutton style (`Page.Toolbutton` or `ClosablePage.Toolbutton`, depending on the value of the `-closabletabs` option). The return value is the path name of the widget used for the tab corresponding to `window` (a ttk::radiobutton or ttk::button, both using a toolbutton style, or a ttk::label, or a ttk::separator).

`pathName insertbutton pos pageIndex text ?image?`
Inserts a new tab implemented as a ttk::button widget of a toolbutton style (`Desc.Toolbutton`) that displays, besides the specified `text` and optional `image`, the

`descend` element, at the specified position. The details are quite similar to the ones regarding the `addbutton` subcommand.

`pathName insertlabel pos text ?image?`
Inserts a new tab implemented as a ttk::label widget of the style `PnbLabel.TLabel` that displays the specified `text` and optional `image`, at the specified position. The details are quite similar to the ones regarding the `addlabel` subcommand.

`pathName insertseparator pos`
Inserts a new tab implemented as a horizontal ttk::separator widget, at the specified position. The details are quite similar to the ones regarding the `addseparator` subcommand.

`pathName instate stateSpec ?script?`
See the **ttk_widget** manual entry.

`pathName see tabId`
This subcommand adjusts the view in the plainnotebook's window so that the tab given by `tabId` becomes visible in it. Note that when a tab gets selected, this command is automatically invoked for it, in order to make that tab visible.

`pathName select ?tabId?`
See the **ttk_notebook** manual entry.

`pathName state ?stateSpec?`
See the **ttk_widget** manual entry.

`pathName style`
Returns the value of the `-style` configuration option (i.e., the style used by the ttk::notebook widget contained in the plainnotebook). For Tk themed widgets this subcommand was introduced in Tk 8.7a4, but the plainnotebook widget provides it for all supported Tk versions.

`pathName tab tabId ?option ?value option value ...??`
See the **ttk_notebook** manual entry.

`pathName tabattrib tabId ?name ?value name value ...??`
Queries or modifies the attributes of the tab given by `tabId`. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for the tab. If `name` is specified with no `value`, then the command returns the value of the one named tab attribute, or an empty string if no corresponding value exists (you can use the `hastabattrib` subcommand to distinguish this case from the one that the value of an *existing* tab attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given tab attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName tabpath tabId`
Returns the path name of the widget used for the specified tab. This widget can be a ttk::radiobutton or ttk::button, both using a toolbutton style, or a ttk::label, or a ttk::separator.

`pathName tabs`
See the **ttk_notebook** manual entry.

`pathName titlepath`
Returns the path name of the plainnotebook's title label, which is a ttk::label widget of the style `PnbTitle.TLabel`.

`pathName unsetattrib name`
Unsets the attribute `name`. Returns an empty string.

`pathName unsettabattrib tabId name`
Unsets the attribute `name` for the tab given by `tabId`. Returns an empty string.

KEYBOARD TRAVERSAL
See the **ttk_notebook** manual entry.

VIRTUAL EVENTS
As described in the **ttk_notebook** manual entry, after a new tab of a ttk::notebook is selected (programmatically or interactively), the widget generates a `<<NotebookTabChanged>>` virtual event. This holds true for plainnotebook widgets, too.

The Scrollutil_tile package defines the virtual event `<<Button3>>` as `<Button-3>` for all windowing systems and additionally as `<Control-Button-1>` for Mac OS X/11+. If this event occurs over a tab of a plainnotebook whose `disabled` widget state flag is not set, and the Tk version is 8.5a2 or later, then the widget generates a `<<MenuItemsRequested>>` virtual event, by invoking `event generate`with the `-data` option set to a list consisting of the path name of an empty pop-up menu and the numerical index of the tab in question. (The support for this option was introduced in Tk 8.5a2.) If the application creates a binding for this virtual event, the binding script can access the user data as the value of the `%d` field, with the goal to populate the menu with application-specific entries. Note that the pop-up menu is posted with a delay of 100 ms, to make sure that it will appear readily populated by the application.

The following example provides a pop-up menu item for closing the tabs of a plainnotebook widget `$nb`:

```
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }

proc populateMenu {nb data} {
    foreach {menu tabIdx} $data {}
    $menu add command -label "Close Tab" -command [list $nb forget
    $tabIdx]
}
```

See the next section for the description of the virtual events `<<NotebookTabMoved>>` and `<<NotebookTabClosed>>`.

FURTHER BINDINGS
The Scrollutil_tile package creates the following additional bindings for the plainnotebook widgets:

1. **Navigation between the selectable tabs of a plainnotebook widget via the mouse wheel or touchpad:**Recall that the selectable tabs are the ones created with the `add` and `insert` subcommands. A mouse wheel tick or a two-finger gesture on the touchpad selects the selectable tab of state `normal` cyclically following/preceding the currently selected one. (Note that the same effect can also be achieved with the keys `Down`, `Up`, `Control-Tab`, and `Control-Shift-Tab`.) On the windowing system `aqua`, if the `Option` key is down while the mouse wheel is being rotated then a wheel tick selects the 10'th selectable tab of state `normal` cyclically following/preceding the currently selected one (modulo the number

of selectable tabs). The same holds true on the windowing systems `x11` and `win32` if the Tk version is 8.7a4 or later and the mouse wheel is rotated with the `Alt` key down.

2. **Moving the tabs of a plainnotebook widget with the mouse**: If mouse button 1 is pressed over a tab but outside the `closebtn` element (if any) and then dragged outside that tab then, provided that the value of the widget's `-movabletabs` option is true, the mouse cursor takes on the shape of an up/down arrow, indicating that the tab in question is being moved to another position. This operation ends when mouse button 1 is released, and can be canceled by pressing the `Escape` key. In both cases, the mouse cursor is reset to its original value, specified by the `-cursor` configuration option. After releasing mouse button 1, the source tab is moved to the position previously pointed to by the arrow (provided that the action was not canceled via `Escape`). If the Tk version is 8.5a2 or later then after moving the tab to its new position, the widget generates a `<<NotebookTabMoved>>` virtual event, by invoking `event generate`with the `-data` option set to a list consisting of the path name of the widget contained in the source pane and the numerical index of the target tab position. (The support for this option was introduced in Tk 8.5a2.)
3. **Closing a tab of a plainnotebook widget with the aid of the closebtn element**: If mouse button 1 is pressed over the `closebtn` element (if any) of a tab and later released over the same element then, provided that the value of the tab's `-state` option is `normal`, the tab is closed by means of the widget's `forget` subcommand. Recall that this operation can get canceled by the command specified as the value of the `-forgetcommand` option; if this was not the case and the Tk version is 8.5a2 or later then, after closing the tab, the widget generates a `<<NotebookTabClosed>>` virtual event, by invoking `event generate` with the `-data` option set to the path name of the widget contained in that tab's pane.

   If the widget's `disabled` state flag is set then none of the above actions occur: the tabs are completely insensitive.

KEYWORDS

plainnotebook, widget, notebook, closebtn

Keywords: plainnotebook

scrollableframe

The `scrollutil::scrollableframe` and `scrollutil::adaptBtn2EventHandling` Commands

For Scrollutil Version 2.11

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The scrollutil::scrollableframe Command – Quick Reference
- The scrollutil::scrollableframe Command – Detailed Reference
- The scrollutil::adaptBtn2EventHandling Command

- Comparison with BWidget ScrollableFrame and iwidgets::scrolledframe

---

The `scrollutil::scrollableframe` Command – Quick Reference

NAME

`scrollutil::scrollableframe` – Create and manipulate scrollableframe widgets

SYNOPSIS

```
scrollutil::scrollableframe pathName ?options?
```

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

WIDGET-SPECIFIC OPTIONS

```
-contentheight screenDistance

-contentwidth screenDistance

-fitcontentheight boolean

-fitcontentwidth boolean

-height screenDistance

-takefocus 0|1|""|command

-width screenDistance

-xscrollincrement screenDistance

-yscrollincrement screenDistance
```

WIDGET COMMAND

```
pathName attrib ?name ?value name value ...??

pathName autofillx ?boolean?

pathName autofilly ?boolean?

pathName autosize ?w|h|wh?

pathName cget option

pathName configure ?option ?value option value ...??

pathName contentframe
```

```
pathName hasattrib name

pathName preparescan

pathName preparescroll

pathName scan option args

pathName scan mark x y

pathName scan dragto x y ?gain?
pathName see widget ?nw|ne|sw|se?

pathName seerect x1 y1 x2 y2 ?nw|ne|sw|se?

pathName unsetattrib name

pathName xview ?args?

pathName xview

pathName xview units

pathName xview moveto fraction

pathName xview scroll number units|pages
pathName yview ?args?

pathName yview

pathName yview units

pathName yview moveto fraction

pathName yview scroll number units|pages
```

KEYWORDS

scrollableframe, widget, frame, scrollable

The `scrollutil::scrollableframe` Command – Detailed Reference

NAME

`scrollutil::scrollableframe` – Create and manipulate scrollableframe widgets

SYNOPSIS

```
scrollutil::scrollableframe pathName ?options?
```

DESCRIPTION

The `scrollutil::scrollableframe` command creates a new window named `pathName` and of the class `Scrollableframe`, and makes it into a **scrollableframe** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the scrollableframe widget such as its width, height, and scrolling increments. The `scrollutil::scrollableframe` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

A scrollableframe is a mega-widget containing a **content frame**, whose dimensions may be larger than those of the widget itself. Arbitrary regions of this frame can be brought into view by scrolling, being that the scrollableframe widget supports the `-xscrollcommand` and `-yscrollcommand` configuration options and the associated Tcl command has the `xview` and `yview` subcommands. The content frame may contain any number of arbitrary widgets. In other words, the `scrollutil::scrollableframe` command implements a **scrollable widget container**.

In the Scrollutil package both the scrollableframe widget and its content frame are implemented as Tk core frames, while in Scrollutil_tile they are ttk::frame widgets. The content frame can be accessed by means of the `contentframe` subcommand. Individual widgets contained in it can be made visible in the scrollableframe window with the aid of the `see` subcommand. In addition to scrolling, scrollableframe widgets also support scanning, with the aid of appropriate mouse event bindings that invoke the `scan` subcommand.

A scrollableframe widget is typically created within a scrollarea, like in the following example:

```
set sa [scrollutil::scrollarea ...]
set sf [scrollutil::scrollableframe $sa.sf ...]
$sa setwidget $sf

set cf [$sf contentframe]
<populate the content frame>

$sf autosize w
update idletasks
$sf configure -height ... -yscrollincrement ...

pack $sa -expand yes -fill both
```

REMARK: After creating the scrollableframe widget, the `scrollutil::scrollableframe` command automatically registers it for scrolling with the mouse wheel or touchpad, by passing its path name to the `scrollutil::enableScrollingByWheel` command.

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

See the **options** manual entry for details on the standard Tk widget options. The `-background`, `-highlightbackground`, `-highlightcolor`, and `-highlightthickness` options are only supported by the Scrollutil package, but not by Scrollutil_tile. They have the same default values as the options of the same names for Tk frame widgets. The default values of the remaining standard options are:

```
-borderwidth 0 -cursor "" -relief flat -xscrollcommand ""
    -yscrollcommand ""
```

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-contentheight
Database Name:	contentHeight
Database Class:	ContentHeight

Specifies the desired height for the content frame in any of the forms acceptable to `Tk_GetPixels`. If the option's value is less than or equal to zero then the content frame's height is made just large enough to hold all its children. This option is only relevant if the value of the `-fitcontentheight` option is false; otherwise the content frame will have the same height as the scrollableframe window (excluding the latter's border and highlight rectangle, if any), regardless of the value of the `-contentheight` option. The default is `0`, which is suitable for the vast majority of applications.

Command-Line Name:	-contentwidth
Database Name:	contentWidth
Database Class:	ContentWidth

Specifies the desired width for the content frame in any of the forms acceptable to `Tk_GetPixels`. If the option's value is less than or equal to zero then the content frame's width is made just large enough to hold all its children. This option is only relevant if the value of the `-fitcontentwidth` option is false; otherwise the content frame will have the same width as the scrollableframe window (excluding the latter's border and highlight rectangle, if any), regardless of the value of the `-contentwidth` option. The default is `0`, which is suitable for the vast majority of applications.

Command-Line Name:	-fitcontentheight
Database Name:	fitContentHeight

Database Class:	FitContentHeight

Specifies a boolean value indicating whether or not the content frame should have the same height as the scrollableframe window (excluding the latter's border and highlight rectangle, if any). If true then the content frame's height will be kept in sync with that of the widget and the value of the `-contentheight` option will be ignored. The default is `0`.

Command-Line Name:	-fitcontentwidth
Database Name:	fitContentWidth
Database Class:	FitContentWidth

Specifies a boolean value indicating whether or not the content frame should have the same width as the scrollableframe window (excluding the latter's border and highlight rectangle, if any). If true then the content frame's width will be kept in sync with that of the widget and the value of the `-contentwidth` option will be ignored. The default is `0`.

Command-Line Name:	-height
Database Name:	height
Database Class:	Height

Specifies the desired height for the scrollableframe widget in any of the forms acceptable to `Tk_GetPixels`. Note that this sets the *inner* height, excluding the border and highlight rectangle (if any) drawn around the outside of the widget. The default is `7c`, which should be overridden with a suitable application-specific value.

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the scrollableframe widget accepts the focus during keyboard traversal. It is almost identical to the standard option of the same name (see the **options** manual entry for details). The only difference is that not the scrollableframe widget itself but its content frame will receive the focus during keyboard traversal with the standard keys (`Tab` and `Shift-Tab`). The default is `0`, being that a scrollableframe widget is esentially a frame used as a view for its content frame component.

Command-Line Name:	-width
Database Name:	width

Database Class:	Width

Specifies the desired width for the scrollableframe widget in any of the forms acceptable to `Tk_GetPixels`. Note that this sets the *inner* width, excluding the border and highlight rectangle (if any) drawn around the outside of the widget. The default is `10c`, which should be overridden with a suitable application-specific value.

Command-Line Name:	-xscrollincrement
Database Name:	xScrollIncrement
Database Class:	ScrollIncrement

Specifies an increment for horizontal scrolling, in any of the forms acceptable to `Tk_GetPixels`. If the value of this option is greater than zero then the horizontal view in the window will be constrained so that the x coordinate within the content frame at the left edge of the window is always an even multiple of `xScrollIncrement`; furthermore, the units for horizontal scrolling (see the `xview` subcommad) will also be `xScrollIncrement`. If the value is less than or equal to zero then the horizontal scrolling is unconstrained. The default is `0`.

Command-Line Name:	-yscrollincrement
Database Name:	yScrollIncrement
Database Class:	ScrollIncrement

Specifies an increment for vertical scrolling, in any of the forms acceptable to `Tk_GetPixels`. If the value of this option is greater than zero then the vertical view in the window will be constrained so that the y coordinate within the content frame at the top edge of the window is always an even multiple of `yScrollIncrement`; furthermore, the units for vertical scrolling (see the `yview` subcommad) will also be `yScrollIncrement`. If the value is less than or equal to zero then the vertical scrolling is unconstrained. The default is `0`.

WIDGET COMMAND

The `scrollutil::scrollableframe` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `args` determine the exact behavior of the command. The following commands are possible for scrollableframe widgets:

`pathName attrib ?name ?value name value ...??`

Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the

`hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName autofillx ?boolean?`
Controls or queries whether the content frame will fill the widget's entire inner width. If the optional boolean argument is true then the command sets the `-fitcontentwidth` option to true if the content frame's requested width is less than the scrollableframe's inner width and to false otherwise. Moreover, the command makes sure that this action will be repeated whenever the content frame's requested width or the scrollableframe's inner width changes. If the optional argument is false then the command clears the flag that controls this dynamic action. In both cases, the return value is an empty string. Without the optional argument the command returns the current value of the above-mentioned flag.

`pathName autofilly ?boolean?`
Controls or queries whether the content frame will fill the widget's entire inner height. If the optional boolean argument is true then the command sets the `-fitcontentheight` option to true if the content frame's requested height is less than the scrollableframe's inner height and to false otherwise. Moreover, the command makes sure that this action will be repeated whenever the content frame's requested height or the scrollableframe's inner height changes. If the optional argument is false then the command clears the flag that controls this dynamic action. In both cases, the return value is an empty string. Without the optional argument the command returns the current value of the above-mentioned flag.

`pathName autosize ?dimensions?`
This command sets the `-width` and/or `-height` option of the scrollableframe to the requested width and/or height of its content frame, according to the optional `dimensions` argument, whose possible values are `w` (width), `h` (height), and `wh` (width and height). The default is an empty string, meaning that neither the width nor the height is *explicitly* requested to be set. Note that if the value of the `-fitcontentwidth` option is true then the command sets the widget's inner width to the requested width of its content frame, regardless of whether this was *explicitly* specified or not. Similarly, if the value of the `-fitcontentheight` option is true then the command sets the widget's inner height to the requested height of its content frame, even if this was not *explicitly* specified.

REMARK: Setting the widget's inner width and/or height on behalf of this subcommand is executed as an `"after 100"`callback, to make sure that by the time the widget's width and/or height are set, the geometry management calculations in its content frame are finished.

`pathName cget option`
Returns the current value of the configuration option given by `option`, which may have any of the values accepted by the `scrollutil::scrollableframe` command.

`pathName configure ?option ?value option value ...??`
Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one

or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `scrollutil::scrollableframe` command.

`pathName contentframe`
Returns the path name of the widget's content frame.

`pathName hasattrib name`
Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName preparescan`
Prepares the scrollableframe for scanning, by passing its descendants located in the same toplevel and having bindings for mouse button 2 events to the `scrollutil::adaptBtn2EventHandling` command.

REMARK: If the scrollableframe has many descendants then it is much safer to invoke this subcommand than to manually pass a lot of widgets to `scrollutil::adaptBtn2EventHandling`, because deciding which descendants to process can become tedious and error-prone.

`pathName preparescroll`
Prepares the scrollableframe for scrolling with the mouse wheel and touchpad, by passing its descendants located in the same toplevel and having bindings for mouse wheel or `<TouchpadScroll>` events to the `scrollutil::adaptWheelEventHandling` command.

REMARK: Invoking this subcommand is just a shortcut for passing `pathName` to the `scrollutil::prepareScrollingByWheel` command.

`pathName scan option args`
This command is used to implement scanning on scrollableframe widgets. It has two forms, depending on `option`:

`pathName scan mark x y`
Records `x` and `y` and the current view in the scrollableframe window; used in conjunction with later `scan dragto`commands. Typically this command is associated with a mouse button press in the widget and `x` and `y` are the coordinates of the mouse. It returns an empty string.

`pathName scan dragto x y ?gain?`
This command computes the difference between its `x` and `y` arguments (which are typically mouse coordinates) and the `x` and `y` arguments to the last `scan mark`command for the widget. It then adjusts the view by `gain` times the difference in coordinates, where `gain` defaults to `10`, except on Android, where the default is `2`, being that this value suits better the emulation of mouse button 2 events with one-finger gestures. This command is typically associated with mouse motion events in the widget, to produce the effect of dragging the content frame at high speed through the window. The return value is an empty string.

`pathName see widget ?corner?`
This command adjusts the view in the scrollableframe's window so that `widget` becomes visible in it. The specified widget must be a descendant of the content frame and must have the same toplevel. In addition, it must be managed by some geometry manager (such as `grid`, `pack`, `place`, `text`, or `canvas`). The optional `corner` argument specifies which corner of `widget` should become visible if it is too large to fit into the window. The possible

values are `nw` (north-west), `ne` (north-east), `sw` (south-west), and `se` (south-east). The default corner is `nw`.

REMARK: By using this subcommand you can make the keyboard navigation in the content frame more user-friendly, like in the following example, in which `$w` is a descendant of the content frame of the scrollableframe widget `$sf` (it is assumed that `$sf` doesn't contain `%` characters):

```
bind $w <<TraverseIn>> [list $sf see %W]
```

If the widget `$w` was embedded into a scrollarea via the latter's `setwidget` subcommand, and the scrollarea in turn is a descendant of the content frame, then it is more user-friendly to bring the scrollarea into view rather than just the widget:

```
bind $w <<TraverseIn>> [list seeScrollarea $sf %W]

proc seeScrollarea {sf w} { $sf see [scrollutil::getscrollarea $w] }
```

`pathName seerect x1 y1 x2 y2 ?corner?`

This command adjusts the view in the scrollableframe's window so that the rectangle given by `x1`, `y1`, `x2`, and `y2` becomes visible in it. These coordinates are relative to the content frame and specify two diagonally opposite corners of the rectangle (which will include its upper and left edges but not its lower or right edges). The optional `corner` argument specifies which corner of the rectangle should become visible if it is too large to fit into the window. The possible values are `nw` (north-west), `ne` (north-east), `sw` (south-west), and `se` (south-east). The default corner is `nw`.

`pathName unsetattrib name`

Unsets the attribute `name`. Returns an empty string.

`pathName xview ?args?`

This command is used to query and change the horizontal position of the information displayed in the widget's window. It can take any of the following forms:

`pathName xview`

Returns a list containing two elements. Each element is a real fraction between `0` and `1`; together they describe the horizontal span that is visible in the window. For example, if the first element is `0.2` and the second element is `0.6` then 20% of the content frame is off-screen to the left, the middle 40% is visible in the window, and 40% of the content frame is off-screen to the right. These are the same values passed to scrollbars via the `-xscrollcommand` option.

`pathName xview units`

Adjusts the view in the window so that the content frame position given by `units` is displayed at the left edge of the window. This argument specifies a number of units of the `xScrollIncrement` option if it is greater than zero, or of one-tenth the window's inner width otherwise.

`pathName xview moveto fraction`

Adjusts the view in the window so that `fraction` of the total width of the content frame is off-screen to the left. `fraction` must be a fraction between `0` and `1`.

`pathName xview scroll number what`

This command shifts the view in the window left or right according to `number` and `what`. `number` must be an integer or a float; if it is a float then it is converted to an integer, rounded away from `0`. `what` must be either `units` or `pages` or an abbreviation of one of these. If `what` is `units`, the view adjusts left or right in units of the `xScrollIncrement` option if it is greater than zero, or in units of one-tenth the window's inner width otherwise. If `what` is `pages` then the view adjusts in units of nine-tenths the window's inner width. If `number` is negative then information farther to the left becomes visible; if it is positive then information farther to the right becomes visible.

`pathName yview ?args?`
This command is used to query and change the vertical position of the information displayed in the widget's window. It can take any of the following forms:

`pathName yview`
Returns a list containing two elements. Each element is a real fraction between `0` and `1`; together they describe the vertical span that is visible in the window. For example, if the first element is `0.2` and the second element is `0.6` then 20% of the content frame is off-screen to the top, the middle 40% is visible in the window, and 40% of the content frame is off-screen to the bottom. These are the same values passed to scrollbars via the `-yscrollcommand` option.

`pathName yview units`
Adjusts the view in the window so that the content frame position given by `units` is displayed at the top edge of the window. This argument specifies a number of units of the `yScrollIncrement` option if it is greater than zero, or of one-tenth the window's inner height otherwise.

`pathName yview moveto fraction`
Adjusts the view in the window so that `fraction` of the total height of the content frame is off-screen to the top. `fraction` must be a fraction between `0` and `1`.

`pathName yview scroll number what`
This command shifts the view in the window up or down according to `number` and `what`. `number` must be an integer or a float; if it is a float then it is converted to an integer, rounded away from `0`. `what` must be either `units` or `pages` or an abbreviation of one of these. If `what` is `units`, the view adjusts up or down in units of the `yScrollIncrement` option if it is greater than zero, or in units of one-tenth the window's inner height otherwise. If `what` is `pages` then the view adjusts in units of nine-tenths the window's inner height. If `number` is negative then information farther to the top becomes visible; if it is positive then information farther to the bottom becomes visible.

BINDINGS

Mouse button 2 may be used for scanning. If it is pressed and dragged over a free area of the scrollableframe window, the content frame drags at high speed in the direction the mouse moves. The same happens if a descendant was clicked that either has no bindings for mouse button 2 events, or was passed to the `scrollutil::adaptBtn2EventHandling` command, and the focus is outside the window `[focusCheckWindow descendant]`.

REMARK: On Android, the support for scanning with mouse button 2 implies scan support via one-finger gestures, because fast wipes with one finger are translated by AndroWish to `<Button-2>`, `<B2-Motion>`, and `<ButtonRelease-2>` events.

KEYWORDS

scrollableframe, widget, frame, scrollable

The `scrollutil::adaptBtn2EventHandling` Command

NAME

`scrollutil::adaptBtn2EventHandling` – Adapt mouse button 2 event handling

SYNOPSIS

```
scrollutil::adaptBtn2EventHandling ?widget widget ...?
```

DESCRIPTION

For each `widget` argument, the command performs the following actions:

- If `widget` is the path name of a tablelist widget then it sets the latter's `-button2window` option to the path name of the containing toplevel window, provided that the Tablelist version is 7.6 or later (for earlier Tablelist versions the command silently ignores any tablelist widget passed to it as argument). As a result, a `<Button-2>`, `<B2-Motion>`, or `<ButtonRelease-2>` event sent to the tablelist's body or edit window (more precisely, a mouse button 2 event sent to any component of the tablelist having the binding tag `TablelistBody` or `TablelistEdit`) will be handled as follows:
- Otherwise it locates the (first) binding tag that has mouse button 2 event bindings and is different from both the path name of the containing toplevel window and `all`. If the search for this tag was successful then the command modifies the widget's list of binding tags by prepending the tag `Btn2EventRedir` and appending the tag `Btn2EventBreak` to this binding tag. As a result, a `<Button-2>`, `<B2-Motion>`, or `<ButtonRelease-2>` event sent to this widget will be handled as follows:
  - If the focus is inside the tablelist widget then the event will will trigger a scanning of the content of the tablelist or its edit window and no further processing of the event will take place.
  - If the focus is outside the tablelist widget then no scanning within the tablelist's body or edit window will happen. Instead, the event will be redirected to the containing toplevel window via `event generate`. This in turn will trigger a scanning of the content of the (innermost) scrollableframe that is an ancestor of `widget` and has the same toplevel (if there is such a scrollableframe widget).
  - If the focus is on or inside the window `[focusCheckWindow widget]`then the event will be handled by the binding script associated with this tag and no further processing of the event will take place.
  - If the focus is outside the window `[focusCheckWindow widget]`then the event will be redirected to the containing toplevel window via `event generate`rather than being handled by the binding script associated with the above-mentioned tag. This in turn will trigger a scanning of the content of the (innermost) scrollableframe that is an ancestor of `widget` and has the same toplevel (if there is such a scrollableframe widget).

REMARK 1: This command is designed to be invoked for widgets that have mouse button 2 event bindings and are descendants of a scrollableframe. The Tk and tile widgets having class bindings for mouse button 2 events area: Tk core entry, ttk::entry, ttk::combobox, listbox, panedwindow, Tk core scale, ttk::scale, Tk core spinbox, ttk::spinbox, Tk core

scrollbar, ttk::scrollbar, and text. Examples of widgets with binding tags other than their class names that have mouse button 2 event bindings are ctext and tablelist widgets. (In case of a ctext widget `w`, the binding tags mentioned in the description above refer to the widget's main text widget child `w.t` rather than to the widget itself.)

REMARK 2: As mentioned above, Tk core scrollbar and ttk::scrollbar widgets have class bindings for mouse button 2 events, hence this command should be invoked for them in case they are descendants of a scrollableframe widget. Since this task can become tedious, Scrollutil makes sure that if you pass a widget to this command and that widget is embedded into a scrollarea via the latter's `setwidget` subcommand, then this command will automatically be invoked for the scrollbars of that scrollarea, too.

REMARK 3: When handling a mouse button 2 event sent to a Tk core or tile scrollbar whose path name was passed to this command, if the focus is on or inside the associated widget then the event will be processed by the scrollbar rather than being redirected to the containing toplevel, just as if the focus were on the scrollbar itself.

KEYWORDS

mouse button 2 event, binding, event handling, scanning, scrollableframe, focus

---

Comparison with BWidget ScrollableFrame and iwidgets::scrolledframe

The scrollutil::scrollableframe widget was designed as a lightweight, theme-able, and full-featured replacement for BWidget ScrollableFrame and iwidgets::scrolledframe. The following table contains a comparison of the options and commands provided by the three widgets:

scrollutil::scrollableframe	BWidget ScrollableFrame	iwidgets::scrolledframe
-background	-background	-background
-borderwidth		-borderwidth
-cursor		-cursor
-contentheight	-areaheight	[pathName component canvas] \ itemconfigure frameTag -height ...
-contentwidth	-areawidth	[pathName component canvas] \ itemconfigure frameTag -width ...
-fitcontentheight	-constrainedheight	bind [pathName component canvas] <Configure> \ { %W itemconfigure frameTag -height %h }

scrollutil::scrollableframe	**BWidget ScrollableFrame**	**iwidgets::scrolledframe**
-fitcontentwidth	-constrainedwidth	bind [pathName component canvas] <Configure> \ { %W itemconfigure frameTag -width %w }
-height (sets the inner height, excluding the border and highlight rectangle, if any)	-height (inner height = total height)	-height (sets the total height, including the border and horizontal scrollbar, if any)
-highlightbackground		
-highlightcolor		
-highlightthickness		
-relief		-relief
-takefocus		
-width (sets the inner width, excluding the border and highlight rectangle, if any)	-width (inner width = total width)	-width (sets the total width, including the border and vertical scrollbar, if any)
-xscrollcommand	-xscrollcommand	
-xscrollincrement	-xscrollincrement	-xscrollincrement for [pathName component canvas]
-yscrollcommand	-yscrollcommand	
-yscrollincrement	-yscrollincrement	-yscrollincrement for [pathName component canvas]
attrib		
autosize		
autofillx		
autofilly		
cget	cget	cget
configure	configure	configure
contentframe	getframe	childsite
hasattrib		

scrollutil::scrollableframe	BWidget ScrollableFrame	iwidgets::scrolledframe
preparescan		
preparescroll		
scan (with mouse event bindings)		
see widget ?nw|ne|sw|se?	see widget ?top|bottom ?left|right??	
seerect x1 y1 x2 y2 ?nw|ne|sw|se?		
unsetattrib		
xview	xview	xview
yview	yview	yview
xview|yview units		
xview|yview moveto 0|1	xview|yview moveto 0|1	justify left|right|top|bottom (shortcut for xview|yview moveto 0|1)

REMARK 1: For the `see` command provided by the BWidget ScrollableFrame widget one cannot specify the horizontal side without the vertical one. More annoying is, however, that in the presence of a positive `-xscrollincrement` or `-yscrollincrement` value, widgets that are small enough to fit into the window are quite often just partially brought into view. In addition, the command works only for direct children of the ScrollableFrame widget's content frame.

REMARK 2: The iwidgets::scrolledframe widget has no equivalents of the scrollutil::scrollableframe options `-contentheight`, `-contentwidth`, `-fitcontentheight`, and `-fitcontentwidth`, but these can be emulated (using undocumented internal information!) as shown above. It provides a `justify` command, which is, however, just a shortcut for `xview|yview moveto 0|1`. What this widget is really missing, is a `see` command for making individual widgets visible in the window (which is important for user-friendly keyboard navigation).

Keywords: scrollableframe

scrollarea

The `scrollutil::scrollarea` and `scrollutil::getscrollarea` Commands

For Scrollutil Version 2.11

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The scrollutil::scrollarea Command – Quick Reference
- The scrollutil::scrollarea Command – Detailed Reference
- The scrollutil::getscrollarea Command

The `scrollutil::scrollarea` Command – Quick Reference

NAME

`scrollutil::scrollarea` – Create and manipulate scrollarea widgets

SYNOPSIS

```
scrollutil::scrollarea pathName ?options?
```

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

WIDGET-SPECIFIC OPTIONS

```
-autohidescrollbars boolean

-lockinterval milliseconds

-respectheader boolean

-respecttitlecolumns boolean

-setfocus boolean

-takefocus 0|1|""|command

-xscrollbarmode static|dynamic|none

-yscrollbarmode static|dynamic|none
```

WIDGET COMMAND

```
pathName attrib ?name ?value name value ...??

pathName cget option

pathName configure ?option ?value option value ...??

pathName hasattrib name

pathName setwidget widget
```

```
pathName unsetattrib name

pathName widget
```

KEYWORDS

scrollarea, widget, scrollbar

The `scrollutil::scrollarea` Command – Detailed Reference

NAME

`scrollutil::scrollarea` – Create and manipulate scrollarea widgets

SYNOPSIS

```
scrollutil::scrollarea pathName ?options?
```

DESCRIPTION

The `scrollutil::scrollarea` command creates a new window named `pathName` and of the class `Scrollarea`, and makes it into a **scrollarea** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the scrollarea such as its borderwidth, relief, and display mode to be used for the scrollbars. The `scrollutil::scrollarea` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

A scrollarea is a mega-widget consisting of a scrollable widget specified with the aid of the `setwidget` subcommand of the associated Tcl command as well as two scrollbars connected with that widget. These components are managed within the scrollarea using `grid`. The scrollbars, named `hsb` (with `-orient horizontal`) and `vsb` (with `-orient vertical`) are direct children of the scrollarea. The display mode of each scrollbar can be `static`, `dynamic`, or `none` (see the `-xscrollbarmode` and `-yscrollbarmode` configuration options). The `-takefocus` option of both scrollbars is set to `0`. In the Scrollutil_tile package the scrollbars are created as ttk::scrollbar widgets, except on Mac OS X/11+ when using a Tk release earlier than 8.6.10, where the native ttk::scrollbar widget of the `aqua` theme didn't yet look as expected.

If the widget embedded into the scrollarea via `setwidget` is a tablelist and Tablelist version 6.5 or later is being used then the scrollarea can also contain siblings of the tablelist widget above the vertical scrollbar and/or to the left of the horizontal one, causing the vertical scrollbar to be displayed below the tablelist's header and/or the horizontal scrollbar to appear to the right of the tablelist's title column area, depending on the values of the `-respectheader` and `-respecttitlecolumns` configuration options.

The following example shows the typical steps involved in creating a widget within a scrollarea:

```
set sa [scrollutil::scrollarea ...]
set lb [listbox $sa.lb ...]
$sa setwidget $lb

pack $sa -expand yes -fill both
```

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

See the **options** manual entry for details on the standard Tk widget options. The `-background`, `-highlightbackground`, `-highlightcolor`, and `-highlightthickness` options are only supported by the Scrollutil package, but not by Scrollutil_tile. They have the same default values as the options of the same names for Tk frame widgets. The default values of the remaining standard options are:

```
-borderwidth 1 -cursor "" -relief sunken
```

REMARK: When configuring the `-borderwidth` or `-relief` option, if as a result of this action the scrollarea has a positive `-borderwidth` value (e.g., the default `1`) and a `-relief` value other than `flat` (e.g., the default `sunken`), then the `-borderwidth` option of the widget embedded into the scrollarea via the `setwidget` subcommand of the associated Tcl command will automatically be set to `0`, provided that the embedded widget supports this option.

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-autohidescrollbars
Database Name:	autoHideScrollbars
Database Class:	AutoHideScrollbars

Specifies a boolean value indicating whether the scrollbars should be obscured if the focus is outside the toplevel window containing the scrollarea or the mouse pointer is outside the scrollarea widget. The default is `0`.

REMARK 1: The easiest way to override the default value for all scrollarea instances of your application is to insert the line

```
option add *Scrollarea.autoHideScrollbars 1
```

somewhere at the top of your script (before creating any scrollarea widgets).

REMARK 2: To obscure a scrollbar, Scrollutil just lays a frame over it, thus keeping the area occupied by it reserved (if it was previously mapped). This is different from unmapping a dynamic scrollbar when there is nothing to scroll in the widget embedded into the scrollarea. Obscuring or unobscuring a scrollbar doesn't change its mapped/unmapped state.

REMARK 3: Regardless of the focus and the value of this option, if the mouse pointer is inside

the scrollarea and the widget embedded into it via `setwidget` gets scrolled (e.g., with the aid of the mouse wheel or touchpad), then the scrollbars will be unobscured.

Command-Line Name:	-lockinterval
Database Name:	lockInterval
Database Class:	LockInterval

Specifies the time interval in milliseconds for which the scrollbars having the display mode `dynamic` (see the `-xscrollbarmode` and `-yscrollbarmode` options) will be protected from being unmapped after being mapped, in order to avoid any shimmering effects. This is also the time interval for which the dynamic scrollbars will mutually protect each other from being mapped after one of them got mapped. Without this locking mechanism, under some rare circumstances a dynamic scrollbar could get mapped and unmapped in an endless loop, thus giving rise to an annoying and often dangerous flickering effect. The same problem can arise due to a too small `-lockinterval` value. The default is `1`, which works as expected in the vast majority of cases. Should you experience any shimmering in one of your scrollarea widgets, just set this option for that scrollarea to a sufficiently large value (e.g., `100`). Note, however, that this value should not be greater than `300`, because, for implementation-specific reasons, a `-lockinterval` value greater than `300` can prevent a dynamic scrollbar that should be hidden from being unmapped, or one that should be shown from being mapped.

Command-Line Name:	-respectheader
Database Name:	respectHeader
Database Class:	RespectHeader

This option is only relevant if the widget embedded into the scrollarea with the aid of the `setwidget` subcommand of the associated Tcl command is a tablelist and the Tablelist version being used is 6.5 or later. Its value must be a boolean specifying whether the vertical scrollbar should appear below the tablelist widget's header, thus respecting the native look & feel on Mac OS X/11+ and on many modern Linux systems. The default is `1` on the windowing systems `aqua` and `x11`, and `0` on `win32`.

Command-Line Name:	-respecttitlecolumns
Database Name:	respectTitleColumns
Database Class:	RespectTitleColumns

This option is only relevant if the widget embedded into the scrollarea with the aid of the `setwidget` subcommand of the associated Tcl command is a tablelist and the Tablelist version being used is 6.5 or later. Its value must be a boolean specifying whether the horizontal scrollbar should start to the right of the tablelist widget's non-scrollable title column area if the value of the `-titlecolumns` tablelist option is positive. The default is `1`.

Command-Line Name:	-setfocus
Database Name:	setFocus
Database Class:	SetFocus

Specifies a boolean value that determines whether a click with mouse button 1 or 2 on one of the scrollbars should set the focus to the widget embedded into the scrollarea with the aid of the `setwidget` subcommand if that widget is not in disabled state. The default is `0`.

REMARK 1: If the option's value is `1` and the embedded widget is a text or ctext widget then a click with mouse button 1 or 2 on one of the scrollbars will set the focus to that widget regardless of the latter's state, provided that the windowing system is `win32` or the Tk version is at least 8.6.11 or 8.7a4. This is the same behavior as the one exhibited by the default bindings of these widgets for `<Button-1>` events. (In case of a ctext widget `w`, the focus is set to the widget's main text widget child `w.t` rather than to the widget itself.)

REMARK 2: If the option's value is `1` and the embedded widget is a ttk::treeview widget then a click with mouse button 1 or 2 on one of the scrollbars will set the focus to that widget regardless of the latter's state, on all windowing systems and for all Tk versions. Again, this is the same behavior as the one exhibited by the default bindings of this widget for `<Button-1>` events.

REMARK 3: The easiest way to override the default value for all scrollarea instances of your application is to insert the line

```
option add *Scrollarea.setFocus 1
```

somewhere at the top of your script (before creating any scrollarea widgets).

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the scrollarea accepts the focus during keyboard traversal. It is almost identical to the standard option of the same name (see the **options** manual entry for details). The only difference is that not the scrollarea itself but the widget embedded into it via the `setwidget` subcommand of the associated Tcl command will receive the focus during keyboard traversal with the standard keys (`Tab` and `Shift-Tab`). The default is `0`, being that a scrollarea is esentially a frame containing the above-mentioned widget and two scrollbars.

Command-Line Name:	-xscrollbarmode
Database Name:	xScrollbarMode
Database Class:	ScrollbarMode

Specifies the display mode to be used for the horizontal scrollbar. The allowed values are

`static`, `dynamic`, and `none`. In `static` mode the scrollbar is displayed at all times. In `dynamic` mode (which is the default) the scrollbar is mapped and unmapped as needed. The display mode `none` disables the scrollbar display.

Command-Line Name:	-yscrollbarmode
Database Name:	yScrollbarMode
Database Class:	ScrollbarMode

Specifies the display mode to be used for the vertical scrollbar. The allowed values are `static`, `dynamic`, and `none`. In `static` mode the scrollbar is displayed at all times. In `dynamic` mode (which is the default) the scrollbar is mapped and unmapped as needed. The display mode `none` disables the scrollbar display.

WIDGET COMMAND

The `scrollutil::scrollarea` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `args` determine the exact behavior of the command. The following commands are possible for scrollarea widgets:

`pathName attrib ?name ?value name value ...??`

Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the `hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName cget option`

Returns the current value of the configuration option given by `option`, which may have any of the values accepted by the `scrollutil::scrollarea` command.

`pathName configure ?option ?value option value ...??`

Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `scrollutil::scrollarea` command.

`pathName hasattrib name`

Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName setwidget widget`

The `widget` argument must be the path name of an existing widget or an empty string. In the first case, the command manages the widget to fill the top-left part of the scrollarea and connects it with the scrollbars by setting its `-xscrollcommand` and `-yscrollcommand` options to appropriate wrappers for the `set` command of the two scrollbars and setting the `-command` option of the scrollbars to `[list widget xview]`and `[list widget yview]`, respectively. If `widget` is an empty string then the widget passed to the most recent `setwidget` invocation (if any) is unmanaged and unconnected from the scrollbars. The return value is the argument passed to the previous successful invocation of this subcommand, or an empty string if there was no successful `setwidget` invocation before.

REMARK 1: If `widget` is nonempty and the value of the `-xscrollbarmode` option is different from `none` then `widget` must be a horizontally scrollable widget, meaning that it must support the `-xscrollcommand` configuration option and the associated Tcl command must have the `xview` subcommand. Similarly, if `widget` is nonempty and the value of the `-yscrollbarmode` option is different from `none` then `widget` must be a vertically scrollable widget, meaning that it must support the `-yscrollcommand` configuration option and the associated Tcl command must have the `yview` subcommand. Consequently, if `widget` is an entry or ttk::entry then this subcommand will only be successful if the `-yscrollbarmode` option was previously set to `none`.

REMARK 2: The widget identified by the `widget` argument must be a child of the scrollarea or of one of the latter's ascendants. This minor restriction is imposed by the `grid` geometry manager.

REMARK 3: When the widget whose path name was passed to `setwidget` gets destroyed, this subcommand is automatically invoked with an empty string as argument.

REMARK 4: This subcommand sets the `-highlightthickness` option of `widget` to `0` if `widget` supports this configuration option. In addition, if the scrollarea has a positive `-borderwidth` value (e.g., the default `1`) and a `-relief` value other than `flat` (e.g., the default `sunken`) then this subcommand sets the `-borderwidth` option of `widget` to `0`, provided that `widget` supports this option.

REMARK 5: This subcommand is the only supported way to embed a widget into a scrollarea. Any attempt to embed further widgets, using any geometry manager, will lead to unpredictable results!

`pathName unsetattrib name`

Unsets the attribute `name`. Returns an empty string.

`pathName widget`

Returns the argument passed to the most recent successful invocation of the `setwidget` subcommand, or an empty string if there was no successful invocation of that subcommand yet.

BINDINGS

When a new scrollarea is created, it has no default event bindings: scrollareas are not intended to be interactive.

KEYWORDS
scrollarea, widget, scrollbar

The `scrollutil::getscrollarea` Command

NAME
`scrollutil::getscrollarea` – Query the scrollarea containing a given widget

SYNOPSIS

```
scrollutil::getscrollarea widget
```

DESCRIPTION
Returns the path name of the scrollarea into which the widget given by the `widget` argument is embedded via the scrollarea's `setwidget` subcommand, or an empty string if there is no such scrollarea widget.

KEYWORDS
scrollarea, widget

Keywords: scrollarea

scrollednotebook

The `scrollutil::scrollednotebook` Command

and the `closetab` Style Element

For Scrollutil Version 2.11

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The scrollutil::scrollednotebook Command – Quick Reference
- The scrollutil::scrollednotebook Command – Detailed Reference
- The closetab Style Element
- The scrollutil::addclosetab Command
- The scrollutil::removeclosetab Command
- The scrollutil::closetabstate Command

The `scrollutil::scrollednotebook` Command – Quick Reference

NAME
`scrollutil::scrollednotebook` – Create and manipulate scrollednotebook widgets

SYNOPSIS

```
package require scrollutil_tile
scrollutil::scrollednotebook pathName ?options?
```

STANDARD OPTIONS

```
-class  -cursor  -style
```

WIDGET-SPECIFIC OPTIONS

```
-forgetcommand command

-height screenDistance

-leavecommand command

-movabletabs boolean

-padding paddingSpec

-takefocus 0|1|""|command

-width screenDistance
```

WIDGET COMMAND

```
pathName add window ?options?

pathName adjustsize

pathName attrib ?name ?value name value ...??

pathName cget option

pathName closetabstate tabId ?normal|disabled?

pathName configure ?option ?value option value ...??

pathName forget tabId

pathName hasattrib name

pathName hastabattrib tabId name

pathName hide tabId

pathName identify component x y

pathName index tabId

pathName insert pos window ?options?
```

```
pathName instate stateSpec ?script?

pathName notebookpath

pathName see tabId

pathName select ?tabId?

pathName state ?stateSpec?

pathName style

pathName tab tabId ?option ?value option value ...??

pathName tabattrib tabId ?name ?value name value ...??

pathName tabs

pathName unsetattrib name

pathName unsettabattrib tabId name
```

KEYWORDS

scrollednotebook, widget, notebook, scrolled

The `scrollutil::scrollednotebook` Command – Detailed Reference

NAME

`scrollutil::scrollednotebook` – Create and manipulate scrollednotebook widgets

SYNOPSIS

```
package require scrollutil_tile
scrollutil::scrollednotebook pathName ?options?
```

Note that the `scrollutil::scrollednotebook` command is provided by the Scrollutil_tile package, but not by Scrollutil.

DESCRIPTION

The `scrollutil::scrollednotebook` command creates a new window named `pathName` and makes it into a **scrollednotebook** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the scrollednotebook widget such as its width, height, and style. The `scrollutil::scrollednotebook` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

A scrollednotebook is a mega-widget that contains a ttk::notebook within a scrollableframe and supports an arbitrary number of unsqueezed tabs. Currently not visible tabs can be

brought into view by navigating with the mouse wheel, touchpad, or keyboard, and by scrolling with the aid of the two arrow buttons placed on demand in the top-left and top-right or bottom-left and bottom-right corners (depending on the notebook's style). The size of the arrows depends on Tk's scaling factor, while their color is theme-specific and is updated whenever the theme changes. An individual tab can be made visible programmatically with the aid of the `see` subcommand. Unlike the ttk::notebook widget, whose `-width` option is quite often overridden by the total width of the tabs, the scrollednotebook widget respects the value of its `-width` option, regardless of the space required by the tabs.

The API of the scrollednotebook widget is nearly identical to that of ttk::notebook. Apart from a few additional, scrollednotebook-specific subcommands, the Tcl command associated with a scrollednotebook widget provides the same subcommands as the one associated with a ttk::notebook.

STANDARD OPTIONS

```
-class  -cursor  -style
```

See the **ttk_widget** manual entry for details on the standard options. The default value of the `-class` option is `Scrollednotebook` and that of `-cursor` an empty string. The `-style` option, whose value defaults to `TNotebook`, refers to the style of the ttk::notebook widget contained in the scrollednotebook. The value of the `-tabposition` option of the style specified by this option must be one of `nw` (the default), `n` (the default for the `aqua` theme), `ne`, `sw`, `s`, or `se`. These values will result in a horizontal tab layout (the scrollednotebook widget doesn't support vertical tab layouts).

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-forgetcommand
Database Name:	forgetCommand
Database Class:	ForgetCommand

Specifies a command to be invoked when attempting to remove a tab and unmanage the associated window with the aid of the `forget` subcommand. If the option's value is a nonempty string then it is concatenated with the path name of the scrollednotebook widget and that of the window to be unmanaged, and the resulting script, which must return a boolean, is evaluated in the global scope. If the return value of this script is false then the `forget` subcommand is aborted, i.e., the tab is not removed and the associated window is not unmanaged. The default is an empty string.

Command-Line Name:	-height
Database Name:	height
Database Class:	Height

If present and greater than zero, specifies the desired height of the pane area in any of the

forms acceptable to `Tk_GetPixels`. Otherwise, the widget's height is set to the requested height of the ttk::notebook contained in it (which in turn depends on the maximum requested height of all panes and tabs and the extra vertical space resulting from the value of the `-padding` option), immediately after the scrollednotebook gets mapped. The default is `0`.

Command-Line Name:	-leavecommand
Database Name:	leaveCommand
Database Class:	LeaveCommand

Specifies a command to be invoked when attempting to leave the currently selected window by selecting a different one. If the option's value is a nonempty string then it is concatenated with the path name of the scrollednotebook widget and that of the currently selected window, and the resulting script, which must return a boolean, is evaluated in the global scope. If the return value of this script is false then the operation of selecting a different window is aborted. The default is an empty string.

Command-Line Name:	-movabletabs
Database Name:	movableTabs
Database Class:	MovableTabs

Specifies a boolean value that determines whether the tabs can be moved with the mouse. See the FURTHER BINDINGS section below for details. The default is `1`.

Command-Line Name:	-padding
Database Name:	padding
Database Class:	Padding

Specifies the amount of extra space to add around the outside of the ttk::notebook widget contained in the scrollednotebook. See the **ttk_notebook** manual entry for details. The default is an empty string. Note that if this default value is used then the ttk::notebook widget will be drawn with the external padding specified by the `-padding` option of the widget's style.

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the scrollednotebook widget accepts the focus during keyboard traversal. It is almost identical to the standard option of the same name (see the **options** manual entry for details). The only difference is that not the scrollednotebook widget itself but the ttk::notebook contained in it will receive the focus during keyboard traversal with the

standard keys (`Tab` and `Shift-Tab`). The default is `"ttk::takefocus"` (just like for ttk::notebook and several other Tk themed widgets).

Command-Line Name:	-width
Database Name:	width
Database Class:	Width

If present and greater than zero, specifies the desired width of the ttk::notebook contained in the widget, in any of the forms acceptable to `Tk_GetPixels`. Otherwise, the widget's width is set to the requested width of the ttk::notebook contained in it (which in turn depends on the maximum requested width of all panes, the total width of the tabs, and the extra horizontal space resulting from the value of the `-padding` option), immediately after the scrollednotebook gets mapped. The default is `10c`, which should be overridden with a suitable application-specific value.

TAB OPTIONS

See the **ttk_notebook** manual entry.

TAB IDENTIFIERS

See the **ttk_notebook** manual entry.

WIDGET COMMAND

The `scrollutil::scrollednotebook` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `args` determine the exact behavior of the command. The following commands are possible for scrollednotebook widgets:

`pathName add window ?options?`

See the **ttk_notebook** manual entry.

`pathName adjustsize`

Sets the scrollednotebook's `-height` option to the maximum requested height of all panes, and its `-width` option to the maximum requested width of all panes, increased by the extra horizontal space resulting from the value of the `-padding` option.

`pathName attrib ?name ?value name value ...??`

Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the `hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName cget option`

Returns the current value of the configuration option given by `option`, which may have any

of the values accepted by the `scrollutil::scrollednotebook` command.

`pathName closetabstate tabId ?normal|disabled?`
Sets or queries the state of the `closetab` element of the tab specified by `tabId`.

REMARK: Invoking this subcommand is just a shortcut for passing `pathName`, `tabId`, and the optional argument (if present) to the `scrollutil::closetabstate` command.

`pathName configure ?option ?value option value ...??`
Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `scrollutil::scrollednotebook` command.

`pathName forget tabId`
See the **ttk_notebook** manual entry.

`pathName hasattrib name`
Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName hastabattrib tabId name`
Returns `1` if the attribute `name` for the tab given by `tabId` exists and `0` otherwise.

`pathName hide tabId`
See the **ttk_notebook** manual entry.

`pathName identify component x y`
See the **ttk_notebook** manual entry.

`pathName index tabId`
See the **ttk_notebook** manual entry.

`pathName insert pos window ?options?`
See the **ttk_notebook** manual entry.

`pathName instate stateSpec ?script?`
See the **ttk_widget** manual entry.

`pathName notebookpath`
Returns the path name of the ttk::notebook widget contained in the scrollednotebook. One of the extremely rare cases where this subcommand is needed, is related to setting a tooltip for one of the tabs. If `nb` denotes the ttk::notebook path name returned by this subcommand, you can set a tooltip `message` for one of its tabs by using either the `-tab` option of the `tooltip::tooltip` command from the tooltip package contained in tklib:

```
tooltip::tooltip nb -tab tabId message
```

or the `-nbktab` option of the `baltip::tip` command from the baltip package by Alex Plotnikov:

```
baltip::tip nb message -nbktab tabWindow ...
```

The support for the `-tab` option of the `tooltip::tooltip` command was added in version

1.6 of the tooltip package. The `baltip::tip` command accepts a great variety of additional options, too. The baltip package can be downloaded from the location

https://chiselapp.com/user/aplsimple/repository/baltip/download/

REMARK: With the above exception, in the vast majority of cases there is no need to access the ttk::notebook widget contained in the scrollednotebook directly. Moreover, using the ttk::notebook's options or subcommands instead of those of the scrollednotebook widget will more often than not result in an unpredictable behavior.

`pathName see tabId`
This subcommand adjusts the view in the scrollednotebook's window so that the tab given by `tabId` becomes visible in it. Note that when a tab gets selected, this command is automatically invoked for it, in order to make that tab visible.

`pathName select ?tabId?`
See the **ttk_notebook** manual entry.

`pathName state ?stateSpec?`
See the **ttk_widget** manual entry.

`pathName style`
Returns the value of the `-style` configuration option (i.e., the style used by the ttk::notebook widget contained in the scrollednotebook). For Tk themed widgets this subcommand was introduced in Tk 8.7a4, but the scrollednotebook widget provides it for all supported Tk versions.

`pathName tab tabId ?option ?value option value ...??`
See the **ttk_notebook** manual entry.

`pathName tabattrib tabId ?name ?value name value ...??`
Queries or modifies the attributes of the tab given by `tabId`. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for the tab. If `name` is specified with no `value`, then the command returns the value of the one named tab attribute, or an empty string if no corresponding value exists (you can use the `hastabattrib` subcommand to distinguish this case from the one that the value of an *existing* tab attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given tab attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName tabs`
See the **ttk_notebook** manual entry.

`pathName unsetattrib name`
Unsets the attribute `name`. Returns an empty string.

`pathName unsettabattrib tabId name`
Unsets the attribute `name` for the tab given by `tabId`. Returns an empty string.

KEYBOARD TRAVERSAL
See the **ttk_notebook** manual entry.

VIRTUAL EVENTS
As described in the **ttk_notebook** manual entry, after a new tab of a ttk::notebook is selected (programmatically or interactively), the widget generates a `<<NotebookTabChanged>>` virtual event. This holds true for scrollednotebook widgets, too.

The Scrollutil_tile package defines the virtual event <<Button3>> as <Button-3> for all windowing systems and additionally as <Control-Button-1> for Mac OS X/11+. If this event occurs over a tab of a ttk::notebook or scrollednotebook whose disabled widget state flag is not set, and the Tk version is 8.5a2 or later, then the widget generates a <<MenuItemsRequested>> virtual event, by invoking event generatewith the -data option set to a list consisting of the path name of an empty pop-up menu and the numerical index of the tab in question. (The support for this option was introduced in Tk 8.5a2.) If the application creates a binding for this virtual event, the binding script can access the user data as the value of the %d field, with the goal to populate the menu with application-specific entries. Note that the pop-up menu is posted with a delay of 100 ms, to make sure that it will appear readily populated by the application.

The following example provides a pop-up menu item for closing the tabs of a ttk::notebook or scrollednotebook widget $nb:

```
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }

proc populateMenu {nb data} {
    foreach {menu tabIdx} $data {}
    $menu add command -label "Close Tab" -command [list $nb forget
    $tabIdx]
}
```

See the next section for the description of the virtual events <<NotebookTabMoved>>, <<NotebookTabClosed>>, and <<CloseTabRequested>>.

FURTHER BINDINGS

The Scrollutil_tile package creates the following additional bindings for the ttk::notebook and scrollednotebook widgets:

1. **Navigation between the tabs of a scrollednotebook or ttk::notebook widget via the mouse wheel or touchpad:**A mouse wheel tick or two-finger gesture on the touchpad selects the tab of state normal cyclically following/preceding the currently selected one. (Note that the same effect can also be achieved with the keys Right, Left, Control-Tab, and Control-Shift-Tab.) On the windowing system aqua, if the Option key is down while the mouse wheel is being rotated then a wheel tick selects the 10'th tab of state normal cyclically following/preceding the currently selected one (modulo the number of tabs). The same holds true on the windowing systems x11 and win32 if the Tk version is 8.7a4 or later and the mouse wheel is rotated with the Alt key down.
2. **Scrolling the tab row of a scrollednotebook widget with the aid of the arrow buttons:**If the tab at the right edge of the scrollednotebook is only partly visible then pressing mouse button 1 over the right arrow adjusts the view in the tab row so that the tab becomes fully visible; otherwise the click brings the next nonhidden tab into view. Similarly, if the tab at the left edge of the scrollednotebook is only partly visible then pressing mouse button 1 over the left arrow adjusts the view in the tab row so that the tab becomes fully visible; otherwise the click brings the previous nonhidden tab into view. If the mouse button is held down for at least 300 milliseconds, the actions described above will auto-repeat every 100 milliseconds.
3. **Moving the tabs of a scrollednotebook or ttk::notebook widget with the mouse**: If

mouse button 1 is pressed over a tab but outside the `closetab` element (if any) and then dragged outside that tab then (in case of a scrollednotebook provided that the value of the widget's `-movabletabs` option is true) the mouse cursor takes on the shape of a left/right arrow, indicating that the tab in question is being moved to another position. This operation ends when mouse button 1 is released, and can be canceled by pressing the `Escape` key. In both cases, the mouse cursor is reset to its original value, specified by the `-cursor` configuration option. After releasing mouse button 1, the source tab is moved to the position previously pointed to by the arrow (provided that the action was not canceled via `Escape`). If the Tk version is 8.5a2 or later then after moving the tab to its new position, the widget generates a `<<NotebookTabMoved>>` virtual event, by invoking `event generate`with the `-data` option set to a list consisting of the path name of the widget contained in the source pane and the numerical index of the target tab position. (The support for this option was introduced in Tk 8.5a2.) All the above works also for a ttk::notebook with vertical tab layout, with the only difference that for the duration of the drag, the cursor is set to one having the shape of an up/down arrow.

4. **Closing a tab of a scrollednotebook widget with the aid of the closetab element**: If mouse button 1 is pressed over the `closetab` element (if any) of a tab and later released over the same element then, provided that both the value of the tab's `-state` option and the state of its `closetab` element is `normal`, the tab is closed by means of the widget's `forget` subcommand. Recall that this operation can get canceled by the command specified as the value of the `-forgetcommand` option; if this was not the case and the Tk version is 8.5a2 or later then, after closing the tab, the widget generates a `<<NotebookTabClosed>>` virtual event, by invoking `event generate`with the `-data` option set to the path name of the widget contained in that tab's pane.
5. **Closing a tab of a ttk::notebook widget with the aid of the closetab element**: If mouse button 1 is pressed over the `closetab` element (if any) of a tab and later released over the same element then, provided that both the value of the tab's `-state` option and the state of its `closetab` element is `normal`, an action depending on the Tk version is taken:
   - If the Tk version is 8.5a2 or later then the tab is not (yet) closed. Instead, the widget generates a `<<CloseTabRequested>>` virtual event, by invoking `event generate`with the `-data` option set to the numerical index of the tab in question. It is the responsibility of the application to create a binding for this virtual event and to close the tab from within the binding script by invoking the widget's `forget` subcommand if appropriate, like in the following example:

```
bind $nb <<CloseTabRequested>> { closeTab %W %d }

proc closeTab {nb tabIdx} {
    set btn [tk_messageBox -title "Close Tab?" -icon question \
             -message "Do you really want to close the tab?" -type
    yesno
    if {$btn eq "yes"} {
        $nb forget $tabIdx

        # Optionally:
        set widget [lindex [$nb tabs] $tabIdx]
```

```
            event generate $nb <<NotebookTabClosed>> -data $widget
        }
    }
```

- If the Tk version is earlier than 8.5a2 then the tab is closed immediately by means of the widget's `forget` subcommand. (Recall that the `-data` option for virtual events was introduced in Tk 8.5a2.)

If the widget's `disabled` state flag is set then none of the above actions occur: the tabs are completely insensitive.

KEYWORDS

scrollednotebook, widget, notebook, scrolled

The `closetab` Style Element

The Scrollutil_tile package automatically creates a new style element named `closetab`, whose shape resembles that of an x character. The element's size depends on Tk's scaling factor, while its foreground color is theme-specific and is updated whenever the theme changes.

You can add this element to the tabs of a ttk::notebook style (like `TNotebook` or `My.TNotebook`) by using the `scrollutil::addclosetab` command, thus making the tabs of any ttk::notebook or scrollednotebook widget of that style closable by the user.

For the opposite operation the Scrollutil_tile package provides the `scrollutil::removeclosetab` command.

The state (`normal` or `disabled`) of of the `closetab` element of a ttk::notebook or scrollednotebook tab can be set and queried with the aid of the `scrollutil::closetabstate` command.

The `scrollutil::addclosetab` Command

NAME

`scrollutil::addclosetab` – Add the `closetab` element to the tabs of a ttk::notebook style

SYNOPSIS

```
package require scrollutil_tile
scrollutil::addclosetab notebookStyle
```

Note that the `scrollutil::addclosetab` command is provided by the Scrollutil_tile package, but not by Scrollutil.

DESCRIPTION

Modifies the style `notebookStyle.Tab` corresponding to a given ttk::notebook style (like `TNotebook` or `My.TNotebook`) by adding the `closetab` element to it. This makes the tabs of any ttk::notebook or scrollednotebook widget whose `-style` option was set to `notebookStyle` closable by the user. If the `closetab` element was already present in the style `notebookStyle.Tab` then the command returns without performing any action.

KEYWORDS

ttk::notebook style, closetab

The `scrollutil::removeclosetab` Command

NAME

`scrollutil::removeclosetab` – Remove the `closetab` element from the tabs of a ttk::notebook style

SYNOPSIS

```
package require scrollutil_tile
scrollutil::removeclosetab notebookStyle
```

Note that the `scrollutil::removeclosetab` command is provided by the Scrollutil_tile package, but not by Scrollutil.

DESCRIPTION

Modifies the style `notebookStyle.Tab` corresponding to a given ttk::notebook style (like `TNotebook` or `My.TNotebook`) by removing the `closetab` element from it. If the `closetab` element was not present in the style `notebookStyle.Tab` then the command returns without performing any action.

KEYWORDS

ttk::notebook style, closetab

The `scrollutil::closetabstate` Command

NAME

`scrollutil::closetabstate` – Set or query the state of the `closetab` element of a ttk::notebook or scrollednotebook tab

SYNOPSIS

```
package require scrollutil_tile
scrollutil::closetabstate notebook tabId ?normal|disabled?
```

Note that the `scrollutil::closetabstate` command is provided by the Scrollutil_tile package, but not by Scrollutil.

DESCRIPTION

Sets or queries the state of the `closetab` element of the tab specified by `tabId` of the ttk::notebook or scrollednotebook widget `notebook`. The optional argument specifies the state as `normal` or `disabled`. Without this argument the command returns the current state of the tab's `closetab` element.

Provided that the widget's `disabled` state flag is not set and the value of the tab's `-state` option is `normal`, the differences between the two `closetab` states are as follows:

- In the default `normal` state, the `closetab` element is shown in white color on light red background when hovered, and in white color on dark red background when pressed with mouse button 1. By releasing mouse button 1 over the same element, the action described under points 4 and 5 of the FURTHER BINDINGS section above is triggered.
- In the `disabled` state, the `closetab` element, both when hovered and when pressed, is

shown in the same theme-specific foreground color as a disabled text, and pressing and later releasing mouse button 1 over it has no effect.

Note that the `closetabstate` scrollednotebook widget subcommand is a convenient shortcut for this command.

KEYWORDS

tab, closetab, state

Keywords: scrollednotebook

scrollsync

The `scrollutil::scrollsync` and `scrollutil::getscrollsync` Commands

For Scrollutil Version 2.11

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The scrollutil::scrollsync Command – Quick Reference
- The scrollutil::scrollsync Command – Detailed Reference
- The scrollutil::getscrollsync Command

The `scrollutil::scrollsync` Command – Quick Reference

NAME

`scrollutil::scrollsync` – Create and manipulate scrollsync widgets

SYNOPSIS

```
scrollutil::scrollsync pathName ?options?
```

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

WIDGET-SPECIFIC OPTIONS

```
-takefocus 0|1|""|command
```

WIDGET COMMAND

```
pathName attrib ?name ?value name value ...??

pathName cget option

pathName configure ?option ?value option value ...??
```

```
pathName hasattrib name

pathName setwidgets widgetList

pathName unsetattrib name

pathName widgets

pathName xview ?args?

pathName xview

pathName xview moveto fraction

pathName xview scroll number units|pages
pathName yview ?args?

pathName yview

pathName yview moveto fraction

pathName yview scroll number units|pages
```

KEYWORDS

scrollsync, widget, scrolling

The `scrollutil::scrollsync` Command – Detailed Reference

NAME

`scrollutil::scrollsync` – Create and manipulate scrollsync widgets

SYNOPSIS

```
scrollutil::scrollsync pathName ?options?
```

DESCRIPTION

The `scrollutil::scrollsync` command creates a new window named `pathName` and of the class `Scrollsync`, and makes it into a **scrollsync** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the scrollsync widget such as its borderwidth and relief. The `scrollutil::scrollsync` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

A scrollsync is a mega-widget, designed for scrolling several widgets simultaneously. It is both horizontally and vertically scrollable, meaning that it supports the `-xscrollcommand` and `-yscrollcommand` configuration options and the associated Tcl command has the `xview` and `yview` subcommands. Whenever the horizontal/vertical position of the view in

the window of one of the horizontally/vertically scrollable widgets contained in the list passed to the `setwidgets` subcommand of the associated Tcl command changes, the view in the windows of all the other horizontally/vertically scrollable elements of that list is automatically adjusted accordingly, thus making sure that the view's position in these windows is kept in sync.

A scrollsync widget is typically created within a scrollarea, like in the following example:

```
set sa [scrollutil::scrollarea ...]
set ss [scrollutil::scrollsync $sa.ss ...]
$sa setwidget $ss

set lb1 [listbox $ss.lb1 ...]
set lb2 [listbox $ss.lb2 ...]
$ss setwidgets [list $lb1 $lb2]

grid $lb1 $lb2 -sticky news -padx {0 2}
grid rowconfigure    $ss 0   -weight 1
grid columnconfigure $ss all -weight 1

pack $sa -expand yes -fill both
```

STANDARD OPTIONS

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

See the **options** manual entry for details on the standard Tk widget options. The `-background`, `-highlightbackground`, `-highlightcolor`, and `-highlightthickness` options are only supported by the Scrollutil package, but not by Scrollutil_tile. They have the same default values as the options of the same names for Tk frame widgets. The default values of the remaining standard options are:

```
-borderwidth 0 -cursor "" -relief flat -xscrollcommand ""
    -yscrollcommand ""
```

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the scrollsync widget accepts the focus during keyboard traversal. It is almost identical to the standard option of the same name (see the **options** manual entry for details). The only difference is that not the scrollsync widget itself but the first element of the widget list passed to the `setwidgets` subcommand of the associated Tcl

command will receive the focus during keyboard traversal with the standard keys (`Tab` and `Shift-Tab`). The default is `0`, being that a scrollsync widget is esentially a frame containing the above-mentioned widgets.

WIDGET COMMAND

The `scrollutil::scrollsync` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `arg`s determine the exact behavior of the command. The following commands are possible for scrollsync widgets:

`pathName attrib ?name ?value name value ...??`
Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the `hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. Each `name` may be an arbitrary string.

`pathName cget option`
Returns the current value of the configuration option given by `option`, which may have any of the values accepted by the `scrollutil::scrollsync` command.

`pathName configure ?option ?value option value ...??`
Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `scrollutil::scrollsync` command.

`pathName hasattrib name`
Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName setwidgets widgetList`
Sets the widgets in whose windows the view's position is to be kept in sync. The `widgetList` argument must be a valid list consisting of path names of existing widgets. Whenever the horizontal/vertical position of the view in the window of one of the horizontally/vertically scrollable widgets contained in this list changes, the view in the windows of all the other horizontally/vertically scrollable elements of the list will be automatically adjusted accordingly, thus making sure that the view's position in these windows is kept in sync. The return value is the argument passed to the previous successful invocation of this subcommand, or an empty list if there was no successful `setwidgets` invocation before.

REMARK: When one of the widgets whose path name is contained in `widgetList` gets destroyed, that widget is automatically removed from the internal list of widgets in whose windows the view's position is kept in sync.

`pathName unsetattrib name`
Unsets the attribute `name`. Returns an empty string.

`pathName widgets`
Returns the argument passed to the most recent successful invocation of the `setwidgets` subcommand, or an empty list if there was no successful invocation of that subcommand yet.

`pathName xview ?args?`

```
pathName xview

pathName xview moveto fraction

pathName xview scroll number units|pages
```

This command passes its arguments to the `xview` command of the current **master widget for the x axis** and returns the result of that command invocation. The master widget for the x axis is the element of the widget list passed to the `setwidgets` subcommand having the smallest relative view width among the horizontally scrollable widgets in that list, i.e., the least difference between the last and first elements of the two-element list returned by its `xview` command. This master widget can vary during program execution (in case of text widgets it can even change depending on the current vertical view position).

`pathName yview ?args?`

```
pathName yview

pathName yview moveto fraction

pathName yview scroll number units|pages
```

This command passes its arguments to the `yview` command of the current **master widget for the y axis** and returns the result of that command invocation. The master widget for the y axis is the element of the widget list passed to the `setwidgets` subcommand having the smallest relative view height among the vertically scrollable widgets in that list, i.e., the least difference between the last and first elements of the two-element list returned by its `yview` command. This master widget can vary during program execution.

BINDINGS
When a new scrollsync widget is created, it has no default event bindings: scrollsync widgets are not intended to be interactive.

KEYWORDS
scrollsync, widget, scrolling

The `scrollutil::getscrollsync` Command

NAME
`scrollutil::getscrollsync` – Query the scrollsync containing a given widget

SYNOPSIS

```
scrollutil::getscrollsync widget
```

DESCRIPTION

Returns the path name of the scrollsync into which the widget given by the `widget` argument is embedded via the scrollsync's `setwidgets` subcommand, or an empty string if there is no such scrollsync widget.

KEYWORDS

scrollsync, widget

Keywords: scrollsync

scrollutil

Scrollutil Programmer's Guide

For Scrollutil Version 2.11

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

Overview

- What Is Scrollutil?
- How to Get It?
- How to Install It?
- How to Use It?
- More on scrollutil::scalingpct
- The Procedure scrollutil::getScalingPct and Tk's Scaling Factor
- More on scrollutil::svgfmt

Examples

- The Helper Script styleUtil.tcl
- A Scrolled tablelist Widget
- A Scrolled text Widget
- A Scrolled canvas Widget
- Synchronizing Two listbox Widgets
- Synchronizing Three tablelist Widgets
- A Script Using a scrollutil::scrollableframe Widget
- A Script Using a BWidget ScrollableFrame Widget
- A Script Using an iwidgets::scrolledframe Widget
- A Script Using Two scrollutil::scrollableframe Widgets
- A Script Using Two BWidget ScrollableFrame Widgets
- A Script Using Two iwidgets::scrolledframe Widgets
- A scrollutil::scrollednotebook Demo

- A scrollutil::plainnotebook Demo
- A scrollutil::pagesman Demo

Overview

What Is Scrollutil?

Scrollutil is a library package for Tcl/Tk versions 8.4 or higher, written in pure Tcl/Tk code. It contains:

- the implementation of the **scrollarea**, **scrollsync**, **scrollableframe**, **scrollednotebook**, **plainnotebook**, and **pagesman** mega-widgets, including a general utility module for mega-widgets. The scrollednotebook and plainnotebook widgets require tile 0.8 or higher;
- the command `scrollutil::addMouseWheelSupport`, which creates mouse wheel event bindings for a given binding tag;
- commands for *user-friendly* mouse wheel event handling in **scrollable widget containers** like scrollutil::scrollableframe, BWidget ScrollableFrame, and iwidgets::scrolledframe. These commands require Tcl/Tk versions 8.4 or higher on X11 and Mac OS X/11+ and Tk 8.6b2 or later on Windows;
- commands for *user-friendly* mouse button 2 event handling in scrollutil::scrollableframe widgets;
- demo scripts illustrating the use of the Scrollutil package in connection with various scrollable widgets and the above-mentioned scrollable widget containers;
- demo scripts illustrating the use of the scrollednotebook, plainnotebook, and pagesman widgets;
- this tutorial;
- reference pages in HTML format.

The scrollutil::scrollarea mega-widget greatly simplifies the creation of arbitrary scrolled widgets. It consists of a scrollable widget and two scrollbars connected with that widget. The display mode of each scrollbar can be `static`, `dynamic`, or `none`. This scrolled window implementation also supports the widgets that are scrollable in one direction only (e.g., entry and ttk::entry) and respects the header component and title columns of tablelist widgets (this is freely configurable).

The scrollutil::scrollarea widget is similar to BWidget ScrolledWindow and its snit-based equivalent widget::scrolledwindow, contributed by Jeffrey Hobbs and contained in tklib. The snit-based scrodget package by Aldo Buratti and its TclOO-based equivalent scrolledwidget contributed by Johann Oberdorfer are further scrolled window implementations. However, *full* tablelist support is only provided by the scrollarea widget, which is free from external dependencies like BWidget, snit, or (for Tcl 8.5) TclOO. It is also free from the shimmering problem in connection with text widgets, which the above-mentioned scrolled window implementations either share with the autoscroll package (contained in tklib) or circumvent in a suboptimal way.

The scrollutil::scrollsync mega-widget is designed for scrolling several widgets simultaneously. Whenever the horizontal/vertical position of the view in the window of one of its widgets changes, the view in the windows of all the other widgets is automatically adjusted accordingly, thus making sure that the view's position in these windows is kept in sync. This mega-widget is horizontally and vertically scrollable, hence it can be embedded into a

scrollutil::scrollarea widget via the latter's `setwidget` subcommand.

The scrollutil::scrollableframe mega-widget is a scrollable widget container. It contains a content frame, whose dimensions are typically larger than those of the widget itself. Arbitrary regions of this frame can be brought into view by scrolling, and the widget also provides a command for making individual widgets contained in the content frame visible in the scrollableframe window.

The scrollutil::scrollableframe widget is similar to BWidget ScrollableFrame and iwidgets::scrolledframe. However, unlike these widgets, which use a canvas for scrolling the content frame, it adjusts the view with the aid of the `place` geometry manager, just like the `scrolledframe::scrolledframe` command of the Scrolledframe package by Maurice Bredelet (ulis) and its optimized and enhanced version contributed by Keith Nash. For details on these commands see the wiki page

https://wiki.tcl-lang.org/page/A+scrolled+frame

The canvas-free approach is more lightweight and integrates better in applications that use tile widgets.

The scrollutil::scrollednotebook mega-widget contains a ttk::notebook within a scrollableframe and supports an arbitrary number of unsqueezed tabs. Currently not visible tabs can be brought into view by navigating with the mouse wheel, touchpad, or keyboard, and by scrolling with the aid of the two arrow buttons placed on demand in the top-left and top-right or bottom-left and bottom-right corners (depending on the notebook's style). The widget also provides a command for making individual tabs visible in the scrollednotebook window. Unlike the ttk::notebook widget, whose `-width` option is quite often overriden by the total width of the tabs, the scrollednotebook widget respects the value of its `-width` option, regardless of the space required by the tabs.

The scrollutil::plainnotebook mega-widget extends a ttk::notebook having an arbitrary number of pages with invisible tabs by a ttk::frame to its left or right containing, among others, a scrollableframe whose content frame is the parent of a series of widgets that play the role of vertically laid-out notebook tabs. Currently not visible "tabs" can be brought into view by navigating with the mouse wheel, touchpad, or keyboard, and by scrolling with the vertical scrollbar of the scrollarea containing the scrollableframe. The widget also provides a command for making individual "tabs" visible in the plainnotebook window. Unlike a ttk::notebook widget with vertically aligned tabs, which in most themes has a suboptimal look and whose `-height` option is quite often overriden by the total height of the tabs, the plainnotebook widget respects the value of its `-height` option, regardless of the space required by the "tabs".

The scrollutil::pagesman mega-widget provides the basic functionality of a pages manager, meaning that it manages a list of windows, called **pages**, of which only one is visible at a time. By using it with plainnotebook widgets as pages, it is quite easy to write applications in which the user can descend from a plainnotebook to another one with a single mouse click and switch back in the same way to the original one. Everything needed for this navigation is provided by appropriate options and subcommands of the plainnotebook widget.

From the point of view of **the commands related to mouse wheel and <TouchpadScroll> event handling** provided by the Scrollutil package, the scrollability of a widget or widget container window means that the associated Tcl command supports the `xview scroll number units`and `yview scroll number units`subcommands. The reason for requiring

at least Tk version 8.6b2 on Windows for the commands related to scrollable widget containers is that in earlier Tk versions on this platform the mouse wheel events were sent to the widget having the focus rather than to the one under the pointer.

To make use of the user-friendly mouse wheel and `<TouchpadScroll>` event handling in scrollable widget containers via the Scrollutil package, follow the steps below:

- Create mouse wheel and `<TouchpadScroll>` event bindings for the binding tag `"all"` or for the toplevel widgets (including `"."`) having scrollable widget containers, by invoking the `scrollutil::createWheelEventBindings` command. In addition, register your scrollable widget containers for scrolling via these bindings with the aid of the `scrollutil::enableScrollingByWheel` command. Note that for the scrollutil::scrollableframe widget this command is automatically invoked at creation time. The above-mentioned bindings handle the mouse wheel and `<TouchpadScroll>` events by scrolling the (innermost) registered scrollable widget container that is an ascendant of the widget under the pointer and is contained in the latter's toplevel.
- Invoke the `scrollutil::adaptWheelEventHandling` command for those widgets contained in registered scrollable widget containers that have mouse wheel or `<TouchpadScroll>` event (class) bindings. This step modifies the mouse wheel and `<TouchpadScroll>` event handling as follows: If the focus is on the widget under the pointer then the above-mentioned events will be handled by the (class bindings of the) widget only, otherwise by the bindings created with the `scrollutil::createWheelEventBindings` command. Note that this command accepts as optional argument the `-ignorefocus` switch, which specifies that the above-mentioned events are to be handled by the widget under the pointer (only), ragardless of whether that widget has the focus or not.
- For some widgets it can be desirable to make the focus check within this modified event handling less restrictive. For example, if the widget under the pointer is an entry component of a mentry of type `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"`, or `"IPv6Addr"` and the focus is on any of its siblings, then the mouse wheel and `<TouchpadScroll>` events sent to this entry should be handled by the entry widget itself rather than scrolling the widget container that is an ascendant of the mentry. The `scrollutil::setFocusCheckWindow` command covers exactly cases like this.

The mouse wheel and `<TouchpadScroll>` event handling with the aid of the Scrollutil package was also tested to work with the `scrolledframe::scrolledframe` command of the Scrolledframe package by Maurice Bredelet (ulis) and its optimized and enhanced version contributed by Keith Nash, as well as with the `sframe` command implemented by Paul Walton. For details on these commands (which provide further implementations of scrollable widget containers) see the above-mentioned wiki page.

The comands related to mouse button 2 event handling are restricted to the scrollutil::scrollableframe widget, which provides scan support via the `scan mark`and `scan dragto`commands. The mouse button 2 event bindings for the binding tag `"all"`, created automatically by Scrollutil, handle the `<Button-2>`, `<B2-Motion>`, and `<ButtonRelease-2>` events by scanning the content of the (innermost) scrollableframe widget that is an ascendant of the clicked widget and is contained in the latter's toplevel. By invoking the `scrollutil::adaptBtn2EventHandling` command for the widgets that are contained in a scrollableframe and have mouse button 2 (class) bindings, the handling of these events will be modified as follows: If the focus is on the clicked widget then the events will be handled by

the (class bindings of the) widget only, otherwise by the bindings created for the `"all"` tag. This comes in handy especially on Android, where it provides user-friendly scan support for scrollableframe widgets via one-finger gestures.

How to Get It?

Scrollutil is available for free download from the Web page

https://www.nemethi.de

The distribution file is `scrollutil2.11.tar.gz` for UNIX and `scrollutil2_11.zip` for Windows. These files contain the same information, except for the additional carriage return character preceding the linefeed at the end of each line in the text files for Windows.

Scrollutil is also included in tklib, which has the address

https://core.tcl-lang.org/tklib

How to Install It?

Install the package as a subdirectory of one of the directories given by the `auto_path` variable. For example, you can install it as a subdirectory of the `lib` directory within your Tcl/Tk installation.

To install Scrollutil *on UNIX*, `cd` to the desired directory and unpack the distribution file `scrollutil2.11.tar.gz`:

```
gunzip -c scrollutil2.11.tar.gz | tar -xf -
```

On most UNIX systems this can be replaced with

```
tar -zxf scrollutil2.11.tar.gz
```

Both commands will create a directory named `scrollutil2.11`, with the subdirectories `demos`, `doc`, and `scripts`.

On Windows, use WinZip or some other program capable of unpacking the distribution file `scrollutil2_11.zip` into the directory `scrollutil2.11`, with the subdirectories `demos`, `doc`, and `scripts`.

Notice that in tklib the Scrollutil `demos` directory is replaced with the subdirectory `scrollutil` of the `examples` directory. Please take this into account when reading the examples below.

How to Use It?

The Scrollutil distribution provides two packages, called **Scrollutil** and **Scrollutil_tile**. The main difference between the two is that Scrollutil_tile enables the tile-based, theme-specific appearance of scrollarea, scrollsync, and scrollableframe widgets, and provides the themed scrollednotebook and plainnotebook widgets; this package requires tile 0.8 or higher. It is not possible to use both packages in one and the same application, because both are implemented in the same `scrollutil` namespace and provide identical commands (except for the commands `scrollutil::scrollednotebook`, `scrollutil::plainnotebook`, `scrollutil::addclosetab`, `scrollutil::removeclosetab`, and `scrollutil::closetabstate`, which are provided by the Scrollutil_tile package only).

To be able to access the commands and variables of the Scrollutil package, your scripts must contain one of the lines

```
package require scrollutil ?version?
package require Scrollutil ?version?
```

You can use either one of the two statements above because the file `scrollutil.tcl` contains both lines

```
package provide scrollutil ...
package provide Scrollutil ...
```

Likewise, to be able to access the commands and variables of the Scrollutil_tile package, your scripts must contain one of the lines

```
package require scrollutil_tile ?version?
package require Scrollutil_tile ?version?
```

Again, you can use either one of the two statements above because the file `scrollutil_tile.tcl` contains both lines

```
package provide scrollutil_tile ...
package provide Scrollutil_tile ...
```

You are free to remove one of these two lines from `scrollutil.tcl` and `scrollutil_tile.tcl`, respectively, if you want to prevent the corresponding packages from making themselves known under two different names each. Of course, by doing so you restrict the argument of `package require`to a single name.

Since the packages Scrollutil and Scrollutil_tile are implemented in the `scrollutil` namespace, you must either invoke the

```
namespace import scrollutil::pattern ?scrollutil::pattern ...?
```

command to import the *procedures* you need, or use qualified names like `scrollutil::scrollarea`. In the examples below we have chosen the latter approach.

To access Scrollutil *variables*, you *must* use qualified names. There are only 5 Scrollutil variables that are designed to be accessed outside the namespace `scrollutil`:

- The variable `scrollutil::version` holds the current version number of the Scrollutil package.
- The variable `scrollutil::library` holds the location of the Scrollutil installation directory.
- The read-only variable `scrollutil::scalingpct` is set when loading the Scrollutil or Scrollutil_tile package via `package require`to the scaling percentage corresponding to the display's DPI scaling level. Scrollutil adapts, among others, the default width of the Tk core scrollbars on X11 and that of the ttk::scrollbar widget for the built-in themes `alt`, `clam`, `classic`, and `default` to the value of this variable. The currently supported values are `100`, `125`, `150`, `175`, and `200`. You can use this variable, e.g., if you want to create images of different sizes, depending on the DPI scaling level. For example, if your application uses images of size 16 x 16 on an unscaled display and `scrollutil::scalingpct` has the value `150`, then the image size for this display should be 24 x 24.
- If the Tk version is either at least 8.7 (with built-in SVG support), or 8.6 and the tksvg extension can be loaded into the interpreter, then the variable `scrollutil::svgfmt` is set to a Tcl list that you can pass to the commands that create or manipulate SVG images as the value of their

`-format` option to make sure that your images will be properly scaled.
- The read-only variable `scrollutil::usingTile` has the value `0` in the Scrollutil package and the value `1` in Scrollutil_tile.

The Scrollutil_tile package checks whether the required Tk and tile versions are present, by executing the commands

```
package require Tk 8.4-
if {$::tk_version < 8.5 || [regexp {^8\.5a[1-5]$} $::tk_patchLevel]} {
    package require tile 0.8-
}
```

The second command above reflects the fact that, beginning with Tk 8.5a6, tile is integrated into the Tk core and therefore it should only be loaded explicitly when using an earlier Tk version.

More on `scrollutil::scalingpct`

The Scrollutil code sets the variable `scrollutil::scalingpct` to the value returned by the public procedure `scaleutil::scalingPercentage` of the scaleutil package, which is bundled with Scrollutil. The way this value is computed depends on the windowing system:

On Windows and Mac OS X/11+ the scaling percentage is computed from Tk's scaling factor given by `[tk scaling]`. Note that on Mac OS X/11+ the result is always `100`, regardless of the display's scaling level. On this system the desktop engine automatically scales everything as needed.

On X11, computing the scaling percentage from `[tk scaling]`is done as fallback method only, because the implementation of display scaling is highly dependent on the desktop environment and it mostly manipulates system resources that are resident outside of Xlib, which Tk is based on. (Traditional X applications like `bitmap` and `xmag` are also affected by this.) With the partial exception of Xfce and MATE (see below), the procedure computes the scaling percentage from the value of the X resource `Xft.dpi`, by executing the `xrdb` application. On GNOME-based systems where `xrdb` is not installed per default (e.g., Solus GNOME and Solus Budgie), it uses the `xrandr` application and the file `~/.config/monitors.xml` instead.

- On *Xfce*, the display scaling mode, called *window scaling*, can be "1x" or "2x" and can be selected in the *Appearance* dialog. Xfce versions 4.16 and later also support fractional scaling, whose parameters can be configured in the *Display* dialog. If the "2x" mode was selected then the scaling percentage will be set by the procedure `scaleutil::scalingPercentage` to `200`, otherwise it will be computed from the value of the font DPI, given by the X resource `Xft.dpi`. The value of the font DPI can be set in the *Fonts* tab of the *Appearance* dialog or (on Linux Lite 5+) via the *HiDPI Settings* dialog.
- On *MATE*, the display scaling mode, called *window scaling factor*, can be "Auto-detect", "Regular", or "HiDPI", and can be selected in the *MATE Tweak* dialog. The same effect can be achieved by using the *Monitor Preferences* dialog and selecting one of the possible scaling values "auto detect", "100%", and "200%". If needed, the procedure `scaleutil::scalingPercentage` auto-detects which one of the "Regular" ("100%") and "HiDPI" ("200%") modes corresponds to the preset "Auto-detect" ("auto detect") mode. If the "HiDPI" ("200%") mode was selected by the user or auto-detected by the procedure, then the scaling percentage will be set to `200`, otherwise it will be computed from the value of the font DPI, given by the X resource `Xft.dpi`. The value of the font DPI can be set via the *Font*

Rendering Details dialog, which in turn can be opened from within the *Fonts* tab of the *Appearance Preferences* dialog.
- In case of *GNOME on Xorg, Budgie, and Cinnamon versions earlier than 4.6*, the display scaling can be either 100 % or 200 %, and the procedure sets the scaling percentage accordingly to `100` or `200`. In newer GNOME and Budgie versions on Ubuntu one can enable the `x11-randr-fractional-scaling` as experimental feature, which adds (at least) 125 %, 150 %, and 175 % to the list of supported scaling percentages. Note that, due to the way this fractional scaling is implemented, the value returned by `scaleutil::scalingPercentage` will be `200`, regardless of the selected display scaling. Cinnamon versions 4.6 and later also support fractional scaling; if activated then the scaling percentage returned by the procedure will be `200`, regardless of the selected display scaling.
- *GNOME on Wayland* traditionally supports the display scaling values 100 % and 200 %, and the scaling percentage will be set accordingly to `100` or `200`. In newer GNOME versions one can enable the experimental feature `scale-monitor-framebuffer`, which adds (at least) 125 %, 150 %, and 175 % to the list of supported scaling percentages. With this feature enabled, the value returned by `scaleutil::scalingPercentage` will be `100` for older GNOME versions and `200` for newer ones, regardless of the scaling level, due to the fact that in this case, instead of window contents, monitor framebuffers will be scaled in a logical pixel coordinate space.
- *KDE Plasma on Xorg* provides fractional scaling support. On this desktop, the procedure will return `100`, `125`, `150`, `175`, or `200`, depending on the display's scaling level.
- *KDE Plasma on Wayland* supports fractional scaling, too. In this case, for older KDE versions the value returned by the procedure will always be `100`, regardless of the display's scaling level, while for newer KDE versions it will be `100`, `125`, `150`, `175`, or `200`.

On GNOME and Budgie, the display scaling can be set in the *Displays* page of the *Settings* dialog, but the value of the X resource `Xft.dpi`, which is used by the procedure `scaleutil::scalingPercentage`, can also be manipulated by setting the scaling factor of the fonts via the *Tweaks* application. Likewise, on Cinnamon the display scaling can be set in the *Display* page of the *System Settings* dialog, but the value of the X resource `Xft.dpi` can also be manipulated by setting the text scaling factor via the *Font Selection* page of the *System Settings* dialog. Finally, on KDE Plasma the display scaling can be set in the *Display Configuration* page of the *System Settings* dialog, and on KDE Plasma on Xorg the value of the font DPI, given by the X resource `Xft.dpi` can also be set via the *Fonts* page of the *System Settings* dialog.

Besides computing the scaling percentage, the procedure `scaleutil::scalingPercentage` performs a series of additional tasks. Among others, it changes the default height of the ttk::treeview rows from the hard-coded value of 20 pixels to a more reasonable one, based on the metrics of the font used by the `Treeview` style (usually `TkDefaultFont`), and makes sure that this step will be repeated whenever the virtual event `<<ThemeChanged>>` is received (e.g., because the value of the `Treeview` style's `-font` option has changed), or the virtual event `<<TkWorldChanged>>` with the `user_data` field (%d) set to `FontChanged` is received. If the Tk version is later than 8.6.11 or 8.7a5, this virtual event is sent by the Tk engine to all widgets when a font is changed, for example, by invoking `font configure`(see TIP 608).

The additional steps described in the rest of this section are only performed if the scaling

percentage is greater than `100`.

On X11, the procedure synchronizes the scaling factor used by Tk to convert between physical units and pixels with the scaling percentage, by passing a value derived from the latter to the `tk scaling`command. While in the vast majority of applications this is the desired and recommended behavior, there might be cases where you want to suppress this step because your application might have its own logic for determining the value of Tk's scaling factor. You can achieve this by inserting the line

```
namespace eval ::scaleutil { set keepTkScaling 1 }
```

before loading Scrollutil or Scrollutil_tile via `package require`.

The procedure `scaleutil::scalingPercentage` also corrects the sizes of the standard fonts if needed. These fonts (`TkDefaultFont`, `TkTextFont`, etc.) are defined in the file `$tk_library/ttk/fonts.tcl`. For quite a long time, the font sizes for X11 given in this file were sizes in pixels, which was not suitable for use on HiDPI displays. This caused several Linux distributions to bundle patched versions of this file, in which the sizes in pixels are replaced with sizes in points. The same fix was committed in February 2020 into the Tk core repository and is now contained in Tk 8.7a5 and later. To make sure that, regardless of the Tk version, the font sizes will suit the display's scaling level, the procedure examines this library file and, if the latter contains sizes in pixels, then it sets the `-size` option of the standard fonts to corresponding sizes in points (without altering the file). In addition, for the "2x" mode on Xfce, the procedure doubles the sizes (in points) of the standard fonts (the way display scaling works on that desktop makes this necessary).

The procedure also scales:

- the default parameters of the Tk core panedwindow sash;
- the default size of the Tk core scale widget and its slider;
- the default width of the Tk core scrollbars on X11;
- the default length of the ttk::scale and ttk::progressbar widgets;
- for the built-in themes `alt` and `default`, some styling options of the ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, and ttk::treeview widgets;
- for the built-in theme `clam`, some styling options of the ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, ttk::treeview, and ttk::labelframe widgets;
- for the built-in theme `classic`, some styling options of the ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, and ttk::treeview widgets;
- for the built-in theme `vista`, some styling options of the ttk::combobox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, and ttk::treeview widgets;
- for the built-in theme `winnative`, some styling options of the ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, and ttk::treeview widgets;
- for the built-in theme `xpnative`, some styling options of the ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook,

ttk::panedwindow, and ttk::treeview widgets.

In addition, the procedure `scaleutil::scalingPercentage` makes sure that in the `vista` and `xpnative` themes the indicators of the ttk::checkbutton and ttk::radiobutton widgets will appear properly scaled, regardless of the Tk release being used. (A long-standing bug in the implementation of these widgets was fixed in May 2020 and is now contained in both Tk 8.6.11 and later and 8.7a5 and later, but the procedure provides an automatic workaround for the Tk versions that are still affected by this bug.)

Note that in Tk versions earlier than 9, the Tk core checkbutton and radiobutton widgets, as well as the indicators of the ttk::checkbutton and ttk::radiobutton widgets of the `alt` and `winnative` themes are not scalable (they scale automatically in Tk 9 and later). The same holds true for the ttk::sizegrip widget in the built-in themes.

The Procedure `scrollutil::getScalingPct` and Tk's Scaling Factor

The Scrollutil package provides the helper procedure `scrollutil::getScalingPct`, which expects no arguments and returns the application's scaling percentage, derived from Tk's *current* scaling factor given by `[tk scaling]`. The procedure's return value is restricted to multiples of `25`, is at least `100`, and can be greater than `200`, especially when running AndroWish on a tablet or smartphone. Scrollutil makes use of this procedure to make sure that the sizes of its GUI components will correspond to Tk's current scaling factor (which might have been changed via `tk scaling`), and it is recommended to do this in the application code, too.

If the Tk version is 9 or later then the functionality of the `scaleutil::scalingPercentage` procedure described in the previous section is integrated in the core. As part of this functionality, Tk 9+ sets the variable `tk::scalingPct` at initialization time to the scaling percentage corresponding to the display's DPI scaling level, and the procedure `scaleutil::scalingPercentage` just returns the minimum of the current value of `tk::scalingPct` and `200`. Recall that in Tk 9+, whenever the scaling factor is changed via `tk scaling`, the value of the variable `tk::scalingPct` is automatically updated. Consequently, if the application changes Tk's scaling factor *before loading Scrollutil or Scrollutil_tile* via `package require`, then the variable `scrollutil::scalingpct` will be set to the minimum of the automatically updated value of `tk::scalingPct` and `200`. These facts result in the following

Recommendation: If your application changes Tk's scaling factor by invoking `tk scaling`, it should do this *before loading Scrollutil or Scrollutil_tile*.

More on `scrollutil::svgfmt`

If the Tk version is either at least 8.7 (with built-in SVG support), or 8.6 and the tksvg package can be loaded into the interpreter, then Scrollutil sets the variable `scrollutil::svgfmt` to

```
[list svg -scale [expr {[scrollutil::getScalingPct] / 100.0}]]
```

Typical values are `{svg -scale 1.0}`, `{svg -scale 1.25}`, `{svg -scale 2.0}`, etc.

It is recommended to pass the value of the variable `scrollutil::svgfmt` to the commands `image create photo`, `imageName configure`, `imageName put`, and `imageName read` as the value of their `-format` option when creating or manipulating SVG images, to make sure that their sizes will correspond to the application's current scaling percentage.

Examples

The Helper Script `styleUtil.tcl`

All the examples in the `demos` directory use tile (ttk) widgets and contain the lines

```
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]
```

The script `styleUtil.tcl` starts with a comment related to the `-autohidescrollbars` and `-setfocus` scrollarea configuration options:

```
#
# To set the "-autohidescrollbars" or "-setfocus" option of all
    scrollarea
# widgets in all demo scripts to true, uncomment the corresponding
    line below:
#
# option add *Scrollarea.autoHideScrollbars 1
# option add *Scrollarea.setFocus           1
```

You are free to follow this hint or to run the demo scripts with the default `-autohidescrollbars 0`and `-setfocus 0`scrollarea settings.

The script patches a few ttk widget styles and defines the style `Small.Toolbutton`.

The patch for the style `TCombobox` makes sure that the (readonly) ttk::combobox widgets of the themes `alt`, `clam`, `classic`, and `default` will show whether they have the focus. This basic requirement, which makes the keyboard navigation more user-friendly, is already fulfilled by the themes `vista`, `xpnative`, and `aqua`.

The ttk::button widgets of the style `Small.Toolbutton` created by the procedure `styleutil::createToolbutton`, implemented in this helper script, will appear raised when they have the focus. Again, this makes the keyboard navigation more user-friendly.

Yet another procedure provided by this script is `styleutil::getCurrentTheme`, which returns the name of the current theme by invoking the `ttk::style theme use`command. If an old tile version is being used which doesn't yet support this method then the procedure returns the value of the variable `::ttk::currentTheme` (which is set by the `ttk::setTheme` command only). This procedure is used by nearly all demo scripts included in the Scrollutil distribution.

The script also patches the `clam` theme, by invoking the public procedure `themepatch::patch` of the themepatch package, which is bundled with Scrollutil (but, contrary to the scaleutil package, is not used by the Scrollutil code and therefore it needs to be loaded explicitly via `package require themepatch`):

```
package require scrollutil_tile

. . .

#
# Patch the clam theme's styles TButton, TMenubutton,
```

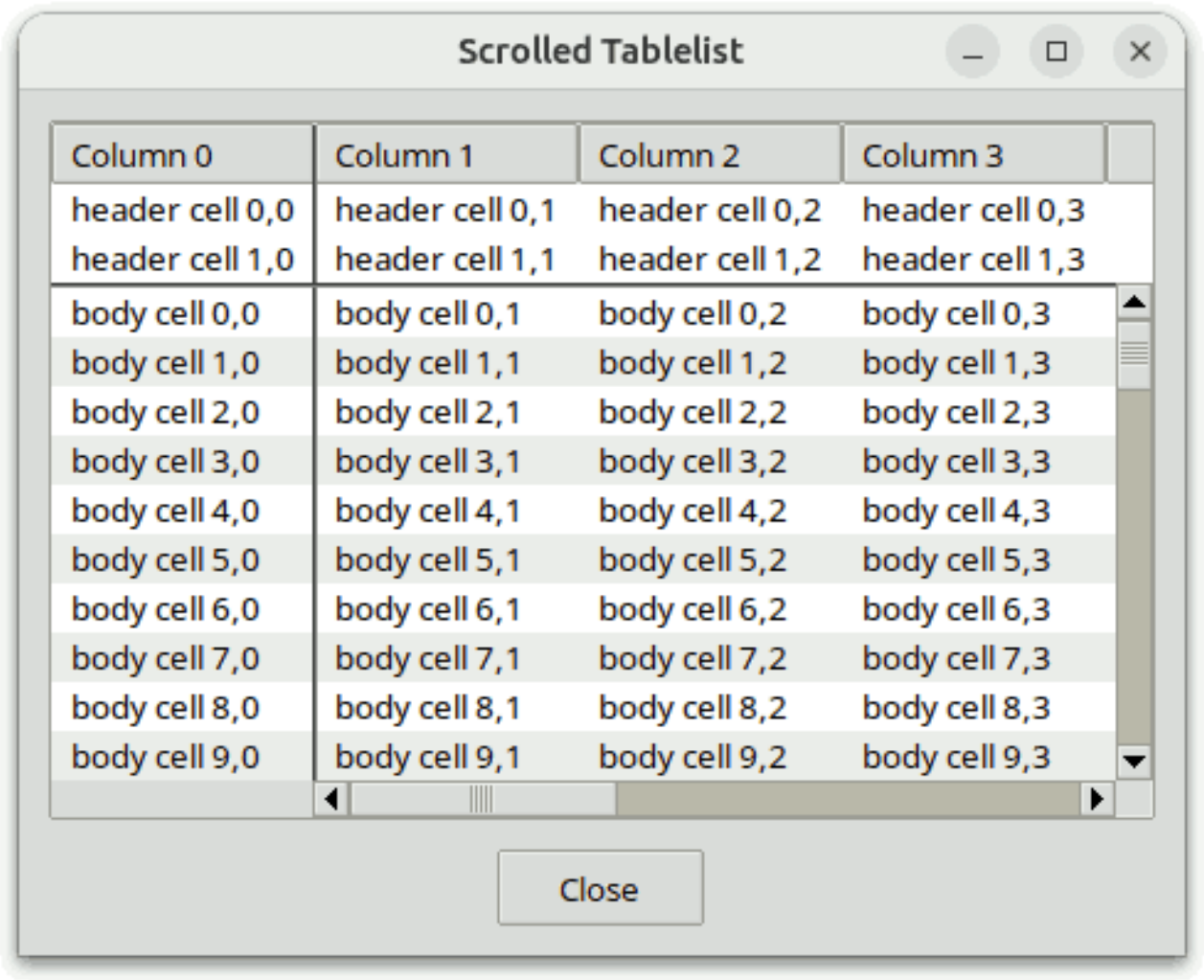
```
# Heading, TCheckbutton, and TRadiobutton
#
package require themepatch
themepatch::patch clam
```

On X11, the script sets the theme to this patched variant of the `clam` theme, which has smaller ttk::button widgets as well as ttk::treeview and tablelist headers, and its scalable ttk::checkbutton and ttk::radiobutton widgets have a significantly improved look and behavior.

A Scrolled tablelist Widget

This example shows how you can greatly simplify the creation of a scrolled tablelist by using a scrollarea widget.

The script `ScrolledTablelist1.tcl` in the `demos` directory creates a horizontally and vertically scrolled tablelist widget having two header rows and one title column, and manages the two scrollbars in such a way that the vertical scrollbar appears below the tablelist's header and the horizontal one starts to the right of the widget's title column area:



ScrolledTablelist

The script achieves these requirements using traditional scrollbar management, which is shown below in red color:

```
package require Tk 8.5-
package require tablelist_tile 6.3-
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Tablelist"

#
# Create the tablelist and the scrollbars as children
# of a frame having -borderwidth 1 and -relief sunken
#
set f   [ttk::frame .f]
set frm [ttk::frame $f.frm -borderwidth 1 -relief sunken]
set tbl $frm.tbl
set vsb $frm.vsb
set hsb $frm.hsb
tablelist::tablelist $tbl ... -borderwidth 0 \
    -xscrollcommand [list $hsb set] -yscrollcommand [list $vsb set]
. . .
ttk::scrollbar $vsb -orient vertical   -command [list $tbl yview]
ttk::scrollbar $hsb -orient horizontal -command [list $tbl xview]

. . .

#
# Manage the widgets within the frame
#
grid $tbl -row 0 -rowspan 2 -column 0 -columnspan 2 -sticky news
if {[tk windowingsystem] eq "win32"} {
    grid $vsb -row 0 -rowspan 2 -column 2 -sticky ns
} else {
    grid [$tbl cornerpath] -row 0 -column 2 -sticky ew
    grid $vsb              -row 1 -column 2 -sticky ns
}
grid [$tbl cornerpath -sw] -row 2 -column 0 -sticky ns
grid $hsb                  -row 2 -column 1 -sticky ew
grid rowconfigure    $frm 1 -weight 1
grid columnconfigure $frm 1 -weight 1

. . .

#
# Manage the frame
#
pack $frm -expand yes -fill both -padx 9p -pady 9p
pack $f   -expand yes -fill both
```

The script `ScrolledTablelist2.tcl` in the `demos` directory replaces the rather technical code above with just a few lines (shown below in red color), by embedding the tablelist into a scrollarea widget. It requires Tablelist version 6.5, which is needed so the `-respectheader` and `-respecttitlecolumns` scrollarea options can work as expected (for earlier Tablelist versions these options are silently ignored). As a further benefit, the scrollbars created with this method will have the default display mode `dynamic`.

```
package require Tk 8.5-
package require tablelist_tile 6.5-
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Tablelist"

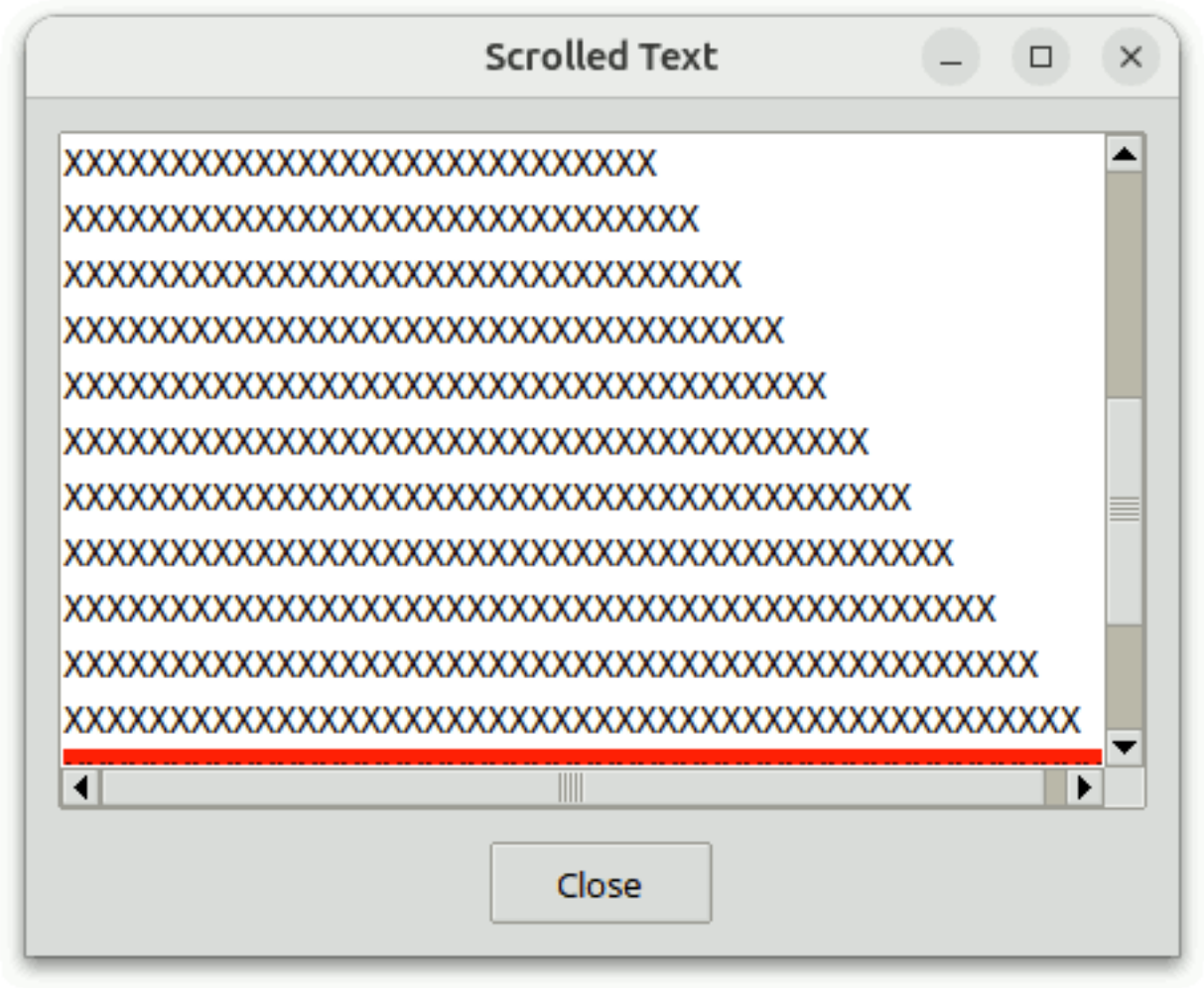
#
# Create the tablelist within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set tbl $sa.tbl
tablelist::tablelist $tbl ...
. . .
$sa setwidget $tbl

. . .

#
# Manage the scrollarea
#
pack $sa -expand yes -fill both -padx 9p -pady 9p
pack $f  -expand yes -fill both
```

A Scrolled text Widget

The script `ScrolledText.tcl` in the `demos` directory shows how the scrollarea widget circumvents the potential shimmering effect in connection with text widgets.



ScrolledText

Here is the relevant code, in which the lines related to the scrollarea widget are shown in red color:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Text"

#
# Create a text widget within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa -lockinterval 10]
set txt [text $sa.txt -font TkFixedFont -width 49 -height 12 \
         -spacing1 1.5p -spacing3 1.5p -wrap none]
$sa setwidget $txt

#
# Populate the text widget and set the background color of line #25 to
```

```
    red
#
for {set i 1} {$i <= 30} {incr i} {
    set j [expr {2*$i}]
    $txt insert end [string repeat x $j]\n
}
$txt delete 30.end
$txt tag configure bgRed -background red
$txt tag add bgRed 25.0 25.end

. . .

#
# Manage the scrollarea
#
pack $sa -expand yes -fill both -padx 9p -pady 9p
pack $f  -expand yes -fill both

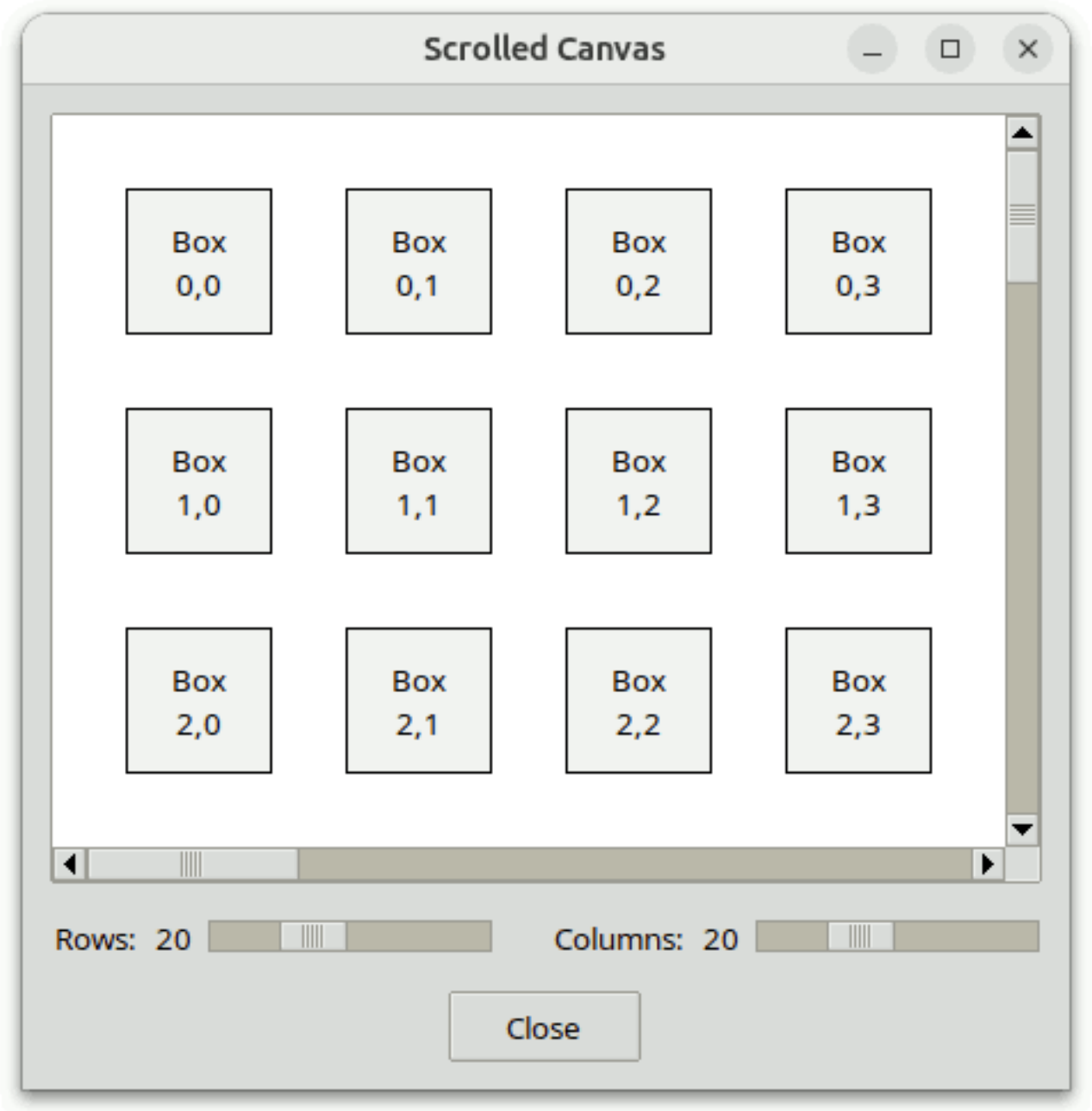
#
# Adjust the vertical view in the text window
# so that line #25 becomes the bottom line
#
tkwait visibility $txt
after 100 [list $txt yview 14.0]
```

The script creates a text widget `$txt` embedded into a scrollarea, populates it with 30 lines, and adjusts the vertical view in the text window so that line #25 becomes the bottom line. This line has 50 characters, hence it doesn't fit completely into the window, whose width is 49 characters. Consequently, the command `$txt xview`will return the list `{0.0 0.98}`, hence the scrollarea's horizontal scrollbar will be mapped and will obscure most part of the bottom line. Since this line has `red` background, it is easy to see how much of it sticks out above the upper edge of the scrollbar.

Let's analyze what happens if the text widget's height is decreased by dragging the main window's upper or lower edge, just until the red pixels get obscured by the horizontal scrollbar. After performing this action, line #25 is completely out of view and the new bottom line is line #24, which has 48 characters, hence the command `$txt xview`will return `{0.0 1.0}`. Normally, this would cause the horizontal scrollbar to be unmapped. However, that would make line #25 to the bottom line, thus causing the horizontal scrollbar to be mapped again. This time the scrollbar would completely obscure this line, which would result in line #24 to become the bottom line, which would cause the scrollbar to be unmapped again, and so on. In other words, the horizontal scrollbar would get mapped and unmapped in an endless loop, giving rise to an annoying flickering effect. The built-in locking mechanism of the scrollarea widget guards against such potential endless loops. To make sure that the locking will work as expected, we have set the `-lockinterval` scrollarea option to `10` (recall that the default value is `1`).

A Scrolled canvas Widget

The script `ScrolledCanvas.tcl` in the `demos` directory shows how to use the Scrollutil package for adding scrollbars as well as mouse wheel and `<TouchpadScroll>` event support to a canvas widget. Further, it shows how to add mouse wheel and `<TouchpadScroll>` event support to ttk::scale widgets.



ScrolledCanvas

Here is the relevant code, in which the lines related to the Scrollutil package are shown in red color:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Canvas"
```

```
set scaleFactor [expr {[scrollutil::getScalingPct] / 100.0}]
set width  [expr {13 * 32 * $scaleFactor}]
set height [expr {10 * 32 * $scaleFactor}]
set scrlIncr [expr {16 * $scaleFactor}]

#
# Create a canvas widget within a scrollarea and add mouse wheel
    support to it
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set c  [canvas $sa.c -background white -width $width -height $height \
        -xscrollincrement $scrlIncr -yscrollincrement $scrlIncr]
bind $c <Configure> { setScrollRegion %W %w %h }
scrollutil::addMouseWheelSupport $c
$sa setwidget $c

set rows 20
set cols 20

#
# Populate the canvas and then rescale the coordinates
# of all of the items by a factor of $scaleFactor
#
proc populate canv {
    $canv delete all
    global rows cols scaleFactor

    for {set row 0; set y 32} {$row < $rows} {incr row; incr y 96} {
        for {set col 0; set x 32} {$col < $cols} {incr col; incr x 96}
    {
            $canv create rectangle $x $y [expr {$x+63}] [expr {$y+63}]
    \
                -fill gray95
            $canv create text [expr {$x+32}] [expr {$y+32}] \
                -text "Box\n$row,$col" -anchor center -justify center
        }
    }

    $canv scale all 0 0 $scaleFactor $scaleFactor
}
populate $c

#
# Sets the scroll region of the canvas.
```

```
#
proc setScrollRegion {canv width height} {
    global cols rows scaleFactor
    set rightX [expr {($cols*96 + 32) * $scaleFactor}]
    set lowerY [expr {($rows*96 + 32) * $scaleFactor}]
    if {$rightX < $width}  { set rightX $width }
    if {$lowerY < $height} { set lowerY $height }
    $canv configure -scrollregion [list 0 0 $rightX $lowerY]
}

. . .

#
# Create a ttk::scale widget for setting the number of box rows
#
set fRows [ttk::frame $f.fRows]
pack [ttk::label $fRows.l1 -text "Rows:"] -side left -padx {0 3p}
pack [ttk::label $fRows.l2 -textvariable rows] -side left -padx {0 3p}
pack [ttk::scale $fRows.scl -from 5 -to 50 -value $rows -command \
      [list setBoxRows $fRows.scl]] -side left -expand yes -fill x

proc setBoxRows {scl val} {
    global rows c
    set val [expr {round($val)}]
    if {$val != $rows} {
        set rows $val
        populate $c
        setScrollRegion $c [winfo width $c] [winfo height $c]
    }
}

#
# Create a ttk::scale widget for setting the number of box columns
#

. . .

#
# Add mouse wheel support to the ttk::scale widgets
#
scrollutil::addMouseWheelSupport TScale

. . .
```

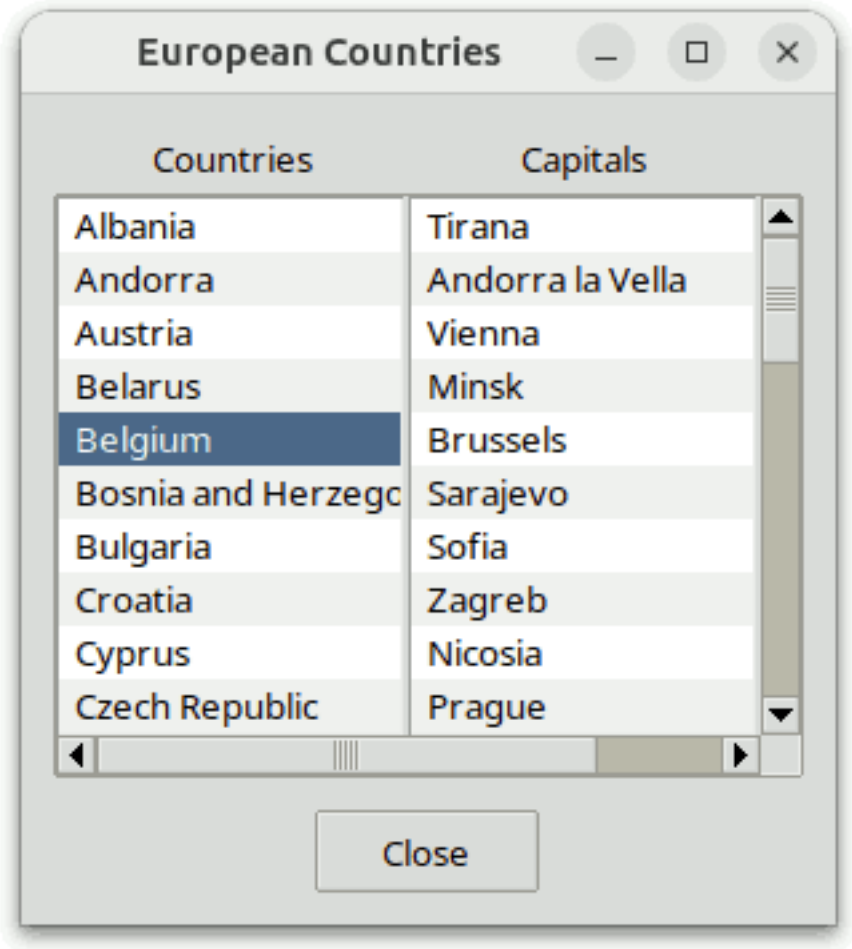
The script creates a canvas widget, populates it with `rectangle` and `text` items, and adds mouse wheel and `<TouchpadScroll>` event support to it by passing its name to the

`scrollutil::addMouseWheelSupport` command. Note that in the canvas we work with pixels rather than points, but make the GUI fully scaling-aware with the aid of a variable `scaleFactor`, whose value (`1.0`, `1.25`, `1.5`, etc.) is derived from `[scrollutil::getScalingPct]`.

The script also creates two ttk::scale widgets, used for setting the numbers of box rows and columns in the canvas, and adds mouse wheel and `<TouchpadScroll>` event support to the ttk::scale widgets by passing the widget class `TScale` to the `scrollutil::addMouseWheelSupport` command.

Synchronizing Two listbox Widgets

The script `SyncListboxes.tcl` in the `demos` directory creates two listboxes within a scrollsync widget, which in turn is embedded into a scrollarea.



SyncListboxes

Here is the relevant code, in which the lines related to the scrollarea and scrollsync widgets are shown in red color:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "European Countries"
```

```
. . .

set f  [ttk::frame .f]

. . .

#
# Create a scrollsync widget within a scrollarea
#
set sa [scrollutil::scrollarea $f.sa]
set ss [scrollutil::scrollsync $sa.ss]
$sa setwidget $ss

#
# Populate the scrollsync widget with two listboxes
#

. . .

set lb1 [listbox $ss.lb1 -activestyle none -highlightthickness 0
    -width 16]
set lb2 [listbox $ss.lb2 -activestyle none -highlightthickness 0
    -width 16]
$ss setwidgets [list $lb1 $lb2]

. . .

grid $lb1 $lb2 -sticky news -padx {0 1.5p}
grid rowconfigure    $ss 0 -weight 1
grid columnconfigure $ss 0 -weight 1
grid columnconfigure $ss 1 -weight 1

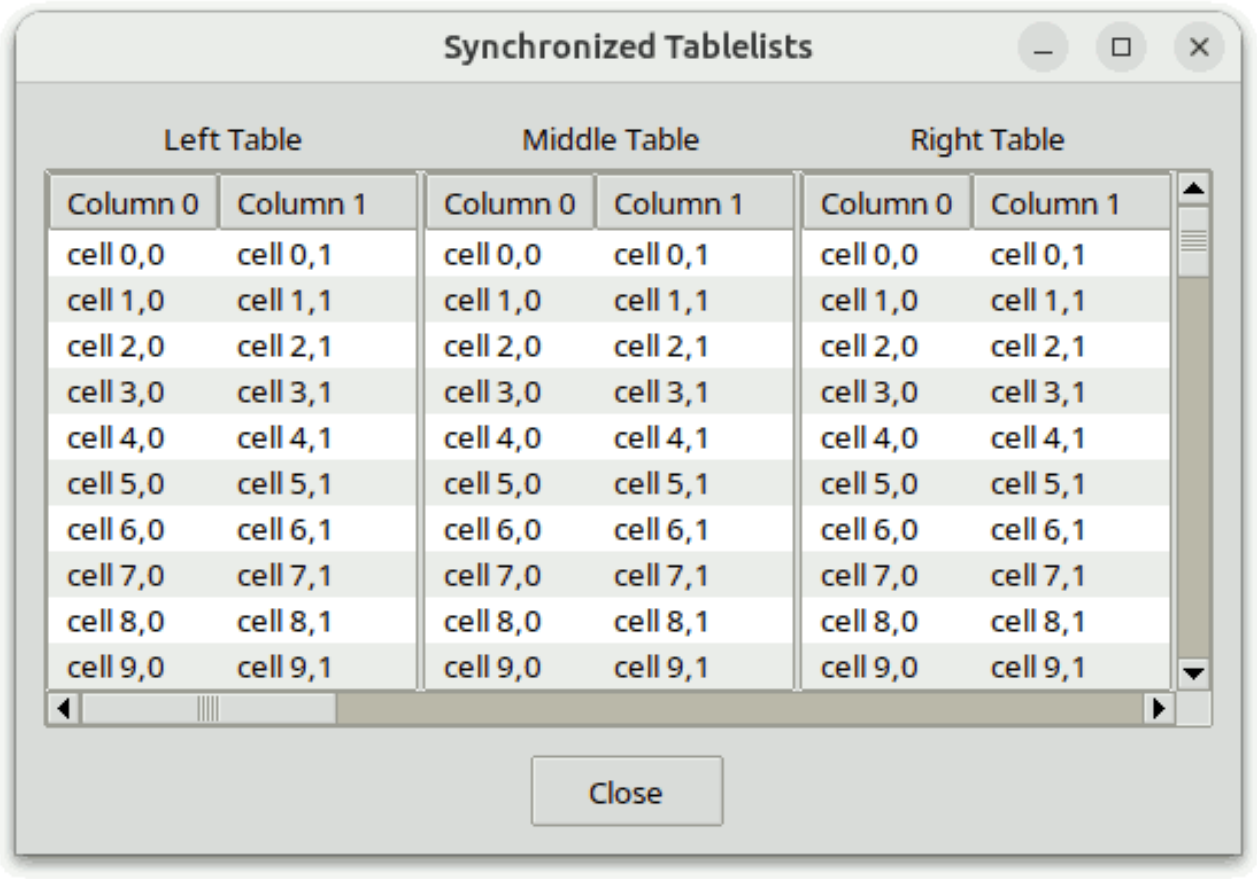
. . .

pack $sa -side top -expand yes -fill both -padx 9p -pady {1.5p 9p}
pack $f  -expand yes -fill both

. . .
```

Synchronizing Three tablelist Widgets

The script `SyncTablelists.tcl` in the `demos` directory creates three tablelists within a scrollsync widget, which in turn is embedded into a scrollarea.



SyncTablelists

The relevant code is similar to the one shown in the previous example:

```
package require tablelist_tile
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Synchronized Tablelists"

. . .

set f  [ttk::frame .f]

. . .

#
# Create a scrollsync widget within a scrollarea
#
set sa [scrollutil::scrollarea $f.sa]
set ss [scrollutil::scrollsync $sa.ss]
$sa setwidget $ss
```

```
#
# Populate the scrollsync widget with three tablelists
#

tablelist::setThemeDefaults
if {$tablelist::themeDefaults(-stripebackground) eq ""} {
    option add *Tablelist.background            white
    option add *Tablelist.stripeBackground      #f0f0f0
}

for {set n 1; set colWidth 40} {$n <= 3} {incr n; incr colWidth 20} {
    set tbl [tablelist::tablelist $ss.tbl$n \
             -columns [list 0 "Column 0" left  $colWidth "Column 1"
    left]]
    set tbl$n $tbl

    for {set i 0} {$i < 40} {incr i} {
        $tbl insert end [list "cell $i,0" "cell $i,1"]
    }
}
$ss setwidgets [list $tbl1 $tbl2 $tbl3]

grid $tbl1 $tbl2 $tbl3 -sticky news -padx {0 1.5p}
grid rowconfigure    $ss 0 -weight 1
grid columnconfigure $ss 0 -weight 1
grid columnconfigure $ss 1 -weight 1
grid columnconfigure $ss 2 -weight 1

. . .

pack $sa -side top -expand yes -fill both -padx 9p -pady {1.5p 9p}
pack $f  -expand yes -fill both

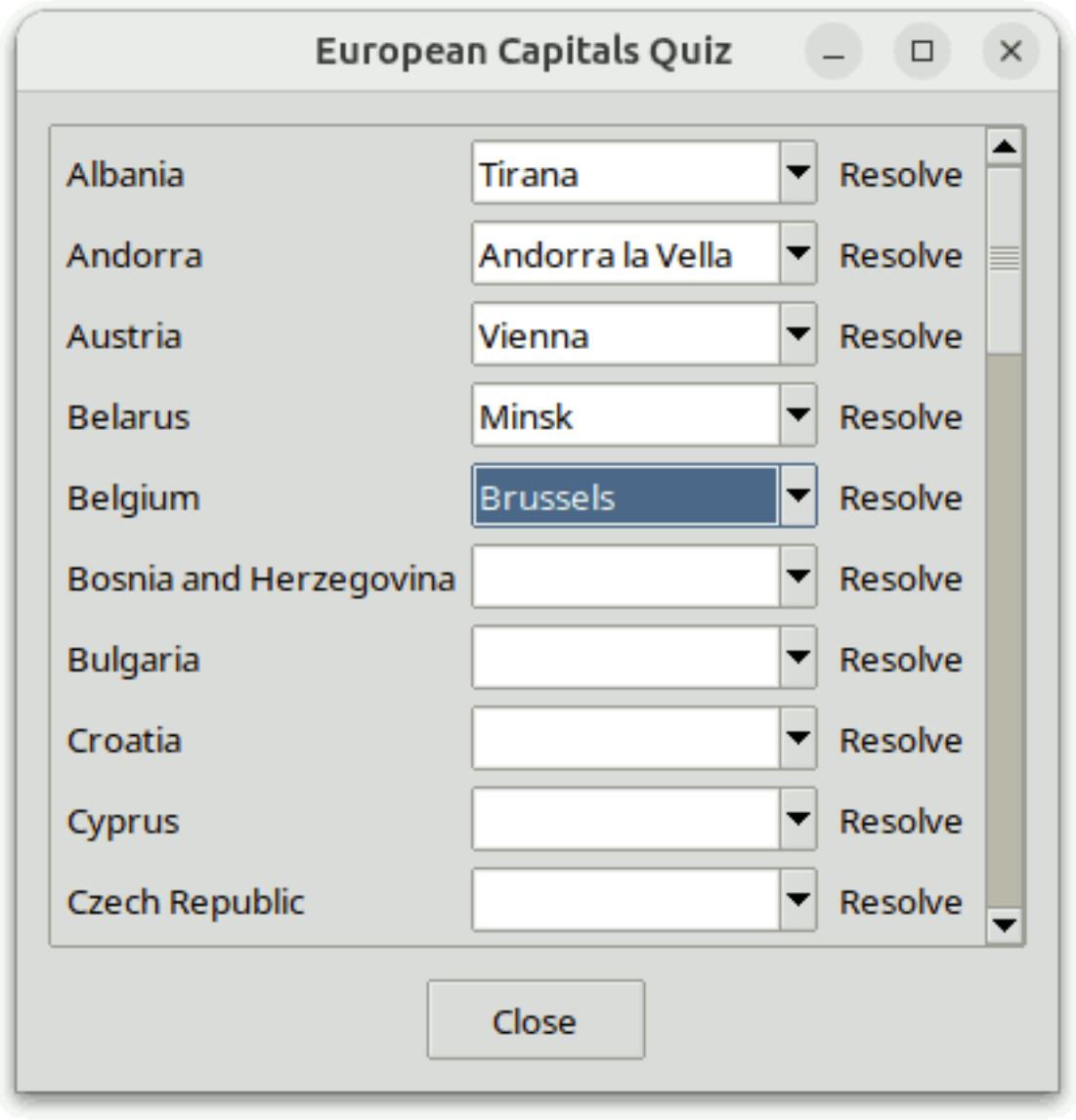
. . .
```

The option database settings above for the `-background` and `-stripebackground` tablelist configuration options are not present in case of the `aqua` theme, because for this theme the default values of these options are not only `aqua`-specific, but in addition on Mac OS 10.14 (Mojave) and later they also depend on the current system appearance (Light Mode or Dark Mode).

Notice that column #1 of the three tablelist widgets is 40, 60, and 80 characters wide, respectively. For this reason, when scrolling horizontally to the right, the left table's view will reach its horizontal end position first, then that of the midde table, and as last one the view of the right table.

A Script Using a scrollutil::scrollableframe Widget

The script `SuScrollableFrmDemo1.tcl` in the `demos` directory creates a scrollutil::scrollableframe widget embedded into a scrollarea and creates mouse wheel and `<TouchpadScroll>` event bindings for the binding tag `"all"` with the aid of the `scrollutil::createWheelEventBindings` command. Recall that the scrollableframe was automatically registered for scrolling by these bindings at creation time, hence there is no need to invoke the `scrollutil::enableScrollingByWheel` command for it again. After that, the script populates the content frame of the scrollableframe with ttk::label widgets displaying the names of the European countries, ttk::combobox widgets for selecting the corresponding capital cities, and ttk::button widgets of the style `Small.Toolbutton` (created by using the procedure `styleutil::createToolbutton`, implemented in the file `styleUtil.tcl`) for the less patient users, displaying the text "Resolve".



ScrollableFrmDemo1

Here is the relevant code:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "European Capitals Quiz"

#
# Create a scrollableframe within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set sf [scrollutil::scrollableframe $sa.sf]
$sa setwidget $sf

#
# Create mouse wheel event bindings for the binding tag "all"
#
scrollutil::createWheelEventBindings all

#
# Get the content frame and populate it
#

set cf [$sf contentframe]

set countryList {
    Albania Andorra Austria Belarus Belgium "Bosnia and Herzegovina"
    Bulgaria
    . . .
}
set capitalList {
    Tirana "Andorra la Vella" Vienna Minsk Brussels Sarajevo Sofia
    . . .
}

. . .

set capitalList [lsort $capitalList]

. . .

set row 0
foreach country $countryList {
    . . .

    set w [ttk::combobox $cf.cb$row -state readonly -width 14 \
```

```
            -values $capitalList]
    . . .

    #
    # Make the keyboard navigation more user-friendly
    #
    bind $w <<TraverseIn>> [list $sf see %W]

    #
    # Adapt the handling of the mouse wheel and mouse
    # button 2 events for the ttk::combobox widget
    #
    scrollutil::adaptWheelEventHandling $w
    scrollutil::adaptBtn2EventHandling $w

    . . .

    incr row
}

. . .
```

We make the keyboard navigation more user-friendly with the aid of the `see` subcommand of the scrollableframe widget when handling the `<<TraverseIn>>` virtual event for the ttk::combobox and (not shown above) ttk::button widgets. In addition, we invoke the `scrollutil::adaptWheelEventHandling` command for every ttk::combobox widget, which is needed for a user-friendly event handling, being that this widget has built-in bindings for the mouse wheel and `<TouchpadScroll>` events. Due to this command, these events over one of the ttk::combobox widgets will only select the next/previous capital city if the widget has the focus, otherwise they will scroll the scrollableframe.

For every ttk::combobox widget, we also invoke the `scrollutil::adaptBtn2EventHandling` command, thus making sure that the mouse button 2 events sent to the widget will only be handled by the widget itself if it has the focus, otherwise they will result in scanning the scrollableframe's content. During the scan the cursor will take on the shape of a pointing hand, and by draggging the mouse, the content frame will drag at high speed through the window, in the direction the mouse moves.

A Script Using a BWidget ScrollableFrame Widget

The script `BwScrollableFrmDemo1.tcl` in the `demos` directory creates a BWidget ScrollableFrame embedded into a scrollarea widget, creates mouse wheel and `<TouchpadScroll>` event bindings for the binding tag `"all"` with the aid of the `scrollutil::createWheelEventBindings` command, and invokes the `scrollutil::enableScrollingByWheel` command for this ScrollableFrame, thus registering the latter for scrolling by these bindings. After that it populates the content frame of the ScrollableFrame with the same widgets as `SuScrollableFrmDemo1.tcl` in the previous example.

Here is the relevant code:

```
package require BWidget
Widget::theme yes
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "European Capitals Quiz"

#
# Create a ScrollableFrame within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set sf [ScrollableFrame $sa.sf]
$sa setwidget $sf

. . .

#
# Create mouse wheel event bindings for the binding tag "all" and
# register the ScrollableFrame for scrolling by these bindings
#
scrollutil::createWheelEventBindings all
scrollutil::enableScrollingByWheel $sf

#
# Get the content frame and populate it
#

set cf [$sf getframe]

set countryList {
    Albania Andorra Austria Belarus Belgium "Bosnia and Herzegovina"
    Bulgaria
    . . .
}
set capitalList {
    Tirana "Andorra la Vella" Vienna Minsk Brussels Sarajevo Sofia
    . . .
}

. . .

set capitalList [lsort $capitalList]
```

```
. . .

set row 0
foreach country $countryList {
    . . .

    set w [ttk::combobox $cf.cb$row -state readonly -width 14 \
           -values $capitalList]
    . . .

    #
    # Make the keyboard navigation more user-friendly
    #
    bind $w <<TraverseIn>> [list $sf see %W]

    #
    # Adapt the handling of the mouse wheel events for the
    ttk::combobox widget
    #
    scrollutil::adaptWheelEventHandling $w

    . . .

    incr row
}

. . .
```

A Script Using an iwidgets::scrolledframe Widget

The script `ScrolledFrmDemo1.tcl` in the `demos` directory creates an iwidgets::scrolledframe widget, creates mouse wheel and `<TouchpadScroll>` event bindings for the binding tag `"all"` with the aid of the `scrollutil::createWheelEventBindings` command, and invokes the `scrollutil::enableScrollingByWheel` command for this scrolledframe, thus registering the latter for scrolling by these bindings. After that it populates the content frame of the scrolledframe with the same widgets as `SuScrollableFrmDemo1.tcl` and `BwScrollableFrmDemo1.tcl` in the two previous examples.

Here is the relevant code:

```
if {[catch {package require iwidgets} result1] != 0 &&
    [catch {package require Iwidgets} result2] != 0} {
    error "$result1; $result2"
}
set dir [file dirname [info script]]
source [file join $dir scrolledwidgetPatch.itk] ;# adds ttk::scrollbar
    widgets
```

```
package require scrollutil_tile
source [file join $dir styleUtil.tcl]

wm title . "European Capitals Quiz"

. . .

#
# Create a scrolledframe
#
set f  [ttk::frame .f]
set sf [iwidgets::scrolledframe $f.sf -borderwidth 1 -relief sunken \
        -scrollmargin 0]
. . .

#
# Create mouse wheel event bindings for the binding tag "all"
# and register the scrolledframe for scrolling by these bindings
#
scrollutil::createWheelEventBindings all
scrollutil::enableScrollingByWheel $sf

#
# Get the content frame and populate it
#

set cf [$sf childsite]
. . .

<exactly as in the previous example, except the stuff related to
    keyboard navigation>

. . .
```

The code related to keyboard navigation is not present in this example, because the iwidgets::scrolledframe widget doesn't provide a `see` subcommand.

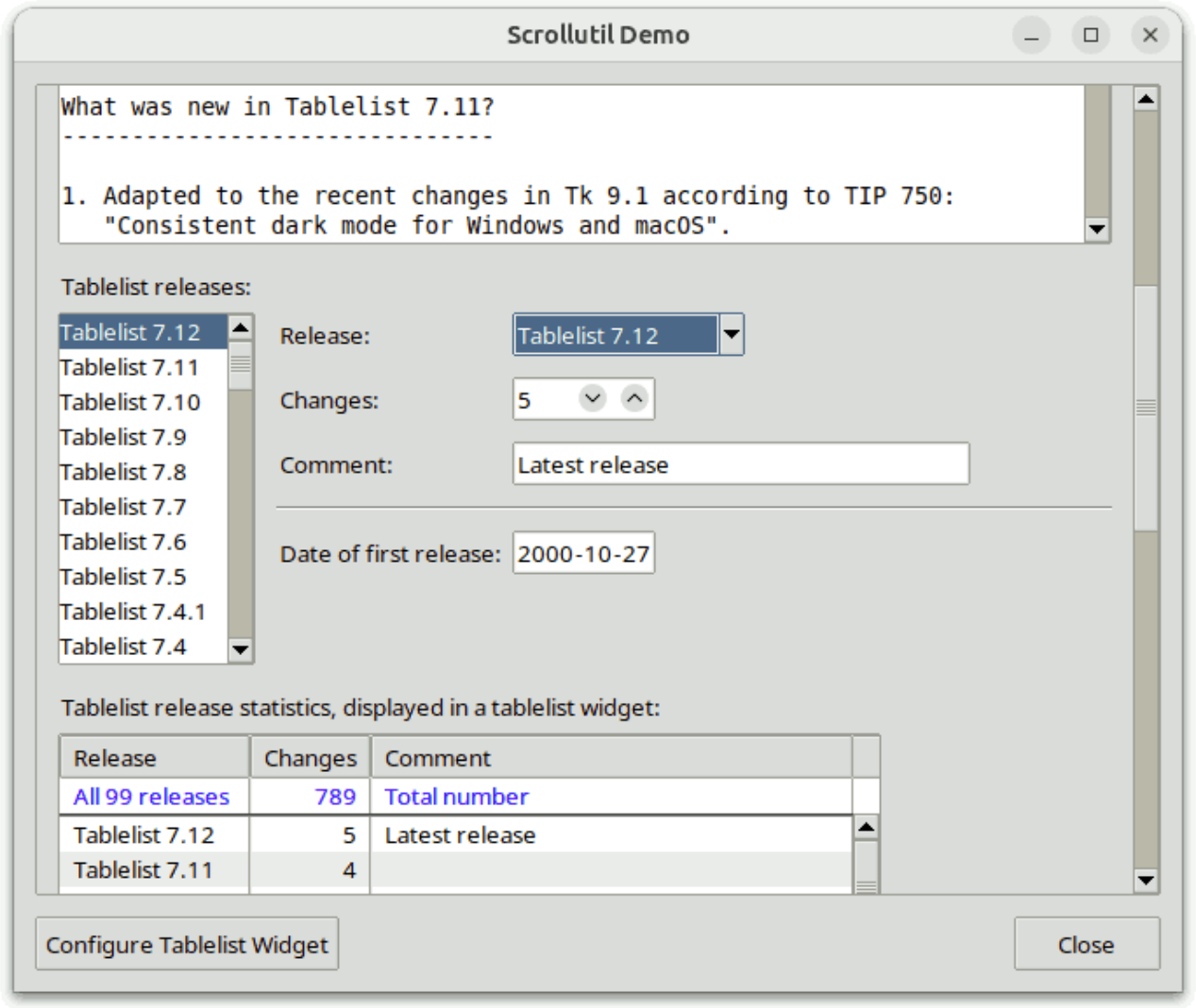
A Script Using Two scrollutil::scrollableframe Widgets

The script `SuScrollableFrmDemo2.tcl` in the `demos` directory creates a scrollutil::scrollableframe widget embedded into a scrollarea and then `sources` the script `SuScrollableFrmContent.tcl`, which populates the content frame of the scrollableframe with the following widgets:

- a series of ttk::label widgets;
- a scrolled text widget `$txt` within a scrollarea;
- a scrolled listbox widget `$lb` within a scrollarea;
- a ttk::combobox widget `$cb`;

- a ttk::spinbox widget `$sb`;
- a ttk::entry widget `$e`;
- a ttk::separator widget;
- a mentry widget `$me` of type `"Date"`;
- a scrolled tablelist widget `$tbl` within a scrollarea;
- a scrolled ttk::treeview widget `$tv` within a scrollarea.

With the exception of ttk::label and ttk::separator, all these widgets have bult-in mouse wheel and `<TouchpadScroll>` event bindings. And with the exception of ttk::label, ttk::separator, and ttk::treeview, all these widgets have bult-in mouse button 2 event bindings.



ScrollableFrmDemo2

Here is the relevant code:

```
package require Tk 8.5.9-                       ;# for ttk::spinbox
package require mentry_tile 3.2-                ;# for mouse wheel
    support
package require tablelist_tile 6.5-             ;# for
    -(x|y)mousewheelwindow
                                                ;# and
```

```
    scrollutil::scrollarea
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrollutil Demo"

#
# Create a scrollableframe within a scrollarea
#
set tf [ttk::frame .tf]
set sa [scrollutil::scrollarea $tf.sa]
set sf [scrollutil::scrollableframe $sa.sf]
$sa setwidget $sf

#
# Get the content frame and populate it
#
set cf [$sf contentframe]
source [file join $dir SuScrollableFrmContent.tcl]

#
# Make the keyboard navigation more user-friendly
#
foreach w [list $cb $sb $e $me] {
    bind $w <<TraverseIn>> [list $sf see %W]
}
foreach w [list $txt $lb $tbl $tv] {
    bind $w <<TraverseIn>> [list seeScrollarea $sf %W]
}
proc seeScrollarea {sf w} { $sf see [scrollutil::getscrollarea $w] }
```

Whenever the `<<TraverseIn>>` virtual event is sent to one of the four widgets created within scrollareas, we query the path name of the corresponding scrollarea via `scrollutil::getscrollarea` and bring that scrollarea (including the scrollbars and the border) into view rather than just the widget in question. While *in this script* we could have used `[winfo parent]`instead, the command `scrollutil::getscrollarea` is the recommended one, being that it works also for widgets that are no children of the corresponding scrollareas.

Here is the additional stuff related to the mouse wheel and `<TouchpadScroll>` events, using the Scrollutil commands described in the What Is Scrollutil? section:

```
#
# For the entry components of the mentry widget
# set the "focus check window" to the mentry
#
set entryList [$me entries]
```

```
scrollutil::setFocusCheckWindow {*}$entryList $me

#
# Create mouse wheel event bindings for the binding tag "all"
#
scrollutil::createWheelEventBindings all

#
# Adapt the handling of the mouse wheel events for the
# text, listbox, ttk::combobox, ttk::spinbox, ttk::entry,
# tablelist, and ttk::treeview widgets, as well as for
# the entry components of the mentry widget of type "Date"
#
### scrollutil::adaptWheelEventHandling $txt $lb $cb $sb $e
    {*}$entryList \
    $tbl $tv
$sf preparescroll
```

Notice that the invocation of the `scrollutil::adaptWheelEventHandling` command is commented out and replaced with the much simpler and safer invocation of the `preparescroll` scrollableframe subcommand.

We also have additional stuff related to the mouse button 2 events:

```
#
# Adapt the handling of the mouse button 2 events for the text,
    listbox,
# ttk::combobox, ttk::spinbox, ttk::entry, and tablelist widgets, as
# well as for the entry components of the mentry widget of type "Date"
#
### scrollutil::adaptBtn2EventHandling $txt $lb $cb $sb $e
    {*}$entryList $tbl
$sf preparescan
```

Again, the invocation of the `scrollutil::adaptBtn2EventHandling` command is commented out and replaced with the much simpler and safer invocation of the `preparescan` scrollableframe subcommand.

As already mentioned, in the file `SuScrollableFrmContent.tcl` the scrolled text, listbox, tablelist, and ttk::treeview widgets are created within scrollarea widgets:

```
set _sa [scrollutil::scrollarea ...]
set txt [text $_sa.txt -font TkFixedFont -width 73]
scrollutil::addMouseWheelSupport $txt
$_sa setwidget $txt
grid $_sa ...

. . .
```

```
set _sa [scrollutil::scrollarea ...]
set lb [listbox $_sa.lb -width 0]
$_sa setwidget $lb
grid $_sa ...

. . .

set _sa [scrollutil::scrollarea ...]
set tbl [tablelist::tablelist $_sa.tbl ...]
. . .
$_sa setwidget $tbl
grid $_sa ...

. . .

set _sa [scrollutil::scrollarea ... -borderwidth 0]
set tv [ttk::treeview $_sa.tv ...]
. . .
$_sa setwidget $tv
grid $_sa ...
```

In the case of the text, listbox, and tablelist widgets we use scrollarea widgets with their default `-borderwidth 1 -relief sunken`settings, which will cause the `setwidget` subcommand of the associated Tcl commands to set the `-borderwidth` option of the text, listbox, and tablelist widgets to `0`. On the other hand, for the ttk::treeview we use a scrollarea widget with `-borderwidth 0`, because the ttk::treeview has a border of width `1` and doesn't support the `-borderwidth` configuration option.

For our text widget we prefer a mouse wheel and `<TouchpadScroll>` event handling that scrolls the widget by lines rather than pixels, as done by the `Text` class bindings in Tk 8.5 and later; we achieve this by passing the path name `$txt` to the `scrollutil::addMouseWheelSupport` command.

The somewhat uncommon look of the ttk::spinbox widget `$sb` displaying the number of changes in a Tablelist release is due to the fact that in the file `SuScrollableFrmContent.tcl` it is created with the `-style` option set to `Wide.TSpinbox` rather than `TSpinbox`:

```
catch {package require wsb}

. . .

set sb [ttk::spinbox $cf.sb -style Wide.TSpinbox -from 0 -to 20 \
        -state readonly -width 4]
```

The small **w**ide **s**pin**b**ox package wsb comes bundled with the Scrollutil distribution, and the attempt to load it via `package require wsb`will be successful if the Tk version is either at least 8.7 (with built-in SVG support), or 8.6 and the tksvg extension can be loaded into the interpreter. If loaded, the wsb package will automatically create the `Wide.TSpinbox` layout.

Otherwise the `-style Wide.TSpinbox`setting for the ttk::spinbox widget will have no effect.

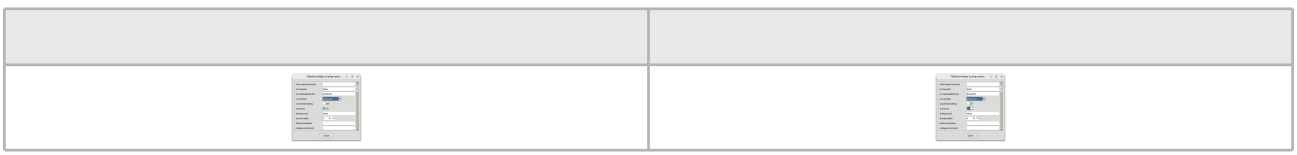
The main benefit of the `Wide.TSpinbox` layout over the `TSpinbox` style is the user-friendly size and placement of its scaling-aware up and down arrows, which comes in handy especially when running AndroWish on a tablet or smartphone:

The file `SuScrollableFrmContent.tcl` implements just a minimal interaction between five of the already mentioned widgets within the content frame: By selecting a Tablelist release from the ttk::combobox `$cb`, the ttk::spinbox `$sb` is set to the corresponding number of changes, the comment associated with that release is inserted into the ttk::entry `$e`, and the corresponding items of the tablelist `$tbl` and ttk::treeview `$tv` are selected and brought into view.

The file `SuScrollableFrmContent.tcl` contains also the implementation of the procedure `configTablelist`, associated with the "Configure Tablelist Widget" button as the value of its `-command` option. This procedure opens a toplevel window that contains a scrollutil::scrollableframe widget created with the `-fitcontentwidth yes`setting within a scrollarea. After that it populates the content frame of the scrollableframe with ttk::label, ttk::combobox, ttk::spinbox, ttk::entry, and ttk::checkbutton or toggleswitch widgets used to display and edit the configuration options of the tablelist widget. If the ttk::toggleswitch widget is available (it was added to the core in Tk 9.1a1) then we use this widget rather than checkbuttons; else if the toggle switch widget package Tsw can be loaded into the interpreter then we use tsw::toggleswitch widgets instead. The ttk::spinbox widgets are created with the setting `-style Wide.TSpinbox`.

The procedure handles the `<<TraverseIn>>` virtual event sent to one of the widgets mentioned above with the aid of the scrollableframe's `see` subcommand. Whenever a ttk::combobox, ttk::spinbox, or ttk::entry is created, the `scrollutil::adaptWheelEventHandling` command is invoked for it, being that these widgets have their own bindings for the mouse wheel and `<TouchpadScroll>` events. Similarly, we pass the ttk::combobox, ttk::spinbox, and ttk::entry widgets to the `scrollutil::adaptBtn2EventHandling` command, because these widgets have built-in bindings for the mouse button 2 events.

The widgets populating the content frame are managed using `grid`. In case of the ttk::entry widgets we invoke `grid` with `-sticky we`. Due to this and the `-fitcontentwidth yes` scrollableframe setting, the ttk::entry widgets will stretch or shrink whenever the width of the scrollableframe changes as a result of resizing the toplevel window.



A Script Using Two BWidget ScrollableFrame Widgets

The script `BwScrollableFrmDemo2.tcl` in the `demos` directory creates a BWidget ScrollableFrame embedded into a scrollarea widget and then `sources` the script `BwScrollableFrmContent.tcl`, which populates the content frame of the ScrollableFrame with the same widgets as `SuScrollableFrmContent.tcl` in the previous example.

Here is the relevant code:

```
package require Tk 8.5.9-                          ;# for ttk::spinbox
package require BWidget
Widget::theme yes
package require mentry_tile 3.2-                   ;# for mouse wheel
    support
package require tablelist_tile 6.5-                ;# for
    -(x|y)mousewheelwindow
                                                   ;# and
    scrollutil::scrollarea
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrollutil Demo"

#
# Create a ScrollableFrame within a scrollarea
#
set tf [ttk::frame .tf]
set sa [scrollutil::scrollarea $tf.sa]
set sf [ScrollableFrame $sa.sf]
$sa setwidget $sf

. . .

#
# Get the content frame and populate it
#
set cf [$sf getframe]
source [file join $dir BwScrollableFrmContent.tcl]

#
# Make the keyboard navigation more user-friendly
#
foreach w [list $cb $sb $e $me] {
    bind $w <<TraverseIn>> [list $sf see %W]
}
foreach w [list $txt $lb $tbl $tv] {
    bind $w <<TraverseIn>> [list seeScrollarea $sf %W]
}
proc seeScrollarea {sf w} { $sf see [scrollutil::getscrollarea $w] }
```

The additional stuff related to the mouse wheel and `<TouchpadScroll>` events contains the same Scrollutil command invocations as the one in the previous example, except that in addition it registers the ScrollableFrame for scrolling with the mouse wheel and touchpad, and invokes the `scrollutil::prepareScrollingByWheel` command:

```
. . .

#
# Create mouse wheel event bindings for the binding tag "all" and
# register the ScrollableFrame for scrolling by these bindings
#
scrollutil::createWheelEventBindings all
scrollutil::enableScrollingByWheel $sf

#
# Adapt the handling of the mouse wheel events for the
# text, listbox, ttk::combobox, ttk::spinbox, ttk::entry,
# tablelist, and ttk::treeview widgets, as well as for
# the entry components of the mentry widget of type "Date"
#
### scrollutil::adaptWheelEventHandling $txt $lb $cb $sb $e
    {*}$entryList \
    $tbl $tv
scrollutil::prepareScrollingByWheel $sf
```

The file `BwScrollableFrmContent.tcl` contains also the implementation of the procedure `configTablelist`, associated with the "Configure Tablelist Widget" button as the value of its `-command` option. This procedure opens a toplevel window that contains a BWidget ScrollableFrame created with the `-constrainedwidth yes` setting within a scrollarea widget and invokes the `scrollutil::enableScrollingByWheel` command for this ScrollableFrame, thus registering the latter for scrolling by the already created mouse wheel and `<TouchpadScroll>` event bindings for the binding tag `"all"`. After that it populates the content frame of the ScrollableFrame with ttk::label, ttk::combobox, ttk::spinbox, ttk::entry, and ttk::checkbutton or toggleswitch widgets used to display and edit the configuration options of the tablelist widget. The ttk::spinbox widgets are created with the setting `-style Wide.TSpinbox`.

The procedure handles the `<<TraverseIn>>` virtual event sent to one of these widgets with the aid of the ScrollableFrame's `see` subcommand. Whenever a ttk::combobox, ttk::spinbox, or ttk::entry is created, the `scrollutil::adaptWheelEventHandling` command is invoked for it, being that these widgets have their own bindings for the mouse wheel and `<TouchpadScroll>` events.

Again, all this is nearly identical to what we did in the previous example.

A Script Using Two iwidgets::scrolledframe Widgets

The script `ScrolledFrmDemo2.tcl` in the `demos` directory creates an iwidgets::scrolledframe widget and then `sources` the file `ScrolledFrmContent.tcl`, which populates the content frame of the scrolledframe with the same widgets as `SuScrollableFrmContent.tcl` and `BwScrollableFrmContent.tcl` in the two previous examples.

Here is the relevant code:

```
package require Tk 8.5.9-                          ;# for ttk::spinbox
if {[catch {package require iwidgets} result1] != 0 &&
    [catch {package require Iwidgets} result2] != 0} {
    error "$result1; $result2"
}
set dir [file dirname [info script]]
source [file join $dir scrolledwidgetPatch.itk] ;# adds ttk::scrollbar
    widgets
package require mentry_tile 3.2-                  ;# for mouse wheel
    support
package require tablelist_tile 6.5-               ;# for
    -(x|y)mousewheelwindow
                                                   ;# and
    scrollutil::scrollarea
package require scrollutil_tile
source [file join $dir styleUtil.tcl]

wm title . "Scrollutil Demo"

. . .

#
# Create a scrolledframe
#
set tf [ttk::frame .tf]
set sf [iwidgets::scrolledframe $tf.sf -borderwidth 1 -relief sunken \
        -scrollmargin 0]
. . .

#
# Get the content frame and populate it
#
set cf [$sf childsite]
. . .
source [file join $dir ScrolledFrmContent.tcl]
```

The additional stuff related to the mouse wheel and `<TouchpadScroll>` events contains the same Scrollutil command invocations as the one in the previous example.

The file `ScrolledFrmContent.tcl` contains also the implementation of the procedure `configTablelist`, associated with the "Configure Tablelist Widget" button as the value of its `-command` option. This procedure opens a toplevel window that contains an iwidgets::scrolledframe widget with a manually implemented equivalent of the `-fitcontentwidth yes` scrollutil::scrollableframe and `-constrainedwidth yes`BWidget ScrollableFrame settings and invokes the `scrollutil::enableScrollingByWheel` command for this scrolledframe, thus registering the latter for scrolling by the already created mouse wheel and `<TouchpadScroll>` event bindings for the binding tag `"all"`. After that it

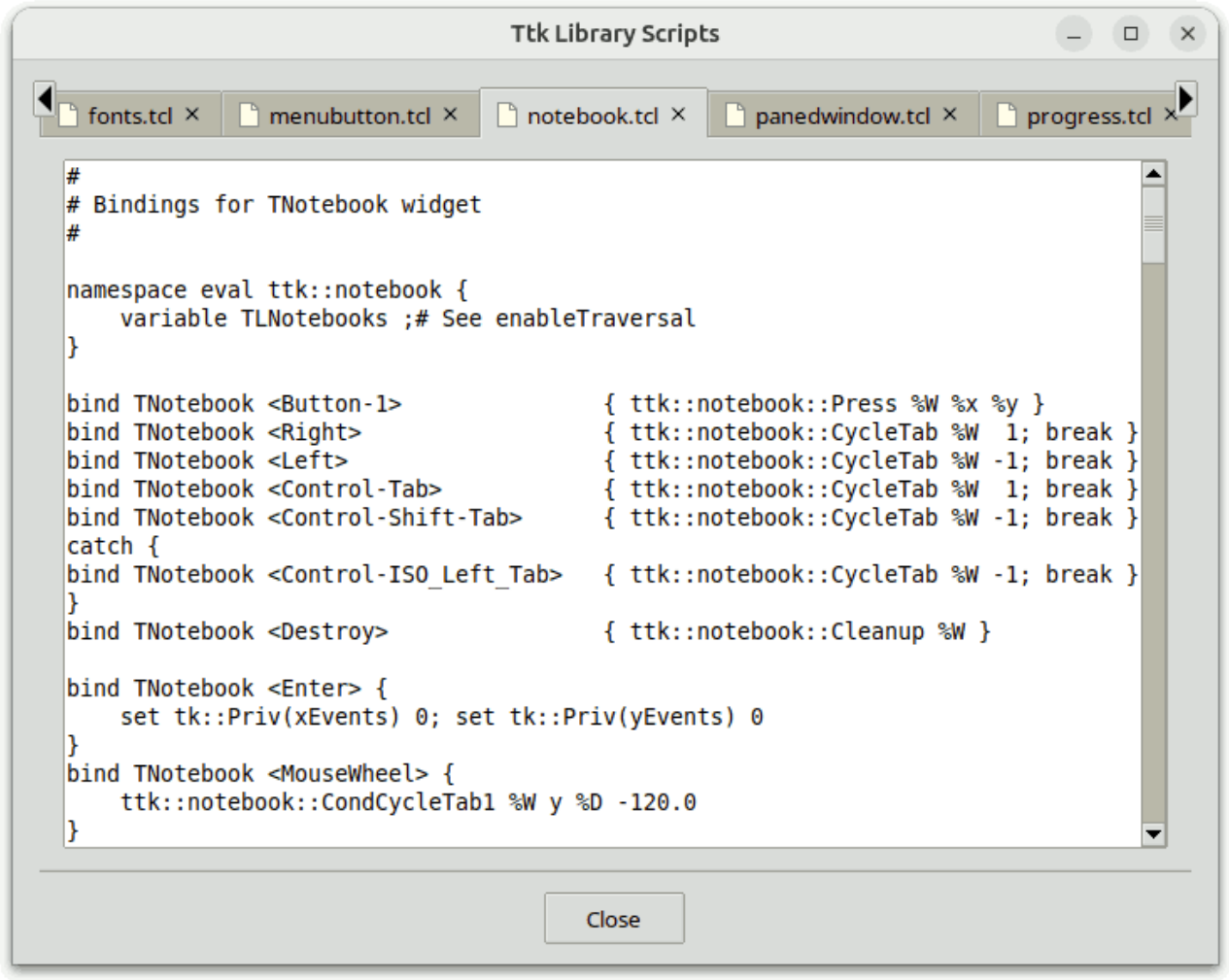
populates the content frame of the scrolledframe with ttk::label, ttk::combobox, ttk::spinbox, ttk::entry, and ttk::checkbutton or toggleswitch widgets used to display and edit the configuration options of the tablelist widget. The ttk::spinbox widgets are created with the setting `-style Wide.TSpinbox`.

Whenever a ttk::combobox, ttk::spinbox, or ttk::entry is created, the `scrollutil::adaptWheelEventHandling` command is invoked for it, being that these widgets have their own bindings for the mouse wheel and `<TouchpadScroll>` events.

Again, all this is nearly identical to what we did in the two previous examples.

A scrollutil::scrollednotebook Demo

The script `ScrolledNotebookDemo.tcl` in the `demos` directory creates a scrollutil::scrollednotebook widget and populates it with panes containing scrollareas that wrap text widgets displaying the contents of the Ttk library files. After that it provides pop-up menu items for left/right moving and closing the panes, and sets the height and width of the scrollednotebook to values computed from the requested height and width of the scrollarea widgets and the padding applied to the panes.



ScrolledNotebookDemo

Here is the relevant code:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Ttk Library Scripts"

scrollutil::addclosetab My.TNotebook

#
# Create an image corresponding to the display's DPI scaling level
#
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set fmt $scrollutil::svgfmt
    image create photo fileImg -file [file join $dir file.svg] -format
    $fmt
} else {
    set pct $scrollutil::scalingpct
    image create photo fileImg -file [file join $dir file$pct.gif]
    -format gif
}

#
# Create a scrollednotebook widget having closable (and, per default,
# movable) tabs and populate it with panes that contain scrolled
# text widgets displaying the contents of the Ttk library files
#
set f  [ttk::frame .f]
set nb [scrollutil::scrollednotebook $f.nb -style My.TNotebook \
        -forgetcommand condCopySel -leavecommand saveSel]
set currentTheme [styleutil::getCurrentTheme]
set panePadding [expr {$currentTheme eq "aqua" ? 0 : "9p"}]
cd [expr {[info exists ttk::library] ? $ttk::library : $tile::library}]
foreach fileName [lsort [glob *.tcl]] {
    set baseName [string range $fileName 0 end-4]
    set sa [scrollutil::scrollarea $nb.sa_$baseName -lockinterval 10]
    $sa configure -yscrollbarmode static        ;# temporarily, for
    adjustsize
    if {$currentTheme eq "vista"} {
        $sa configure -relief solid
    }
    set txt [text $sa.txt -font TkFixedFont -takefocus 1 -wrap none]
    catch {$txt configure -tabstyle wordprocessor}      ;# for Tk 8.5
    and later
    scrollutil::addMouseWheelSupport $txt       ;# old-school wheel
    support
    $sa setwidget $txt
```

```
    set chan [open $fileName]
    $txt insert end [read -nonewline $chan]
    close $chan
    $txt configure -state disabled
    bind $txt <Button-1> { focus %W }   ;# for Tk versions <
    8.6.11/8.7a4

    $nb add $sa -text $fileName -image fileImg -compound left \
                -padding $panePadding
}

proc condCopySel {nb widget} {
    set txt $widget.txt
    if {[llength [$txt tag nextrange sel 1.0 end]] == 0} {
        return 1
    }

    set btn [tk_messageBox -title "Copy Selection?" -icon question \
             -message "Do you want to copy the selection to the
    clipboard?" \
             -type yesnocancel]
    switch $btn {
        yes     { tk_textCopy $txt; return 1 }
        no      { return 1 }
        cancel  { return 0 }
    }
}

proc saveSel {nb widget} {
    set selRange [$widget.txt tag nextrange sel 1.0 end]
    if {[llength $selRange] == 0} {
        $nb unsettabattrib $widget "selRange"
    } else {
        $nb tabattrib $widget "selRange" $selRange
    }

    return 1
}

#
# Create bindings for moving and closing the tabs interactively,
# as well as for the virtual event <<NotebookTabChanged>>
#
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }
bind $nb <<NotebookTabChanged>> { restoreSel %W }
```

```
proc populateMenu {nb data} {
    foreach {menu tabIdx} $data {}
    set tabCount [$nb index end]
    set prevIdx [expr {($tabIdx - 1) % $tabCount}]
    set nextIdx [expr {($tabIdx + 1) % $tabCount}]
    set widget [lindex [$nb tabs] $tabIdx]

    $menu add command -label "Move Tab Left"  -command \
        [list $nb insert $prevIdx $widget]
    $menu add command -label "Move Tab Right" -command \
        [list $nb insert $nextIdx $widget]
    $menu add separator
    $menu add command -label "Close Tab" -command \
        [list $nb forget $tabIdx]
}

proc restoreSel nb {
    set widget [$nb select]
    if {$widget ne "" && [$nb hastabattrib $widget "selRange"]} {
        set txt $widget.txt
        $txt tag remove sel 1.0 end
        $txt tag add sel {*}[$nb tabattrib $widget "selRange"]
    }
}

. . .

#
# Set the scrollednotebook's -height and -width options to the
# maximum requested height and width of all panes, respectively
#
after 50 [list resizeNb $nb]

proc resizeNb nb {
    $nb adjustsize
    foreach sa [$nb tabs] {
        $sa configure -yscrollbarmode dynamic                   ;# the
    default
    }
}
```

We add the `closetab` element to the tabs of the `My.TNotebook` style with the aid of the `scrollutil::addclosetab` command, and create a scrollednotebook widget of this style. Note that, while in this example we could have used the default ttk::notebook style `TNotebook` instead, working with different ttk::notebook styles is necessary in applications having both

notebooks with the `closetab` element in their tabs and notebooks without this element.

If the Tk version is at least 8.7 or the tksvg package can be loaded into the interpreter, then the image `fileImg` is created from the file `file.svg`, using the public variable `scrollutil::svgfmt`. Otherwise it is created from files whose names contain the display's DPI scaling percentage, given by the public variable `scrollutil::scalingpct`. The file `file100.gif` contains an image of size 16 x 16, the file `file125.gif` contains an image of size 20 x 20, the file `file150.gif` contains an image of size 24 x 24, and so on. In the first case a single `file.svg` file is needed and the image will be scaled automatically, according to the display's real scaling percentage, which can be greater than the maximum value `200` of the variable `scrollutil::scalingpct`. This benefit comes in handy especially when running AndroWish on a tablet or smartphone. In the second case a total number of 5 `file*.gif` (or `file*.png`) files are needed to make sure that the size of the image will correspond to the value of the variable `scrollutil::scalingpct`.

We set the `-yscrollbarmode` option of the scrollarea widgets that we `add` to the scrollednotebook to `static`, to make sure that the vertical scrollbars are visible when the widget's `adjustsize` subcommand is called to set its dimensions. After invoking this subcommand (towards the end of the script), we restore the default value `dynamic`.

Note that the state of the text widgets is set to `disabled`, but the user can select a text range and then copy the selection into the clipboard via `Control-c` (`Command-c` on Mac OS X/11+). When attempting to close a tab, the path name of the corresponding scrollarea widget is automatically passed to the `condCopySel` procedure, which was specified as the value of the `-forgetcommand` option. This procedure checks the text widget child of the scrollarea for the existence of a selection and enables the user to make sure that the selection will be copied to the clipboard before closing the tab, or to cancel the attempted operation on it. In a real-world application, the procedure specified as the value of the above-mentioned option would typically ask the user whether to save the changes before closing the tab, or to cancel the attempted operation on it.

When attempting to leave the currently selected window by selecting a different one, the path name of the scrollarea widget is automatically passed to the `saveSel` procedure, which was specified as the value of the `-leavecommand` option. This procedure saves the two-element list of text indices delimiting the selection in the text widget child of the scrollarea as the value of the `"selRange"` tab attribute, with the aid of the `tabattrib` subcommand, or resets that tab attribute via `unsettabattrib` if the text widget has no selection. Later, when this tab is selected again, the `restoreSel` procedure, bound to the virtual event `<<NotebookTabChanged>>`, checks the existence of the `"selRange"` tab attribute with the aid of the `hastabattrib` subcommand and restores the text widget selection specified by the value of this tab attribute, retrieved with the aid of the `tabattrib` subcommand. We chose this approach for saving and restoring the text selection instead of creating the text widgets with `-exportselection 0`in order to demostrate the usage of the subcommands related to tab attributes.

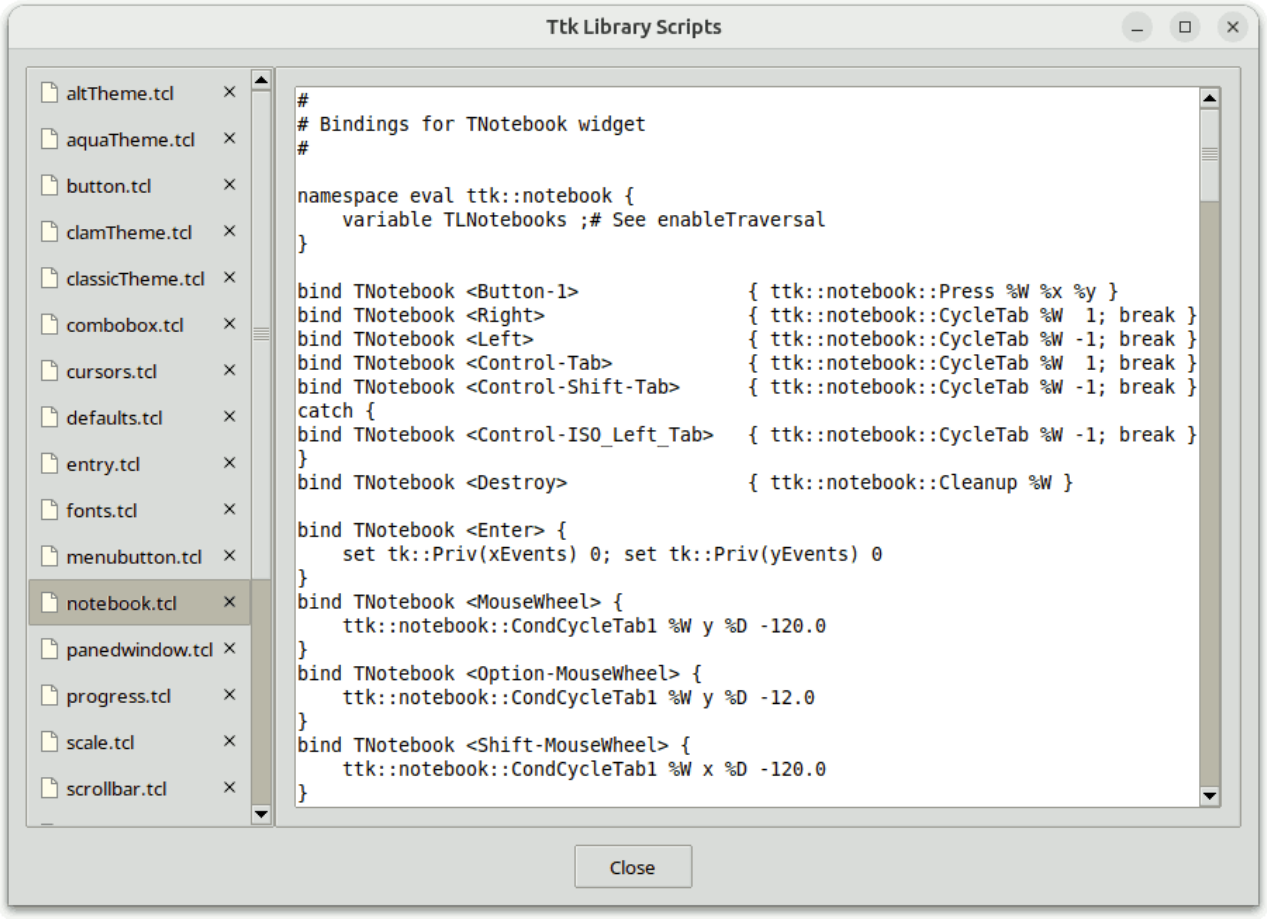
We also make use of the virtual event `<<MenuItemsRequested>>` to populate the pop-up menu shown when the user clicks a tab with mouse button 3.

When running the script, the scrollednotebook widget appears with the configured width and tabs displaying the unstripped file names. For comparison: The very similar script `TtkNotebookDemo.tcl` in the `demos` directory creates a ttk::notebook widget and populates

it in the same way as the script `ScrolledNotebookDemo.tcl` discussed above. Although the script also sets the notebook's `-width` option to a value corresponding to the requested width of the scrollarea widgets and the padding applied to the panes, the ttk::notebook appears with an exorbitantly large width and (on most displays) with squeezed tabs. If you resize it to a reasonable width, its tabs become so small that their texts are no longer readable.

A scrollutil::plainnotebook Demo

The script `PlainNotebookDemo.tcl` in the `demos` directory creates a scrollutil::plainnotebook widget and populates it with panes containing scrollareas that wrap text widgets displaying the contents of the Ttk library files. After that it provides pop-up menu items for moving the tabs upward/downward and closing them.



PlainNotebookDemo

The relevant code is nearly identical to the one shown in the previous example, except that here we make the tabs closable by using the `-closabletabs` plainnotebook option and the widget's width is set automatically to fit that of its panes:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Ttk Library Scripts"

#
```

```
# Create an image corresponding to the display's DPI scaling level
#
. . .

#
# Create a plainnotebook widget having closable (and, per default,
# movable) tabs and populate it with panes that contain scrolled
# text widgets displaying the contents of the Ttk library files
#
set f  [ttk::frame .f]
set nb [scrollutil::plainnotebook $f.nb -closabletabs 1 \
        -forgetcommand condCopySel -leavecommand saveSel]
. . .

proc condCopySel {nb widget} {
    . . .
}

proc saveSel {nb widget} {
    . . .
}

#
# Create bindings for moving and closing the tabs interactively,
# as well as for the virtual event <<NotebookTabChanged>>
#
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }
bind $nb <<NotebookTabChanged>> { restoreSel %W }

proc populateMenu {nb data} {
    . . .
}

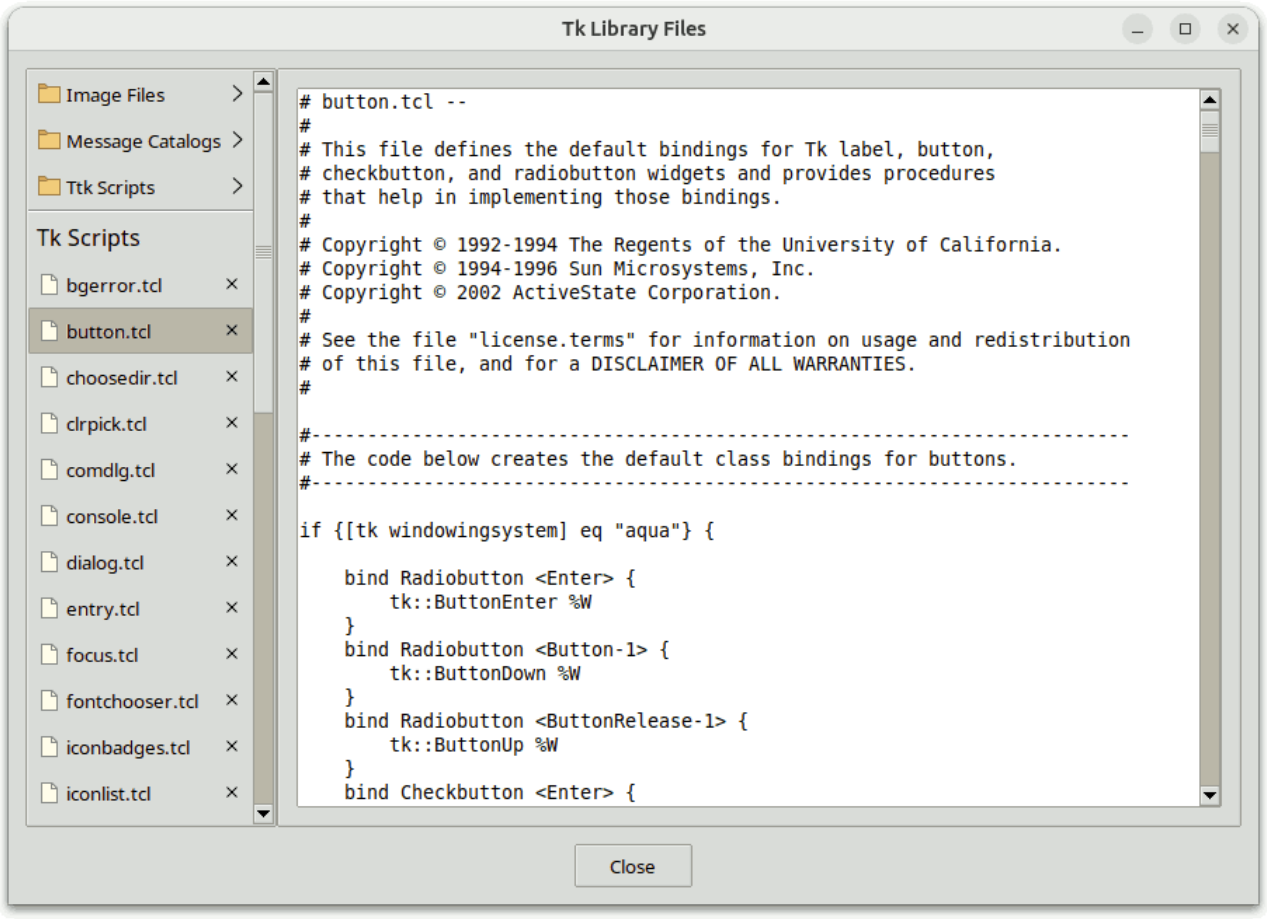
proc restoreSel nb {
    . . .
}

. . .
```

A scrollutil::pagesman Demo

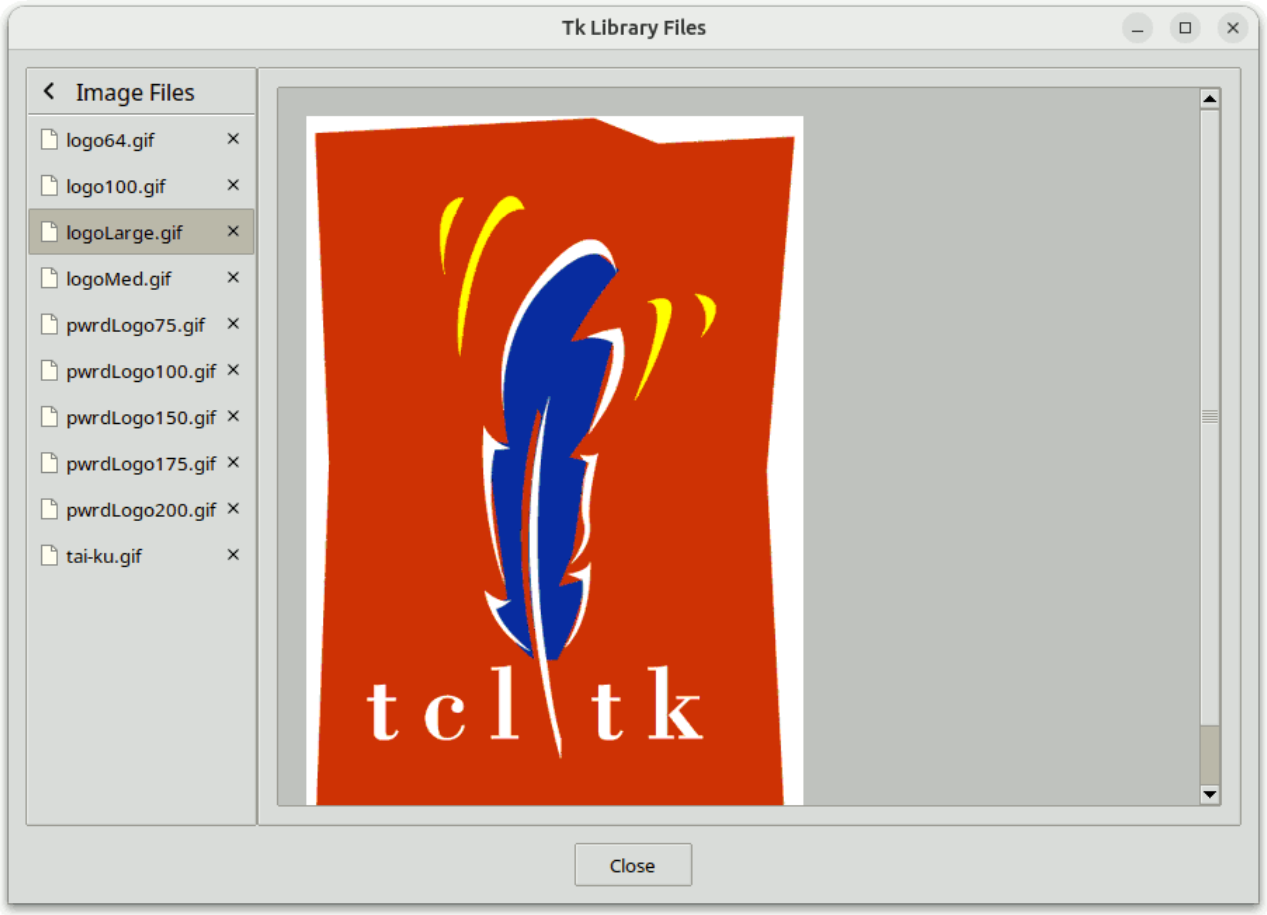
The script `PagesManDemo.tcl` in the `demos` directory demonstrates the use of the scrollutil::pagesman widget having scrollutil::plainnotebook widgets as pages. It creates a pagesman widget and adds four plainnotebook children as pages to it. The panes of the first page display the contents of the Tk library files, to be found in the directory `$tk_library`. The panes of the three other pages display the contents of the GIF image files, message catalogs, and Ttk scripts, situated in the subdirectories `images`, `msgs`, and (for Tk 8.5a5 and

later) `ttk`, respectively.



PagesManDemo

The user can switch from the first page to the three other ones with the aid of the three toolbuttons displaying a folder image and the `descend` style element. To switch back, he or she has to use the `ascend` toolbutton, shown in the top-left corner of the respective plainnotebook widget.



PagesManDemoImages

Here is the relevant code:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Tk Library Files"
wm withdraw .

#
# Create two images corresponding to the display's DPI scaling level
#
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set fmt $scrollutil::svgfmt
    image create photo fileImg   -file [file join $dir file.svg]
    -format $fmt
    image create photo folderImg -file [file join $dir folder.svg]
    -format $fmt
} else {
    set pct $scrollutil::scalingpct
    image create photo fileImg   -file [file join $dir file$pct.gif]
    \
```

```
        -format gif
    image create photo folderImg -file [file join $dir folder$pct.gif]
    \
        -format gif
}

#
# Populates a given plainnotebook widget with panes that display the
    contents
# of the files of the specified suffix within the current working
    directory
#
proc populateNotebook {nb sfx} {
    set currentTheme [styleutil::getCurrentTheme]
    set panePadding [expr {$currentTheme eq "aqua" ? 0 : "9p"}]
    foreach fileName [lsort -dictionary [glob *.$sfx]] {
        set baseName [string range $fileName 0 end-4]
        set sa [scrollutil::scrollarea $nb.sa_$baseName]
        $sa configure -xscrollbarmode none -yscrollbarmode static ;#
    temporarily
        if {$sfx eq "gif"} {
            set canv [canvas $sa.canv -background #c0c0c0]
            set img [image create photo -file $fileName -format gif]
            $canv create image 15p 15p -anchor nw -image $img
            bind $canv <Configure> [list setScrollRegion %W %w %h $img]
            scrollutil::addMouseWheelSupport $canv
            $sa setwidget $canv
        } else {
            $sa configure -lockinterval 10
            . . .
        }
        $nb add $sa -text $fileName -image fileImg -compound left \
                    -padding $panePadding
    }
}

#
# Create a pagesman widget
#
set f [ttk::frame .f]
set pm [scrollutil::pagesman $f.pm -leavecommand pmLeaveCmd]

#
# Add option database entries for the -closabletabs,
# -forgetcommand, and -leavecommand plainnotebook options
#
```

```
option add *Plainnotebook.closableTabs  1
option add *Plainnotebook.forgetCommand condCopySel
option add *Plainnotebook.leaveCommand  saveSel

#
# Create a plainnotebook child displaying the contents of the Tk
    library files
#
set nbTk [scrollutil::plainnotebook $pm.nbTk]
$pm add $nbTk
$nbTk addbutton 1 "Image Files"      folderImg
$nbTk addbutton 2 "Message Catalogs" folderImg
$nbTk addbutton 3 "Ttk Scripts"      folderImg
$nbTk addseparator
$nbTk addlabel "Tk Scripts"
cd $tk_library
populateNotebook $nbTk "tcl"

#
# Create a plainnotebook child displaying the images for the Tcl
    (Powered) Logo
#
set nbImgs [scrollutil::plainnotebook $pm.nbImgs -caller 0 -title
    "Image Files"]
$pm add $nbImgs
cd $tk_library/images
populateNotebook $nbImgs "gif"

#
# Create a plainnotebook child displaying the contents of the message
    catalogs
#
set nbMsgs [scrollutil::plainnotebook $pm.nbMsgs -caller 0 -title \
            "Message\nCatalogs"]
$pm add $nbMsgs
cd $tk_library/msgs
populateNotebook $nbMsgs "msg"

#
# Create a plainnotebook child displaying the contents of the Ttk
    library files
#
set nbTtk [scrollutil::plainnotebook $pm.nbTtk -caller 0 -title "Ttk
    Scripts"]
$pm add $nbTtk
### cd $tk_library/ttk          ;# works for Tk versions 8.5a5 and
```

```
    later only
cd [expr {[info exists ttk::library] ? $ttk::library : $tile::library}]
populateNotebook $nbTtk "tcl"

proc setScrollRegion {canv canvWidth canvHeight img} {
    #
    # Use a margin of 15p around the image
    #
    set pixels [expr {30 * [tk scaling]}]
    set rightX [expr {[image width  $img] + $pixels}]
    set lowerY [expr {[image height $img] + $pixels}]
    if {$rightX < $canvWidth}  { set rightX $canvWidth }
    if {$lowerY < $canvHeight} { set lowerY $canvHeight }
    $canv configure -scrollregion [list 0 0 $rightX $lowerY]
}

proc pmLeaveCmd {pm nb} {
    set widget [$nb select]
    if {$widget eq ""} {
        return 1
    } else {
        return [saveSel $nb $widget]
    }
}

proc condCopySel {nb widget} {
    global nbImgs
    if {$nb eq $nbImgs || [winfo class $widget] ne "Scrollarea"} {
        return 1
    }

    . . .
}

proc saveSel {nb widget} {
    global nbImgs
    if {$nb eq $nbImgs || [winfo class $widget] ne "Scrollarea"} {
        return 1
    }

    . . .
}


#
# For each plainnotebook create bindings for moving and closing its
```

```
    tabs
# interactively, as well as for the virtual event
    <<NotebookTabChanged>>
#
foreach nb [$pm pages] {
    bind $nb <<MenuItemsRequested>> { populateMenu %W %d }
    bind $nb <<NotebookTabChanged>> { restoreSel %W }
}

proc populateMenu {nb data} {
    . . .
}

proc restoreSel nb {
    . . .
}

. . .

wm deiconify .
after 400 [list configScrollareas $pm]

proc configScrollareas pm {
    wm geometry . [wm geometry .]
    foreach nb [$pm pages] {
        foreach w [$nb tabs] {
            if {[winfo class $w] eq "Scrollarea"} {
                $w configure -xscrollbarmode dynamic -yscrollbarmode
    dynamic
            }
        }
    }
}
```

The four pages of the pagesman widget, created by using the latter's `add` subcommand, will have the numerical indices `0`, ..., `3`. We pass the page indices `1`, `2`, and `3` to the first plainnotebook's `addbutton` subcommand, with which we create the three toolbuttons used to descend from this page to the other ones. When we create the three other plainnotebook widgets, we set their `-caller` option to `0`, which will make their ascend toolbutton visible and will make sure that invoking this button will switch back to the first plainnotebook.

The panes of all the four plainnotebook widgets are scrollareas that wrap text and canvas widgets, respectively. By temporarily configuring the scrollarea widgets with `-xscrollbarmode none -yscrollbarmode static`, we make place for up to 80 characters of up to 30 text lines, plus the vertical scrollbar. 400 milliseconds after mapping the main window we invoke the `configScrollareas` proceedure, which freezes the application window's geometry and resets the above-mentioned scrollarea options to their default value

dynamic. Due to the frozen geometry, the horizontal scrollbars, when shown, won't make the window higher.

Keywords: scrollutil

wheelEvent

Commands Related to Mouse Wheel and <TouchpadScroll> Event Handling

For Scrollutil Version 2.11

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- The scrollutil::addMouseWheelSupport Command
- The scrollutil::createWheelEventBindings Command
- The scrollutil::enableScrollingByWheel Command
- The scrollutil::disableScrollingByWheel Command
- The scrollutil::adaptWheelEventHandling Command
- The scrollutil::prepareScrollingByWheel Command
- The scrollutil::setFocusCheckWindow Command
- The scrollutil::focusCheckWindow Command

The scrollutil::addMouseWheelSupport Command

NAME

scrollutil::addMouseWheelSupport – Add mouse wheel event support to (ttk::)menubutton, (ttk::)scale, or scrollable widgets

SYNOPSIS

```
scrollutil::addMouseWheelSupport tag ?axes?
```

REQUIRED TK VERSION

8.4 or higher.

DESCRIPTION

Adds mouse wheel and <TouchpadScroll> event support to the widgets having the specified binding tag by creating bindings for the mouse wheel and <TouchpadScroll> events along the axes given by the optional axes argument, which must be xy (the default, meaning both the x and y axes), x (meaning the x axis only), or y (meaning the y axis only).

- If tag is the path name of a menubutton or ttk::menubutton widget, or the class name Menubutton or TMenubutton, then axes is ignored and the binding scripts created by this command will activate the cyclically next or previous entry of the menu associated with the widget given by the %W event field and invoke the action of that menu entry, provided that its

type is not `cascade` or `tearoff`.
- If `tag` is the path name of a scale or ttk::scale widget, or the class name `Scale` or `TScale`, then `axes` is ignored and the binding scripts created by this command will increment or decrement the value of the widget given by the `%W` event field and move its slider accordingly.
- Otherwise the binding scripts will scroll the window given by the `%W` event field with the aid of the `xview scroll number units`and/or `yview scroll number units`subcommands of the associated Tcl command, depending on `axes`.

REMARK 1: If `tag` is the path name of a window then the binding scripts created by this command are terminated by an invocation of the `break` command, in order to prevent the processing of the mouse wheel or `<TouchpadScroll>` events by further binding scripts. For example, if `tag` is the path name of a text widget then the terminating `break` command makes sure that the mouse wheel or `<TouchpadScroll>` events will not *additionally* be processed by the class bindings (associated with the binding tag `Text`), which in Tk 8.5 and later trigger a scrolling by *pixels*, unlike the bindings created by this command, which scroll the widget by *units* (i.e., lines and characters).

REMARK 2: The canvas widget has no built-in bindings, but you can use this command to add mouse wheel and `<TouchpadScroll>` event support to the widget class `Canvas` or individual canvas widgets.

REMARK 3: *In Tk versions prior to 8.7a4*, the set of mouse wheel events depends on the windowing system:

- The mouse wheel events along the vertical axis are `<MouseWheel>` on Windows, `<MouseWheel>` and `<Option-MouseWheel>` on Mac OS X/11+, and `<MouseWheel>`, `<Button-4>` and `<Button-5>` on X11 (where `<MouseWheel>` is not triggered by the X server, but can be produced using `event generate`).
- The mouse wheel events along the horizontal axis are `<Shift-MouseWheel>` on Windows, `<Shift-MouseWheel>` and `<Shift-Option-MouseWheel>` on Mac OS X/11+, and `<Shift-MouseWheel>`, `<Shift-Button-4>` and `<Shift-Button-5>` on X11 (where `<Shift-MouseWheel>` is not triggered by the X server, but can be produced using `event generate`). On X11, when using Tk 8.7a3, there are two more mouse wheel events along the horizontal axis: `<Button-6>` and `<Button-7>`, which are handled just like `<Shift-Button-4>` and `<Shift-Button-5>`, respectively. These events are commonly triggered by left/right tilting the scroll wheel of a mouse having one or two additional (thumb) buttons. (In Tk versions 8.6.x, with x >= 10, left/right tilting the scroll wheel of such a mouse gives rise to `<Shift-MouseWheel>` events on Windows and Mac OS X/11+, and to `<Shift-Button-4>` and `<Shift-Button-5>` events on X11.)

REMARK 4: *In Tk versions 8.7a4 and later*, the set of mouse wheel events is the same on all windowing systems:

- The mouse wheel events along the vertical axis are `<MouseWheel>` and `<Option-MouseWheel>`, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11.
- The mouse wheel events along the horizontal axis are `<Shift-MouseWheel>` and `<Shift-Option-MouseWheel>`, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11. In these Tk versions, left/right tilting

the scroll wheel of a mouse having one or two additional (thumb) buttons gives rise to <Shift-MouseWheel> (and <Shift-Option-MouseWheel>) events on all windowing systems.

KEYWORDS

mouse wheel event, binding, menubutton, scale widget, scrolling

---

The `scrollutil::createWheelEventBindings` Command

NAME

`scrollutil::createWheelEventBindings` – Create mouse wheel event bindings for toplevel widgets or the binding tag `all`

SYNOPSIS

```
scrollutil::createWheelEventBindings ?tag tag ...?
```

REQUIRED TK VERSION

8.4 or higher on X11 and Mac OS X/11+; 8.6b2 or later on Windows.

DESCRIPTION

Creates mouse wheel and <TouchpadScroll> event bindings for the specified binding tags such that if the widget under the pointer is (a descendant of) one of the scrollable widget containers having the same toplevel as the widget and registered via `scrollutil::enableScrollingByWheel` then these events will trigger a scrolling of that widget container. In case of several nested registered scrollable widget containers fulfilling these conditions the innermost one will be scrolled. Each `tag` argument must be `all` or the path name of an existing toplevel widget (including `.`).

REMARK: The reason for restricting the `tag` arguments to `all` and path names of existing toplevel widgets rather than supporting also tags like `"Scrollableframe"` (for scrollutil::scrollableframe), `"BwScrollableFrame"` (for BWidget ScrollableFrame) or `"Scrolledframe"` (for iwidgets::scrolledframe) is that the mouse wheel and <TouchpadScroll> events should trigger a scrolling of the widget container under the pointer not only if the widget under the pointer is the widget container itself but also if it is a descendant of the latter (recall that for each window, the path name of its nearest toplevel ancestor and the tag `all` are contained in the window's default list of binding tags).

KEYWORDS

mouse wheel event, binding, scrolling, scrollable widget container

---

The `scrollutil::enableScrollingByWheel` Command

NAME

`scrollutil::enableScrollingByWheel` – Register scrollable widget containers for scrolling by the mouse wheel

SYNOPSIS

```
scrollutil::enableScrollingByWheel ?scrollableWidgetContainer
    scrollableWidgetContainer ...?
```

REQUIRED TK VERSION

8.4 or higher on X11 and Mac OS X/11+; 8.6b2 or later on Windows.

DESCRIPTION

Adds the specified scrollable widget containers to the internal list of widget containers that are registered for scrolling by the mouse wheel and `<TouchpadScroll>` event bindings created by the `scrollutil::createWheelEventBindings` command.

REMARK 1: When a scrollable widget container whose path name was passed to this command gets destroyed, it is automatically removed from the above-mentioned internal list of registered widget containers.

REMARK 2: The `scrollutil::scrollableframe` command automatically invokes this command for the scrollableframe widget it creates.

KEYWORDS

mouse wheel event, binding, scrolling, scrollable widget container

---

The `scrollutil::disableScrollingByWheel` Command

NAME

`scrollutil::disableScrollingByWheel` – Deregister scrollable widget containers for scrolling by the mouse wheel

SYNOPSIS

```
scrollutil::disableScrollingByWheel ?scrollableWidgetContainer
    scrollableWidgetContainer ...?
```

REQUIRED TK VERSION

8.4 or higher on X11 and Mac OS X/11+; 8.6b2 or later on Windows.

DESCRIPTION

Removes the specified scrollable widget containers from the internal list of widget containers that are registered for scrolling by the mouse wheel and `<TouchpadScroll>` event bindings created by the `scrollutil::createWheelEventBindings` command.

KEYWORDS

mouse wheel event, binding, scrolling, scrollable widget container

---

The `scrollutil::adaptWheelEventHandling` Command

NAME

`scrollutil::adaptWheelEventHandling` – Adapt mouse wheel event handling

SYNOPSIS

```
scrollutil::adaptWheelEventHandling ?-ignorefocus? ?widget widget ...?
```

REQUIRED TK VERSION

8.4 or higher on X11 and Mac OS X/11+; 8.6b2 or later on Windows.

DESCRIPTION

If the optional `-ignorefocus` switch is *not* present then, for each `widget` argument, the

command performs the following actions:

- If `widget` is the path name of a tablelist widget then it sets the latter's `-xmousewheelwindow` and `-ymousewheelwindow` options to the path name of the containing toplevel window, provided that the Tablelist version is 6.4 or later (for earlier Tablelist versions the command silently ignores any tablelist widget passed to it as argument). As a result, a mouse wheel or `<TouchpadScroll>` event over the tablelist's body or edit window (more precisely, a mouse wheel or `<TouchpadScroll>` event sent to any component of the tablelist having the binding tag `TablelistBody` or `TablelistEdit`) will be handled as follows:
- Otherwise it locates the (first) binding tag that has mouse wheel or `<TouchpadScroll>` event bindings and is different from both the path name of the containing toplevel window and `all`. If the search for this tag was successful then the command modifies the widget's list of binding tags by prepending the tag `WheeleventRedir` and appending the tag `WheeleventBreak` to this binding tag. As a result, a mouse wheel or `<TouchpadScroll>` event sent to this widget will be handled as follows:
  - If the focus is inside the tablelist widget then the event will scroll the tablelist or its edit window and no further processing of the event will take place.
  - If the focus is outside the tablelist widget then no scrolling of the tablelist's body or edit window will happen. Instead, the event will be redirected to the containing toplevel window via `event generate`. This in turn will trigger a scrolling of the (innermost) widget container that is an ancestor of `widget` and has the same toplevel (if there is such a scrollable widget container), provided that the path name of the containing toplevel widget or the binding tag `all` was passed to the `scrollutil::createWheelEventBindings` command and this widget container was registered for scrolling via `scrollutil::enableScrollingByWheel`.
  - If the focus is on or inside the window `[focusCheckWindow widget]`then the event will be handled by the binding script associated with this tag and no further processing of the event will take place.
  - If the focus is outside the window `[focusCheckWindow widget]`then the event will be redirected to the containing toplevel window via `event generate`rather than being handled by the binding script associated with the above-mentioned tag. This in turn will trigger a scrolling of the (innermost) widget container that is an ancestor of `widget` and has the same toplevel (if there is such a scrollable widget container), provided that the path name of the containing toplevel widget or the binding tag `all` was passed to the `scrollutil::createWheelEventBindings` command and this widget container was registered for scrolling via `scrollutil::enableScrollingByWheel`.

If the `-ignorefocus` option is specified then, for each `widget` argument, the command performs the following actions:

- If `widget` is the path name of a tablelist widget then it resets the latter's `-xmousewheelwindow` and `-ymousewheelwindow` options to `""`, provided that the Tablelist version is 6.4 or later (for earlier Tablelist versions the command silently ignores any tablelist widget passed to it as argument). As a result, if the Tablelist version is 6.16 or later, a mouse wheel or `<TouchpadScroll>` event over the tablelist's body or edit window will scroll the tablelist or its edit window and no further processing of the event will take place.
- Otherwise it locates the (first) binding tag that has mouse wheel or `<TouchpadScroll>`

event bindings and is different from both the path name of the containing toplevel window and `all`. If the search for this tag was successful then the command modifies the widget's list of binding tags by appending the tag `WheeleventBreak` to this binding tag. As a result, a mouse wheel or `<TouchpadScroll>` event sent to this widget will be handled by the binding script associated with this tag and no further processing of the event will take place.

REMARK 1: This command is designed to be invoked for widgets that have mouse wheel or `<TouchpadScroll>` event bindings and are descendants of a scrollable widget container. The Tk and tile widgets having class bindings for mouse wheel and `<TouchpadScroll>` events are: listbox, text, Tk core scrollbar, ttk::scrollbar, ttk::combobox, ttk::spinbox, ttk::entry, ttk::notebook, and ttk::treeview. Examples of widgets with binding tags other than their class names that have mouse wheel and `<TouchpadScroll>` event bindings are ctext and tablelist widgets, as well as the entry components of mentry widgets of type `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"`, and `"IPv6Addr"` (for Mentry versions 3.2 and above). (In case of a ctext widget `w`, the binding tags mentioned in the description above refer to the widget's main text widget child `w.t` rather than to the widget itself.)

REMARK 2: As mentioned above, Tk core scrollbar and ttk::scrollbar widgets have class bindings for mouse wheel and `<TouchpadScroll>` events, hence this command should be invoked for them in case they are descendants of a scrollable widget container. Since this task can become tedious, Scrollutil makes sure that if you pass a widget to this command and that widget is embedded into a scrollarea via the latter's `setwidget` subcommand, then this command will automatically be invoked for the scrollbars of that scrollarea, too.

REMARK 3: When handling a mouse wheel or `<TouchpadScroll>` event sent to a Tk core or tile scrollbar whose path name was passed to this command without the `-ignorefocus` option, if the focus is on or inside the associated widget then the event will be processed by the scrollbar rather than being redirected to the containing toplevel, just as if the focus were on the scrollbar itself.

REMARK 4: The mouse wheel event class bindings for the Tk core scrollbar on Windows and X11 were added in Tk 8.6. Prior to this Tk version there were such bindings only for the windowing system `aqua` on the Macintosh. Scrollutil eliminates this discrepancy by automatically creating the `Scrollbar` class bindings for mouse wheel events on Windows and X11. Note also that in Tk versions prior to 8.7a4 the ttk::scrollbar widget had no *built-in* class bindings for mouse wheel events, but Scrollutil automatically creates the missing bindings by copying the mouse wheel event bindings of the widget class `Scrollbar` to the binding tag `TScrollbar`.

REMARK 5: The mouse wheel event class bindings for the ttk::entry widget were added in Tk 9.1a1, but Scrollutil automatically creates them for all Tk versions starting with 8.4.

REMARK 6: The mouse wheel event class bindings for the ttk::notebook widget were added in Tk 8.7a4, but Scrollutil automatically creates them for all Tk versions starting with 8.4. These bindings implement the navigation between the tabs of a ttk::notebook or scrollutil::scrollednotebook widget via the mouse wheel.

REMARK 7: The recommended invocation of this command is without the `-ignorefocus` option, because it significantly reduces the probability that the user will inadvertently scroll a window within a widget container containing many scrollable widgets.

KEYWORDS

mouse wheel event, binding, event handling, scrolling, scrollable widget container, focus

---

The `scrollutil::prepareScrollingByWheel` Command

NAME

`scrollutil::prepareScrollingByWheel` – Prepare scrollable widget containers for scrolling by the mouse wheel

SYNOPSIS

```
scrollutil::prepareScrollingByWheel ?-ignorefocus?
    ?scrollableWidgetContainer scrollableWidgetContainer ...?
```

REQUIRED TK VERSION

8.4 or higher on X11 and Mac OS X/11+; 8.6b2 or later on Windows.

DESCRIPTION

For each scrollable widget container specified as argument, this command prepares the scrolling with the mouse wheel and touchpad, by passing its descendants located in the same toplevel and having bindings for mouse wheel or `<TouchpadScroll>` events to `scrollutil::adaptWheelEventHandling`.

REMARK: If a scrollable widget container has many descendants then it is much safer to invoke this command than to manually pass a lot of widgets to `scrollutil::adaptWheelEventHandling`, because deciding which descendants to process can become tedious and error-prone.

KEYWORDS

mouse wheel event, binding, scrollable widget container

The `scrollutil::setFocusCheckWindow` Command

NAME

`scrollutil::setFocusCheckWindow` – Set the "focus check window"

SYNOPSIS

```
scrollutil::setFocusCheckWindow widget ?widget ...? otherWidget
```

REQUIRED TK VERSION

8.4 or higher.

DESCRIPTION

For each `widget` argument, the command sets the associated "focus check window" to `otherWidget`. This is the window to be used in the binding scripts for the tags `WheeleventRedir` and `Btn2EventRedir` instead of the widget when checking whether the focus is on/inside or outside that window. For each `widget` argument, `otherWidget` must be an ancestor of or identical to `widget`.

REMARK 1: When a widget whose path name was passed to this command as one of its `widget` arguments gets destroyed, the association between the widget and its "focus check window" is automatically removed.

REMARK 2: This command comes in handy if for some widgets you want to make the focus check within the binding scripts for the tags `WheeleventRedir` and `Btn2EventRedir` less restrictive. For example, if the widget under the pointer is an entry component of a mentry me of type `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"`, or `"IPv6Addr"` and the focus is on any of its siblings, then the mouse wheel and `<TouchpadScroll>` events sent to this entry should be handled by the entry widget itself rather than scrolling the widget container that is an ascendant of the mentry. You can achieve this by invoking

```
set entryList [$me entries]
eval scrollutil::adaptWheelEventHandling $entryList
eval scrollutil::setFocusCheckWindow     $entryList [list $me]
```

With Tcl/Tk 8.5 or above, you can use the more compact form

```
set entryList [$me entries]
scrollutil::adaptWheelEventHandling {*}$entryList
scrollutil::setFocusCheckWindow     {*}$entryList $me
```

REMARK 3: As a similar example, suppose that `ss` is a scrollsync widget that was populated via its `setwidgets` subcommand with child widgets. Then, if the widget under the pointer is one of these children and the focus is on any of the other children passed to that subcommand, then the mouse wheel and `<TouchpadScroll>` events sent to the child under the pointer should be handled by that child widget itself rather than scrolling the widget container that is an ascendant of the scrollsync. You can achieve this with the following code:

```
set widgetList [$ss widgets]
eval scrollutil::adaptWheelEventHandling $widgetList
eval scrollutil::setFocusCheckWindow     $widgetList [list $ss]
```

Again, with Tcl/Tk 8.5 or above, you can use the more compact form

```
set widgetList [$ss widgets]
scrollutil::adaptWheelEventHandling {*}$widgetList
scrollutil::setFocusCheckWindow     {*}$widgetList $ss
```

KEYWORDS

binding, focus, "focus check window"

The `scrollutil::focusCheckWindow` Command

NAME

`scrollutil::focusCheckWindow` – Query the "focus check window"

SYNOPSIS

```
scrollutil::focusCheckWindow widget
```

REQUIRED TK VERSION

8.4 or higher.

DESCRIPTION

Returns the path name of the "focus check window" associated with the `widget` argument. This is the window that is used in the binding scripts for the tags `WheeleventRedir` and `Btn2EventRedir` instead of the widget when checking whether the focus is on/inside or outside that window. If the command `scrollutil::setFocusCheckWindow` was not invoked for `widget` then the return value is `widget` itself.

KEYWORDS

binding, focus, "focus check window"

Keywords: wheelEvent

License

Scrolling utilities package Scrollutil 2.11 Copyright (c) 2019-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Index

I

index 3

P

pagesman 10
plainnotebook 21

S

scrollableframe 35
scrollarea 43
scrollednotebook 55
scrollsync 60
scrollutil 110

W

wheelEvent 118